

Ενσωμάτωση Γλωσσών Ειδικού Πεδίου με Γλώσσες Προγραμματισμού Γενικού Σκοπού

Βασίλειος Καρακίδας

Περίληψη

Οι Γλώσσες Ειδικού Πεδίου (Domain-specific Languages), γνωστές και ως μικρό-γλώσσες (micro languages), αποτελούν γλώσσες προγραμματισμού των οποίων ο σχεδιασμός εστιάζει στην αντιμετώπιση προβλημάτων ενός συγκεκριμένου πεδίου ενδιαφέροντος. Πολύ συχνά, οι γλώσσες ειδικού πεδίου χρησιμοποιούνται στη βελτιστοποίηση της διαδικασίας ανάπτυξης λογισμικού (Software Development Process). Γνωστές γλώσσες ειδικού πεδίου αποτελούν οι κανονικές εκφράσεις (regular expressions), η γλώσσα markdown και η SQL.

Στην αντίπερα όχθη, οι Γενικές Γλώσσες Προγραμματισμού (General-purpose Languages) έχουν ευρύτερες δυνατότητες. Αυτές παρέχουν όλες τις απαραίτητες δομές επεξεργασίας οι οποίες τις καθιστούν αποδοτικές σε ένα μεγαλύτερο εύρος προβλημάτων. Τυπικά παραδείγματα γενικών γλωσσών προγραμματισμού αποτελούν οι γλώσσες Java, C++ και Scala.

Οι σύγχρονες προσεγγίσεις στην ανάπτυξη λογισμικού επιβάλλουν οι γλώσσες ειδικού πεδίου να χρησιμοποιούνται παράλληλα με τις γενικές γλώσσες προγραμματισμού. Για παράδειγμα, η ενσωμάτωση της SQL σε διάφορες γενικές γλώσσες προγραμματισμού αποτέλεσε το πεδίο ενδιαφέροντος για πολλούς ερευνητές. Στην περίπτωση της Java, για την ενσωμάτωση της γλώσσας προγραμματισμού χρησιμοποιείται η βιβλιοθήκη εφαρμογής της Συνδετικότητας Βάσης Δεδομένων (JDBC, Java Database Connectivity). Ο προγραμματιστής εκτελεί το ερώτημα SQL στη βάση ως συμβολοσειρά (string). Μέσω αυτής της διαδικασίας ο μεταγλωττιστής της Java (Java compiler) αγνοεί εντελώς την SQL που περιέχεται στον κώδικα της Java, με αποτέλεσμα πολλά σφάλματα τύπων (type errors) ή συντακτικά σφάλματα (syntax errors) να εντοπίζονται τελικά κατά την εκτέλεση του προγράμματος. Ορισμένα από τα εν λόγω σφάλματα παραμένουν μη ανιχνεύσιμα ακόμα και μετά από παρατεταμένες δοκιμές κατά το στάδιο ανάπτυξης του λογισμικού.

Υπάρχουν βέβαια και πιο εξελιγμένες μέθοδοι, η πλειοψηφία των οποίων χρησιμοποιούν κοινές τεχνικές προερχόμενες από μεταγλωττιστές και μεταπρογραμματισμό. Όλες όμως αυτές οι προσεγγίσεις πάσχουν κατά κανόνα από μια ή παραπάνω από τις κάτωθι αδυναμίες:

- Αντιμετωπίζουν το πρόβλημα για ένα μόνο ζεύγος γλωσσών, μια γλώσσα ειδικού πεδίου και μια γλώσσα γενικού προγραμματισμού. Η ενσωμάτωση και άλλων γλωσσών ειδικού πεδίου είτε δεν υποστηρίζεται ή υποστηρίζεται μέσω πολύπλοκων συντακτικών μηχανισμών.
- Συστήνουν σύνθετες λύσεις οι οποίες επιβαρύνουν τις κύριες γλώσσες προγραμματισμού, υποχρεώνοντας τους προγραμματιστές να χρησιμοποιούν πολύπλοκες σημασιολογίες οι οποίες δεν είναι εναρμονισμένες με τη φιλοσοφία της εκάστοτε γλώσσας προγραμματισμού και των καλών πρακτικών προγραμματισμού.
- Δεν είναι πρακτικές, αφού εισαγάγουν νέες γλώσσες προγραμματισμού, οι οποίες ενώ αντιμετωπίζουν το πρόβλημα αποτελεσματικά, απαιτούν την επανεγγραφή πολύπλοκων βιβλιοθηκών εφαρμογής για την εκάστοτε γλώσσα ειδικού πεδίου. Οι προτεινόμενες γλώσσες προγραμματισμού αποτελούν την επέκταση μιας υπάρχουσας γλώσσας, ή μιας νέας γλώσσας με τελείως διαφορετική σύνταξη.

Μελετώντας τη σύγχρονη βιβλιογραφία, στο πεδίο της ενσωμάτωσης των γλωσσών ειδικού πεδίου με γενικές γλώσσες προγραμματισμού, γίνεται σαφές ότι οι ερευνητικές προσεγγίσεις μοιράζονται κοινές τεχνικές σχεδιασμού. Αυτές είναι:

Ενότητες Κώδικα (Language Code Blocks) Σύμφωνα με σύγχρονες ερευνητικές προσεγγίσεις, μια επιτυχημένη μέθοδος ενσωμάτωσης θα πρέπει να παρέχει τη δυνατότητα να απομονώνονται συντακτικά τον κώδικα της γλώσσας ειδικού πεδίου από τη γενική γλώσσα προγραμματισμού. Η προσθήκη ειδικών τελεστών προσανατολισμένων στη γλώσσα ειδικού πεδίου θα πρέπει να αποφεύγεται, εκτός αν η ύπαρξή τους είναι απαραίτητη για την εξειδικευμένη

υποστήριξη της γλώσσας ειδικού πεδίου. Η ενσωμάτωση της Perl με κανονικές εκφράσεις (regular expressions) είναι το τέλειο παράδειγμα προς αποφυγήν. Η Perl εισαγάγει ειδικούς τελεστές οι οποίοι παρέχουν έναν πολύ αποτελεσματικό τρόπο χρήσης κανονικών εκφράσεων. Το πρόβλημα που προκύπτει είναι η αδυναμία ενσωμάτωσης άλλων γλωσσών ειδικού πεδίου χωρίς την προσθήκη πολύπλοκου συντακτικού στη Perl. Αυτός είναι και ο λόγος για τον οποίο η Perl ενσωματώνει άλλες γλώσσες ειδικού πεδίου, όπως κάνει η sql χρησιμοποιώντας μια απλή βιβλιοθήκη εφαρμογής. Οι ενότητες κώδικα αποτελούν την τέλεια λύση για να ξεπεραστεί το πρόβλημα, καθώς επιτρέπουν τη χρήση της γλώσσας ειδικού πεδίου στην κανονική της μορφή χωρίς να επιβαρύνουν το συντακτικό της γενικής γλώσσας προγραμματισμού.

Εξειδικευμένοι Τύποι Γλωσσών Ειδικού Πεδίου (Special DSL Types) Για τη διευκόλυνση της διαδικασίας μεταγλώττισης καλό είναι να χρησιμοποιούνται εξειδικευμένοι τύποι. Αυτοί οι τύποι παρέχουν πληροφορίες στο μεταγλωττιστή οι οποίες ενεργοποιούν συγκεκριμένες συμπεριφορές κατά τη διάρκεια της μεταγλώττισης. Οι εν λόγω τύποι θα πρέπει να χρησιμοποιούνται κατά περίπτωση στο εκάστοτε πρόγραμμα, χωρίς να επιβαρύνουν συνεχώς τη σύνταξη της γενικής γλώσσας προγραμματισμού.

Ρυθμιστές Τύπων (Configuration Hooks) Θα πρέπει να οριστούν ρυθμιστές τύπων κατά το χρόνο μεταγλώττισης, οι οποίοι ενημερώνουν τον μεταγλωττιστή για τις παραμέτρους της γλώσσας ειδικού πεδίου. Η Java υποστηρίζει αυτού του είδους ρύθμιση τύπων μέσω του μηχανισμού των επισημάνσεων (annotations) κατά τη φάση της μεταγλώττισης. Παρόμοιο μηχανισμό διαθέτει και η Python, μέσω των στοιχείων decorators. Όλες σχεδόν οι κύριες γλώσσες προγραμματισμού διαθέτουν παρόμοιους μηχανισμούς που μπορούν να χρησιμοποιηθούν για την επιτυχή ενσωμάτωση μιας γλώσσας ειδικού πεδίου.

Αντιστοίχιση τύπων (Type Mapping) Όλοι οι τύποι δεδομένων στη γλώσσα ειδικού πεδίου, θα πρέπει να διαθέτουν ένα μηχανισμό, ώστε να αλληλεπιδρούν με τη γενική γλώσσα προγραμματισμού. Ο μεταγλωττιστής θα πρέπει να κατανοεί τη συμβατότητα αυτών των τύπων και να ενεργοποιεί προειδοποιήσεις και σφάλματα κατά το χρόνο μεταγλώττισης. Τυπικά παραδείγματα αντιστοίχισης τύπων έχουν ήδη οριστεί σε διάφορες περιπτώσεις, πχ. στην προδιαγραφή της Συνδετικότητας Βάσεων Δεδομένων (JDBC), η οποία αντιστοιχίζει Java τύπους με τους συμβατούς τύπους της SQL.

Για την στοιχειοθέτηση της έρευνας μας και μέσω της μελέτης των σύγχρονων ερευνητικών τάσεων, οδηγήθηκαμε στον ορισμό ενός συνόλου λειτουργικών απαιτήσεων.

Έλεγχος Συντακτικού Γλώσσας Ειδικού Πεδίου (DSL Syntax Validation) Η γλώσσα ειδικού πεδίου θα πρέπει να ελέγχεται συντακτικά κατά τη διάρκεια της μεταγλώττισης. Στην πλειοψηφία των μελετών περίπτωσης που έχουν παρουσιαστεί, αυτό συνέβαινε μόνο για μια γλώσσα ειδικού πεδίου. Κατά συνέπεια, κάθε αποτελεσματική ολοκλήρωση ενσωμάτωσης μιας γλώσσας ειδικού πεδίου θα πρέπει να υλοποιεί αυτή την λειτουργική απαίτηση.

Στατικός έλεγχος τύπων (Static Typing) Η ενσωμάτωση της γλώσσας ειδικού πεδίου κατά τη διάρκεια μεταγλώττισης θα πρέπει να διασφαλίζει τον έλεγχο τύπων καθώς και των τιμών που ανταλλάσσονται στο πρόγραμμα μεταξύ της γενικής γλώσσας προγραμματισμού και της γλώσσας ειδικού πεδίου.

Επεκτασιμότητα (Extensibility) Κοινό πρόβλημα της εν εξελίξεως έρευνας και των συστημάτων που εφαρμόζονται στη βιομηχανία, είναι η έλλειψη επεκτασιμότητας. Σχεδόν όλες οι εφαρμογές υποστηρίζουν μια ή το πολύ δυο γλώσσες ειδικού πεδίου με προσαρμοσμένο συντακτικό.

Οι εφαρμογές αυτές δεν είναι σε θέση να υποστηρίξουν αποτελεσματικά τη ενσωμάτωση οποιασδήποτε άλλης γλώσσας. Επίσης, τα συστήματα που υποστηρίζουν πληθώρα γλωσσών ειδικού πεδίου είναι στενά συνδεδεμένα με μια συγκεκριμένη γενική γλώσσα προγραμματισμού.

Εκτός από τις προαναφερθείσες λειτουργικές απαιτήσεις, ορίζεται και μια σειρά από μη λειτουργικές απαιτήσεις. Αυτές οι απαιτήσεις μπορούν να εξαχθούν από ήδη υπάρχουσες εργασίες.

Σημασιολογική πολυπλοκότητα (Semantic Complexity) Η προτεινόμενη λύση δε θα πρέπει να επιβαρύνει τη σημασιολογική πολυπλοκότητα της γενικής γλώσσας προγραμματισμού. Στην πράξη, δεν είναι δυνατόν να προσθέτεις κάποια λειτουργία χωρίς να χρησιμοποιήσεις κάποιας μορφής επέκταση του συντακτικού της γλώσσας. Από την άλλη, οι προτεινόμενες επεκτάσεις θα πρέπει να ακολουθούν κάποια λογική και να είναι όσο το δυνατόν πιο απλές.

Διατήρηση μοντέλων ειδικού πεδίου (Domain Model Maintenance) Η εκάστοτε γλώσσα ειδικού πεδίου θα πρέπει να διατηρεί το συντακτικό της. Ένα από τα κύρια πλεονεκτήματα των γλωσσών ειδικού πεδίου, κατά την ανάπτυξη λογισμικού, είναι το ότι προσφέρουν μια οπτική των μοντέλων του ειδικού πεδίου επιτρέποντας την αποτελεσματική ανάπτυξη εφαρμογών σε αυτά.

Επαναχρησιμοποίηση (Re-usability) Η προτεινόμενη λύση θα πρέπει να ενθαρρύνει την επαναχρησιμοποίηση. Αυτό μπορεί να επιτευχθεί με δύο τρόπους: πρώτον με την επιβολή της επαναχρησιμοποίησης του κώδικα της γλώσσας ειδικού πεδίου, αναγκάζοντας τον προγραμματιστή να τον οργανώσει σε αφαιρετικές δομές της γενικής γλώσσας προγραμματισμού και δεύτερον προάγοντας τη χρήση των υπάρχοντων βιβλιοθηκών εφαρμογής.

Βελτιστοποιήσεις Ειδικού Πεδίου (Domain Specific Optimisations) Η μεταγλώττιση και η ανάλυση της γλώσσας ειδικού πεδίου αποτελεί ιδανική ευκαιρία για την εισαγωγή βελτιστοποιήσεων παραγόμενου κώδικα.

Η παρούσα διδακτορική διατριβή εισαγάγει την J%(j-mod), μια επέκταση της Java για την αποδοτική υποστήριξη γλωσσών ειδικού πεδίου. Η επέκταση J% παρέχει έναν τρόπο ενσωμάτωσης γλωσσών ειδικού πεδίου στην Java με τη μορφή τμημάτων (modules) στον μεταγλωττιστή. Κατά την συγγραφή της παρούσης, το J% υποστηρίζει τη γλώσσα SQL και τις κανονικές εκφράσεις. Εκμεταλεύεται τις ήδη υπάρχουσες προγραμματιστικές διεπαφές της Java, όπως για παράδειγμα τη βιβλιοθήκη της Συνδετικότητας Βάσης Δεδομένων (JDBC). Εισαγάγει δε, τα κάτωθι χαρακτηριστικά:

Τμηματικός Σχεδιασμός (Modular Design) Οι προγραμματιστές μπορούν να σχεδιάσουν το δικό τους τμήμα μεταγλώττισης το οποίο, είναι σε θέση να υποστηρίξει οποιαδήποτε γλώσσα ειδικού πεδίου χωρίς περιορισμούς, με τη χρήση μιας καλά καθορισμένης προγραμματιστικής διεπαφής τμημάτων του μεταγλωττιστή.

Στατικός Έλεγχος Τύπων (Static Typing) Στη φάση μεταγλώττισης πραγματοποιείται στατικός έλεγχος τύπων. Παρέχεται μηχανισμός αντιστοίχισης τύπων μεταξύ της γλώσσας ειδικού πεδίου και της γλώσσας γενικού προγραμματισμού, ο οποίος ενεργοποιείται στη φάση του στατικού ελέγχου τύπων.

Έλεγχος Συντακτικού Γλώσσας Ειδικού Πεδίου (DSL Syntax Validation) Η γλώσσα ειδικού πεδίου ελέγχεται κατά τη διάρκεια της μεταγλώττισης. Όλα τα σφάλματα αναφέρονται ως σφάλματα μεταγλώττισης.

Συντακτικό Γλώσσας Ειδικού Πεδίου (DSL Syntax) Κάθε φιλοξενούμενη γλώσσα ειδικού πεδίου διατηρεί το συντακτικό της. Απαιτείται μόνο μια μικρή προσθήκη στη γραμματική της γλώσσας ειδικού πεδίου έτσι ώστε να υποστηρίξει την αντιστοίχιση τύπων με την J%.

Pragmatic Σε πολλές περιπτώσεις η αντιμετώπιση του συνδυασμού γλωσσών προγραμματισμού από διάφορες επεκτάσεις που χρησιμοποιούν μετά-προγραμματισμό για την επέκταση τους, συντελείται εις βάρος της απλότητας και του συντακτικού της γενικής γλώσσας προγραμματισμού. Σε άλλες προσεγγίσεις, ορίζεται μια νέα γλώσσα προγραμματισμού η οποία μεταγλωττίζει τις γλώσσες ειδικού πεδίου σε κώδικα της γενικής γλώσσας προγραμματισμού. Η J% αποτελεί μια ρεαλιστική προσέγγιση, η οποία επεκτείνει μια κύρια γλώσσα προγραμματισμού, την Java, για να επιτύχει με τρόπο εκλεπτυσμένο και αποτελεσματικό την ενσωμάτωση εφαρμογών της βιβλιοθήκης εφαρμογής της υπάρχουσας γλώσσας ειδικού πεδίου.

Βελτιστοποιήσεις Ειδικού Πεδίου (Domain-specific optimisations) Η J% παρέχει όλες τις απαραίτητες διευκολύνσεις για την εκτέλεση των βελτιστοποιήσεων της γλώσσας ειδικού πεδίου κατά τη φάση μεταγλώττισης. Σε αυτήν την εργασία παρουσιάζονται δυο μελέτες περίπτωσης: μια μελέτη για τη βελτιστοποίηση της απόδοσης των κανονικών εκφράσεων με τη χρήση της βιβλιοθήκης εφαρμογής FIRE/J και μια αυτόματης χρήσης έτοιμων δηλώσεων (prepared statements) της SQL. Το μοντέλο κανονικών εκφράσεων, το οποίο περιγράφεται αναλυτικά στην παράγραφο 4.4, παράγει κώδικα, κατά τη διάρκεια μεταγλώττισης, για κάθε έκφραση χρησιμοποιώντας τη βιβλιοθήκη FIRE/J. Οι βελτιστοποιήσεις κατά το χρόνο μετα-γλώττισης δεν αποτελούν καινοτόμα ιδέα, μιας και ο μετά-προγραμματισμός προσφέρει τέτοιους μηχανισμούς (Ενότητα 2.5). Παράλληλα με τις τεχνικές βελτιστοποιήσεις κατά το χρόνο της μεταγλώττισης, θα πρέπει να παρέχεται και η δυνατότητα εφαρμογής βελτιστοποιήσεων που αφορούν στην ασφάλεια λογισμικού. Ο κώδικας της γλώσσας ειδικού πεδίου θα πρέπει να αναλύεται και να ενισχύεται έναντι πιθανών κινδύνων ασφάλειας. Οι εν λόγω κίνδυνοι, μεταξύ άλλων, περιλαμβάνουν προβλήματα κατά την επικύρωση των δεδομένων εισόδου (input validation) και επιθέσεις κακόβουλου κώδικα. Στο προτεινόμενο σύστημα που παρουσιάζεται, οι βελτιστοποιήσεις της ασφάλειας της γλώσσας ειδικού πεδίου παρέχονται για το μοντέλο της SQL. Η παρούσα εφαρμογή αναπαράγει αυτόματα κώδικα ο οποίος χρησιμοποιεί έτοιμες δηλώσεις (Prepared Statements), που παρέχουν εξ' αρχής προστασία από τυχόν επιθέσεις ένεσης κώδικα SQL.

Integrating Domain-Specific Languages with General-Purpose Languages

Vassilios Karakoidas

Department of Management Science and Technology,
Athens University of Economics and Business,
76 Patission Street, 10434, Athens, Greece.
Email: bkarak@aueb.gr

Thesis Advisor: Professor Diomidis Spinellis

Last Update: *June 29, 2015*

Contents

List of Figures	vii
List of Tables	ix
Preface	1
Acknowledgements	3
Summary	5
1 Introduction	9
1.1 Motivation	9
1.2 On the Use of Domain-Specific Languages	10
1.2.1 Experiment Methodology	12
1.2.2 Results	13
1.3 Thesis Outline	13
1.4 Research Methodology	15
1.5 Contribution	15
1.6 J% in Action	17
1.7 Typographical Conventions and Diagrams	19
2 Current Research on Domain-Specific Languages	23
2.1 General-purpose Programming Languages	23
2.1.1 Definitions	23
2.1.2 Programming Languages Categorisation	24
2.2 Domain-Specific Languages	25
2.2.1 Definitions	26
2.2.2 Software Development Process and Domain-Specific Language Usage: Advantages and Disadvantages	26
2.3 Design and Implementation of DSLs	28
2.3.1 Classification of DSL Patterns	29
2.3.2 Other Approaches	32
2.4 Case Studies of DSL Integrations	32
2.4.1 Design: Language Exploitation (extension)	32
2.4.2 Design: Language Exploitation (specialization)	35
2.4.3 Implementation: Extensible Compiler/Interpreter	35
2.4.4 Implementation: Embedding	35
2.4.5 Implementation: Preprocessor	37
2.5 Metaprogramming and Language Transformation Tools	37
2.5.1 Program Transformation Systems	38

2.5.2	Metaborg, Converge and Other Approaches	38
2.5.3	The C Preprocessor	39
2.5.4	C++ and Template Metaprogramming	40
2.5.5	Java Annotations & Python Decorators	41
2.5.6	Extensible Compilers	41
2.5.7	Language Boxes	42
2.6	Discussion	42
3	Design Concepts of the J% Programming Language	45
3.1	Design Strategy	45
3.1.1	Requirements	45
3.1.2	Key Elements for Efficient Language Mixing	46
3.2	The J% Language	48
3.2.1	Basic Terminology	48
3.2.2	Overview	49
3.2.3	External Types	50
3.2.4	External Declaration	50
3.2.5	Package Declaration and Importing of External Types	51
3.2.6	External Type Modifiers	52
3.2.7	External Body	52
3.2.8	Configuration Types	52
3.2.9	External References	53
3.3	The J% Type System	54
3.3.1	Why do we need a Type System	55
3.3.2	The Java Type System	56
3.3.3	J% Type System Hierarchy	57
3.3.4	Type Mapping	60
3.3.5	The J% Approach	60
3.4	Compilation Process	61
4	Implementation of the J% Compiler System	63
4.1	The J% Compiler	63
4.1.1	Compiler Implementation	67
4.1.2	External Modules	69
4.1.3	External References	72
4.1.4	The Type Mapping Mechanism	73
4.1.5	Code Generation	75
4.2	The J% Compiler Interface	76
4.2.1	The CompilerContext Class	77
4.2.2	The ExternalModule Class	78
4.2.3	Using the Compiler's API	78
4.3	Extending J%	79
4.3.1	Create the Basic Module	79
4.3.2	Define the External Base Types	81
4.3.3	Define the Configuration Types	81
4.3.4	Type Mapping	82
4.4	Regular Expression Module	83
4.4.1	Module Architecture	85
4.4.2	External Type and Configuration Parameters	87
4.4.3	Code Generation	88

4.4.4	Domain-Specific Optimisation	88
4.5	SQL Module	94
4.5.1	Design & Features	94
4.5.2	Module Architecture	95
4.5.3	Configuration Parameters	97
4.5.4	SQL Types and Type Conversions	97
4.5.5	Code Generation	97
4.5.6	Domain-Specific Optimisation	100
4.6	Cassandra CQL Module	100
4.6.1	Design and Features	100
4.6.2	Module Architecture	101
4.6.3	Configuration Parameters	102
4.6.4	CQL Types and Type Conversions	102
4.6.5	Code Generation	103
5	Evaluation	105
5.1	Evaluating Against the Design Requirements	105
5.1.1	Functional Requirements	105
5.1.2	Non-functional Requirements	106
5.2	Empirical Evaluation	107
5.2.1	Porting Applications to J%	107
5.2.2	Compile-time Error Detection	108
5.2.3	Measuring the Impact of J% on Size and Product Metrics	110
5.2.4	Compilation Overhead	113
5.3	Conclusions	117
6	Conclusions, Open Issues and Future Research Directions	119
6.1	Summary	119
6.2	Contribution	120
6.3	Future Work	122
A	Publication List	125
A.1	Book Chapters	125
A.2	Journal Articles	125
A.3	Conference Articles	125
A.4	Magazines	126
B	FIRE/J Benchmarking	127
B.1	Evaluation Criteria	127
B.2	Benchmark Participants	128
B.3	Real-World Regular Expressions	128
B.4	Hand Crafted Regular Expressions	132
B.4.1	Calculating Compilation Overhead	132
C	Cross Language Metric Tool	135
C.1	Cross-Language Metric Tool	135
C.2	Result DTD	135
D	Domain-Specific Languages Bibliography	139

E	Summary of the J% Grammar	141
F	Code Generation Templates	153
F.1	Regular Expression Module	153
F.2	SQLModule	154
	Acronyms	155
	Bibliography	159

List of Figures

1.1	A UML class diagram depicting the heirarchy of the Maven repository	12
1.2	DSL language usage in Java projects	13
1.3	Number of DSLs' used in Java projects	14
1.4	Engineering research process	16
1.5	Scientific areas according to reference [176]	16
1.6	Published research work categorised (Appendix A)	17
1.7	UML class diagram notation [240]	20
1.8	UML activity diagram notation [240]	21
2.1	Language definitions hierarchy	24
2.2	Programming languages categorisation	25
2.3	Advantages and disadvantages of DSL usage	27
2.4	DSL Design and Implementation	29
2.5	DSL Patterns as in reference [187]	30
2.6	Activity Diagram for Implementation: Embedding Pattern	30
2.7	Activity Diagram for Implementation: Preprocessor Pattern	31
2.8	DSL embedding approaches according to Tratt [252]	32
2.9	Popular DSL integrations categorised by design pattern	33
3.1	DSL embedding requirements correlated with the basic aspects of ISO/IEC 9126 quality model	47
3.2	Simplified J% grammar	50
3.3	A typical external type hierarchy	51
3.4	Syntax diagram for TypeDeclaration rule	51
3.5	Syntax diagram for ExternalDeclaration rule	51
3.6	External References Syntax	53
3.7	Overview of the Java Type System	57
3.8	Internal Type Signature in Java	57
3.9	Overview of the J% type system extensions	59
3.10	How type mapping works	61
3.11	Pipeline of the compilation process	62
4.1	The J% Compiler as pipe and filter architecture [234]	64
4.2	Overview of compilation process	65
4.3	The compilation process per file	66
4.4	The J% Platform	66
4.5	J% compiler architecture	68
4.6	J% standard modules export (--module-list option)	70
4.7	J% module package class diagram	70
4.8	External modules configuration hierarchy	72

4.9	Example of configuration hierarchies	73
4.10	Class diagram of the compiler API	77
4.11	Regular expression module class diagram	86
4.12	Regular expression module configuration hierarchy	88
4.13	Architecture of the Fast Implementation of Regular Expressions (FIRE/J) Engine . .	89
4.14	SQL module architecture	96
4.15	A typical configuration type hierarchy	98
4.16	Cassandra supports linear scaling.	101
4.17	UML class diagram for the CQL module	102
5.1	Function and non-functional requirements	105
5.2	Metrics Categorisation	111
6.1	Research on DSL-GPL integration	120
6.2	Optimising the example's code structure with the proposed feature	124
B.1	Matching Performance for Java engines - Execution time / Size (S)	130
B.2	Matching Performance for non-Java engines - Execution Time / Size (S)	131
B.3	Matching Performance for Java engines - Execution Time / Length (L)	131
B.4	Matching Performance for non-Java engines - Execution Time / Length (L)	132

List of Tables

1.1	Popular built-in DSLs for the Java platform	11
1.2	The selected Maven projects' size metrics	11
1.3	List of selected DSL application libraries	12
1.4	Popular DSL Usage Combinations	14
2.1	Dominant design and implementation patterns for DSL integration	31
2.2	Multiparadigm Languages Implementations	43
2.3	DSL integrations score according to syntax checking, static typing and complexity	44
3.1	J% terms compared to key concepts	49
3.2	External References Examples	54
3.3	Types System Properties on Popular Programming Languages [204]	56
3.4	Interpretation of base type characters	58
3.5	Examples of Java Type Signatures	58
3.6	jsonsimple list of mapped types	60
4.1	J% compiler size metrics	67
4.2	J% compiler standard options	67
4.3	List of valid configuration field types and values	71
4.4	Method list of the Compiler class	76
4.5	Methods list for the CompilerContext class	78
4.6	Methods list for the ExternalModule class	78
4.7	Time & Space complexities for NFA and DFA automatons	84
4.8	Typical regular expressions operators	85
4.9	Examples of regular expression	85
4.10	Regular expression module size metrics	86
4.11	Regex type public methods	87
4.12	Regex module configuration parameters	87
4.13	SQL Module Size Metrics	96
4.14	Configuration Parameters	97
4.15	Configuration Parameters Requirements	97
4.16	Type mapping definitions for the MySQL database dialect	98
4.17	Type mapping. Dominant types are marked in italic (CQL module)	103
4.18	Type mapping for composite types (CQL module)	103
5.1	The size metrics that were used in the evaluation process.	112
5.2	Size metrics for JCrontab, Address Book, ExamJ, and RuBiS.	113
5.3	LoC differentiation per file.	114
5.4	Size metrics for the participants.	115
5.5	Hardware and software configuration.	116

5.6	Basic descriptive statistic measures for compilation time (<i>ms</i>).	116
5.7	Basic descriptive statistic measures for compiler overhead (<i>ms</i>).	116
B.1	Average Operator Occurrence	129
B.2	Participants Success Rate (Total 849)	129
B.3	Statistical Measures	129
B.4	Matching Performance Statistical Measures (<i>ms</i>)	130
B.5	Matching Performance (μ s)	133
B.6	Compilation Performance (μ s)	133
C.1	Built-in Supported Metrics	136
C.2	Built-in Metric Groups	137

Preface

"The Babel fish," said The Hitch Hiker's Guide to the Galaxy quietly, "is small, yellow and leech-like, and probably the oddest thing in the Universe. It feeds on brainwave energy not from its carrier but from those around it. It absorbs all unconscious mental frequencies from this brainwave energy to nourish itself with. It then excretes into the mind of its carrier a telepathic matrix formed by combining the conscious thought frequencies with nerve signals picked up from the speech centres of the brain which has supplied them. The practical upshot of all this is that if you stick a Babel fish in your ear you can instantly understand anything said to you in any form of language. The speech patterns you actually hear decode the brainwave matrix which has been fed into your mind by your Babel fish. Now it is such a bizarrely improbable coincidence that anything so mindbogglingly useful could have evolved purely by chance that some thinkers have chosen to see it as the final and clinching proof of the non-existence of God."

--- Hitchhicker's Guide to the Galaxy

Acknowledgements

Gratitude unlocks the fullness of life. It turns what we have into enough, and more. It turns denial into acceptance, chaos to order, confusion to clarity. It can turn a meal into a feast, a house into a home, a stranger into a friend.

– Melody Beattie (author of Codependent No More)

A long time ago (not very long actually), in a galaxy **not** so far away (actually not at all) I began this great voyage that is called *PhD*. Now it seems that this voyage is ending (even for me, also known as the *eternal revisionist*) and it is time to put some things in perspective; get a real job and finally make a meaningful contribution to society.

So, my first contribution is this thesis, which it's mine, since it has my name on it, but not only, since many people have also worked for it (and I take all the credit, nice huh?). The least I can do for them, for all their time and patience is to mention them here, as the eternal token of my gratitude and I say eternal because I will submit this to a central archive, I will post it to the cloud and finally I will create an account in *longaccess* and store it there.

So let's take them one by one; the first two are (easily to guess) my father and my mother, who never questioned my judgment and supported me during these ten (10) long years. They were there for me all my life and provided me with a healthy and stable environment to grow and for that I could never thank them enough.

Next in line is my advisor, Professor Diomidis Spinellis, who believed in my skills and in the potential to complete this work. But this worked both ways, since I know that I have helped him also, by providing him the stimulus to explore new frontiers in terms of patience and self restraint. The truth is, that he was indeed a real mentor, that helped me understand the true meaning of the whole PhD process, life, universe and everything.

Panagiotis Louridas also provided me with valuable comments and by doing so, he shaped chapters 1, 2, and 3 into their current form. He also has been a good friend, who was there, every time I needed to share an idea and work it further.

Georgios Zouganelis was and still is the "secret member" of the SENSE Lab. Since 2006, we have done hundreds of meetings, discussing various aspect of technology, tossing links and ideas to each other. He is one of the people that I entrust with the early version of my ideas, including those of the PhD. He was always there for me; giving me advice and helping me size things up, from a more modest perspective.

In these acknowledgments, I also have to include the members of the SENSE group (in order of appearance as member of the lab); Stephanos-Androutsellis Theotokis, Vassilios Vlachos, Georgios Gousios, Kostantinos Strogyllos, and Dimitrios Mitropoulos. All of them are true (as in true metal) friends who provided valuable help all these years. Especially the first 3-4 years in the SENSE Lab with Vassilis and Georgios were the most fun ones. This situation is best described in Georgios's PhD [112]. I also have to note that Kostantinos Strogyllos also reviewed a premature version of this work and provided me with many helpful comments.

With Dimitris (Mitropoulos) we have spent a significant amount of time the last five years in meetings where we discussed various ideas, from which many papers have emerged, in various areas. We had that 4-day trip in Kerkira (PCI'09 if I remember correctly), which reminds me that *“What happened in Kerkira, stays in Kerkira.”*.

Along with all the people that I have already mentioned, there are a group of friends that began their PhD almost the same time as me. George Oikonomou, which became a very close friend, supported me greatly as a friend and as an academic. He is the only person outside the PhD committee that actually has read the whole thesis multiple times (at least three times if I remember correctly) and provided significant comments. The most fun part is that most from our scientific conversations took place in beer houses or rock bars :). We were not very popular among the other clients, but we had lots of fun discussing all our ideas.

Apostolos Malatras was there in the early years (then he moved to England to do his own PhD). We were classmates in our BSc years in University of Piraeus (UniPi), then we followed the same path in MSc (with one year difference) and we began our PhDs, again with one year difference.

The rest of the group is consisting by Marianthi Theocharidou and Aggeliki Tsohou. Marianthi and Aggeliki were there from day one, sharing common problems, discussing ideas etc. The odd thing is that we never worked on a paper with each other, but the fact is that all have contributed ideas in each other's works. In addition, Marianthi Theocharidou have read an early first chapter of this thesis and contributed valuable comments.

Christos K Loverdos is a brilliant guy (and Software Engineer) that gave me numerous pointers, references, books and advices. He is a seasoned software engineer and one of the few people that truly understood the theoretical concepts behind this discipline.

Georgios Makridakis is the only tech-savvy doctor (oh yes, he is a neurologist) that I know. He is a maniac with C++ and especially with template metaprogramming. Now that I think of it, he is obsessed with metaprogramming in any language. Since this thesis concerns his interest, we have discussed thoroughly various aspects of this work in our coffee meetings. In addition he provided various interesting research papers. Not bad for a doctor, right?

Apostolos Syropoylos also provided some helpful comments and various corrections in a very late version of this work.

I would like also to thank Athina Tzirozidou, who was the one person that pushed me finish this work, in times were other things seemed more important. She even bought a PDF editor in her iPad to proof-read the text (and never used by the way; the produced PDF crashed the editor, when she opened it and try to add a remark).

Finally, there are people that I would like to thank them for their influence and support during this PhD; Lazaros Gymnopoulos, Dimitrios Liappis, Simoudis Simos, Elias Polychronopoulos, Ar-giris Bendilas, Maria Sifnioti, Dimitris Bendilas, Maria Kechagia, Marios Fragoulis, Xristos Zotos, Anastasia Zioga, Georgios Maniatis, Agapios Avramidis, Kostantinos Saidis, Sofoklis Stouraitis, Christos Zotos, and Kostantinos Pialoglou. To those who I missed, sorry :).

June 29, 2015,
Vassilios Karakoidas

eg. *So long and thanks for the fish!*

Summary

Domain-specific Languages (DSL) [97, 269], also known as *micro-languages* or *little languages* [29, 257], are programming languages designed to specifically solve problems within a particular domain. They are used to improve the efficiency of the software development process [237, 242]. Well-known DSLs include regular expressions, markdown and SQL.

On the contrary, General-purpose languages (GPL) have a wider scope. They provide a set of processing capabilities applicable to different problem domains [29]. Typical examples of GPLs are Java, C++ and Scala.

In modern software engineering approaches, DSLs are often used together with GPLs [124, 164, 215, 251]. For example, the integration of SQL with various GPLs constitutes a field that drew the attention of researchers and of practitioners [50, 107, 227, 274]. For the case of Java, the language integration is executed with the use of a JDBC (Java Database Connectivity) application library [91]. The programmer has to pass the SQL query to the database as a string. Through this process, the Java compiler is completely oblivious to the SQL language contained within the Java code with many SQL syntax and type errors detected at runtime. Such errors remain undiscovered, even with extensive testing during the development process.

There are more sophisticated methods; many of them use common compiler, metaprogramming and generative programming techniques and methodologies. All those approaches suffer from one or more of the following weaknesses:

- They deal with the problem for only one DSL-GPL pair. The integration of other DSLs is either not supported or supported via complex mechanisms.
- They recommend complex solutions that syntactically burden mainstream programming languages, obliging programmers to remember difficult semantics that are not aligned with the language philosophy and best practices.
- They are not practical, since they offer new programming languages that deal with the problem efficiently, and this usually requires a complete rewrite of complex DSL systems. The proposed programming languages are an extension of an existing language, or a language with a completely new syntax.

A review of current research literature in the field of DSL integration with GPLs reveals that these approaches share many common design concepts. These concepts are:

Language Code Blocks Recent research approaches [213, 214, 253] state that a successful integration method should provide a way to syntactically isolate the DSL code from the host language, within its syntax. The addition of special DSL-oriented operators must be avoided, unless it is crucial for providing refined support for a specific DSL. The integration of Perl [272] with regular expressions is the perfect example for the latter. Perl introduces custom operators that provide a very efficient way to use regular expression patterns in a program. The problem is, that other languages cannot be integrated without introducing syntax noise in

the host language, and this is the main reason that Perl integrates other DSLs, like SQL with a simple application library. Language code blocks constitute the perfect trade-off to avoid such, since they permit native DSL syntax, without breaking or polluting the host language syntax.

Special DSL Types Specialised types should also be defined to aid the compilation process. These types may give the compiler information that will initiate specific behaviours at compile-time. These special types should be defined arbitrarily with each compilation and should not constantly burden semantically the type system of the host language.

Configuration Hooks Compile-time configuration hooks should be defined to instruct the compiler about the parameters of the DSL. Java supports these types of hooks via compile-time annotations [110]. Python has a similar concept with decorators [175]. Almost all mainstream languages have similar mechanics.

Type Mapping All data types within a DSL should be able to interact with the host language. The compiler should be able to understand the compatibility of these types and raise compile-time warnings and errors. Typical type mapping examples are already defined in various cases e.g. in the JDBC specification, which map normal Java types with SQL ones.

Current research and industry implementation lead to the definition of a set of functional requirements.

DSL Syntax Validation The DSL should be syntactically checked at compile-time. In most case studies presented, this feature included. Usually this was done only for one DSL. Consequently, every efficient DSL integration facility must syntactically check hosted languages.

Static Typing The language embedding should ensure type safety at compile-time and the values that are exchanged between the GPL and the DSL in a program should be checked at compile-time.

Extensibility One common problem in all ongoing research and industry systems is the lack of extensibility. Almost all implementations support one or two DSLs with custom syntax, but this is it. They cannot support embedding of any other language in an efficient manner. The systems that support many DSLs, are tightly coupled with one specific host language.

In addition to the aforementioned functional requirements, a set of non-functional ones should also be defined. Those requirements were derived from existing works [97, 106, 187].

Semantic Complexity The proposed solution should not increase the semantic complexity of the host language. In practice, it is not possible to add functionality without some form of extension. On the other hand, the proposed extensions should be logical and as simple as possible.

Domain Model Maintenance Each DSL must maintain its syntax. One of the major advantages, when using DSLs in software development, is that the DSL provides a view of the domain model, permitting the efficient applications development in these areas.

Re-usability The proposed solution should encourage re-usability. This is expressed through two facets; first by enforcing the re-usability of DSL code, which is repetitive and secondly by enhancing the usage of existing DSL application libraries.

Domain-Specific Optimisations When the DSL code is analysed, it is an excellent opportunity to induce DSOs (Domain-specific Optimisations).

This thesis introduces J% (j-mod), a DSL-aware extension of the Java programming language. J% provides an extensible way to embed DSLs into Java in the form of compiler modules. So far, it supports SQL and regular expressions. It facilitates existing Java DSL APIs, for example SQL support uses the standard JDBC application library [91]. However, this is not a requirement and a module can translate DSL code to the host language or even directly to JVM bytecode. It introduces the following new characteristics:

Modular Design Developers can create their own compiler module, supporting any DSL without restrictions, with the usage of a well-defined compiler module API.

Static typing The compilation process is type-safe. The compiler performs static typing in one or several DSLs.

DSL Syntax Validation The DSL is syntactically checked at compile-time. All errors are reported as compiler error messages.

DSL Syntax Each hosted DSL retains its syntax. There is only a minor addition to each DSL grammar of each language in order to support type mapping with J%.

Pragmatic Many meta-programming and language extensions deal with language inter-mixture by sacrificing simplicity in the host-language syntax. Other approaches define new languages, which translate DSLs to host languages. J% is a pragmatic approach that extends a mainstream language, like Java, for achieving integration with elegance and usage of current DSL application library implementations.

Domain-Specific Optimisations J% Provides the facilities to perform DSOs at compile-time. This work presents two case studies; a performance optimisation for regular expressions with the utilisation of the FIRE/J application library and automatic usage of prepared statements for SQL. The regular expression module, which is analysed in detail in Section 4.4, performs compile-time code generation for each expression using the FIRE/J library [146]. Compile-time optimisations are not a novel concept, since metaprogramming exists and offers such mechanisms (see Section 2.5). Alongside with compile-time optimization techniques, it should be possible to apply compile-time security enhancements. The DSL code should be analysed and fortified against possible security risks. These risks, among others, could involve input validation problems and code injection. Domain-specific security enhancements are demonstrated in the proposed system by the SQL module. Current implementation automatically generates code that uses *Prepared Statements*, which provide out-of-the-box protection against SQL injection attacks.

Chapter 1

Introduction

Domain-specific language (noun): a computer programming language of limited expressiveness focused on a particular domain.

Martin Fowler, Domain-Specific Languages

Programming languages form an interesting and dynamic field of computer science. The multiparadigm programming, which was established in the early 1990's [205, 285], demanded that each problem solved with the most suitable programming language.

Many languages were developed to address the multiparadigm call, some extended their abstractions and syntax to encapsulate complex systems and models, while others narrowed those down to deal with specific problems. By their design and focus, programming languages are divided into two main categories, Domain-Specific Languages (DSL) and General-Purpose Languages (GPL).

Domain-Specific Language (DSL)s, as the name implies, are designed to deal with a specific problem domain. DSLs are used to make the development process more efficient [105, 113, 237, 242]. To achieve that goal, DSLs sacrifice syntactic flexibility. They are often called *micro-languages* or *little languages* [29, 97, 223, 257]. Well-known DSLs include regular expressions, Structured Query Language (SQL), eXtensible Markup Language (XML), $\text{\LaTeX}$ and Hyper-Text Markup Language (HTML). Usage of DSLs in the software development process reduces cost and enhances productivity [187, 257].

General-Purpose Language (GPL)s have a wider scope, providing a set of processing capabilities and abstractions applicable to various problem domains [29, 221, 222, 223]. Typical examples of GPLs are Java, C, C++, and Python.

1.1 Motivation

This thesis emphasises on the integration of DSLs with GPLs. The case is simple and practical; each software system's development effort adopts a language to be developed with, more commonly a GPL. This language comes along with an ecosystem, which typically includes development platforms, standard libraries etc. DSLs are usually included in these platforms as application libraries and some of them integrated with some GPLs. This thesis focuses on the interactions of these languages. More specifically on those between one GPL (defined as host language) and many DSLs.

DSLs exist in many forms inside mainstream development platforms. Table 1.1 shows a list of the most popular DSLs that are included in the Java development platform. Typically, DSLs are used through library calls of a *host language*. The interactions between GPLs and DSLs (mainstream languages) follow the *Implementation: Embedding* integration pattern [187, 238]. This

pattern dictates that the DSL code is passed as a verbatim alphanumeric series of characters, and is executed by the application library at the program runtime. If an error occurs, then the application library raises a runtime exception, thus informing that the DSL code contains an error. This method is very practical and easy to implement, but also carries a great disadvantage. The host language is completely unaware of the DSL. If the DSL has an error in the syntax, it will be discovered only in the testing phase of the development cycle, or never at all.

An example includes the integration of SQL with the Java programming language, which is currently implemented by the Java Database Connection (JDBC) application library [91]. Practically, JDBC defines a series of interfaces, which are implemented by each database driver. Since it follows the *Implementation: Embedding* pattern, the developer is compelled to pass the SQL query as a String (`java.lang.String`). Since, the Java compiler remains completely unaware of the SQL query and syntactical errors can only be found at runtime, usually in the form of an exception raised by the JDBC driver. Such errors remain undetected if during the testing phase of the product the SQL query is never invoked by the appropriate execution path.

Another typical example resides in the modern web development. A standard web application typically contains the following DSLs to implement dynamic characteristics: Cascading Style Sheets (CSS), Javascript, HTML, XML IDs. Many libraries have been developed to enhance the development process by providing Javascript enhancements in the form of libraries like *jQuery*¹ and *mootools*², CSS frameworks like *bootstrap.css*³ or even more sophisticated libraries like the one presented in reference [178]. But the problem remains; all the above technologies are loosely coupled, thus testing is the only way to see if something really works properly.

The goal of this research is to identify current research trends on the area of DSL-GPL integration and define possible solutions and techniques that will allow GPLs, to efficiently integrate DSLs.

1.2 On the Use of Domain-Specific Languages

The Java Platform exists more than a decade and provides a development environment for several application areas, such as web, grid computing, and traditional client-server applications. The official development environment of the Java platform is released by Oracle and is widely-known as the Java Standard Edition SDK. The process of development and innovation in the Java ecosystem is built around a community, that writes proposals of features, known as Java Specification Request (JSR)s, which are supported by prototype implementations.

In the literature, it is referred that DSLs are used to reduce software development cost, by introducing domain efficiency [187, 257]. This thesis focuses on practical research aspects as well as with the theoretical ones; thus the first problem that needs to be examined, is to measure the adoption of DSLs in software projects. In addition, the average number of DSLs usually used in a software project needs to be investigated.

A simple experiment conducted on the software [144, 147], which resides in the Maven repository. Maven is a build automation tool used primarily for Java projects, and it is hosted by the Apache Software Foundation⁴. To describe the software project being built, its dependencies on other libraries, and the build order Maven uses XML. The repository contains more than 400,000 Java Archive (JAR)s, in a variety of languages that are all using the Java Virtual Machine (JVM) platform as basis. These languages include Java, Clojure, Groovy, and Scala.

¹<http://jquery.com/>

²<http://mootools.net/>

³<http://twitter.github.com/bootstrap/>

⁴<http://www.apache.org/>

Table 1.1: Popular built-in DSLs for the Java platform

DSL	Description
Regular expressions	Pattern-matching language, implemented in <code>java.util.regex</code>
SQL	Database query language, implemented in <code>java.sql</code>
XML	Encoding documents in machine-readable form, implemented in <code>javax.xml</code>
XPATH	Query language for XML, implemented in <code>javax.xml.xpath</code>
XSLT	Transformation language for XML, implemented in <code>javax.xml.transform</code>
Oid	Kerberos v5 Identifier [170], implemented in <code>org.ietf.jgss.Oid</code>
HTML	HyperText Mark-up Language, implemented in <code>javax.swing.text.html</code>
RTF	Rich Text Format, implemented in <code>javax.swing.text.rtf</code>
URI	Uniform Resource Identifier (RFC2396 [31]), implemented in <code>java.net.URI</code>
Formatter	An interpreter for <code>printf</code> -style format strings, implemented in <code>java.util.Formatter</code>

Initially, a snapshot (January 2012) of the Maven repository was downloaded locally. The repository contains various projects and their releases. A project version can be uniquely identified by the following triplet: *group id*, *artifact id* and *version*. The Maven repository contains several versions of each project. Figure 1.1 illustrates the hierarchy of the Maven repository.

All versions were filtered out and only the latest was kept. For each version in the Maven repository, each binary JAR file is accompanied by another JAR, which contains the source code. For this experiment, the source code JARs were downloaded, and the code was processed by the Cross-Language Metric Tool (CLMT) tool [243], by using the `DSLUsage` plug-in. Table 1.2 presents several size metrics regarding the final version corpus. It included 12,959 projects, with more than 110 million lines of code.

Table 1.2: The selected Maven projects' size metrics

Metric	Value
Project Count	12,959
File Count	604,821
Module Count	72,302
Lines of Code	110,156,703
Source Lines of Code	61,246,807
Comment Lines of Code	36,696,217
Number of Classes	499,588
Number of Interfaces	89,145
Number of Enumerations	12,732

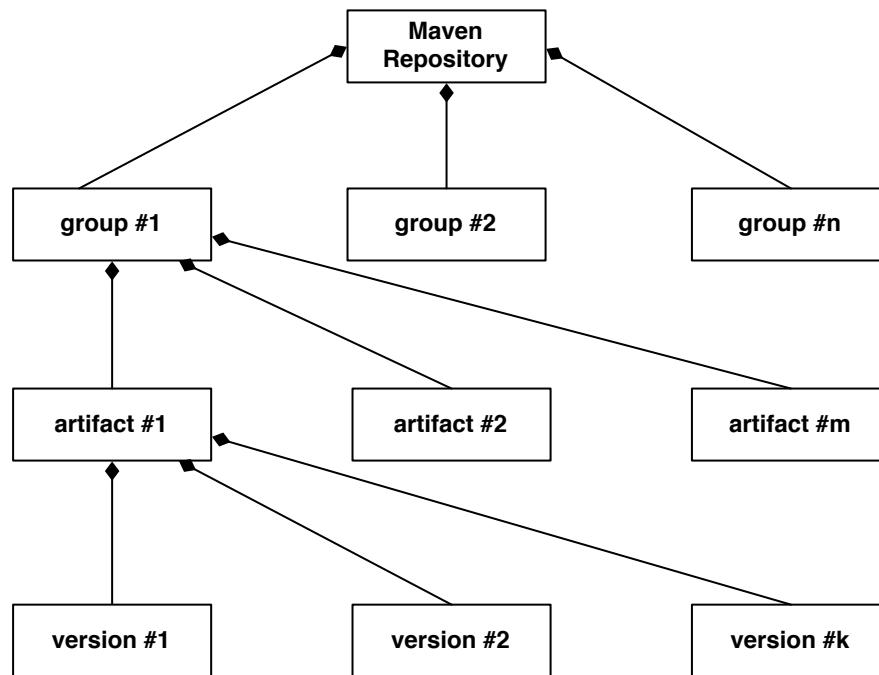


Figure 1.1: A UML class diagram depicting the heirarchy of the Maven repository

Table 1.3: List of selected DSL application libraries

DSL	Java Package
Regular Expressions	<code>java.util.regex</code>
XML	<code>javax.xml</code> , <code>org.w3c</code> and <code>org.xml</code>
SQL	<code>java.sql</code> and <code>javax.sql</code>
XPath	<code>java.xml.xpath</code>
XSLT	<code>javax.xml.transform</code>
RTF	<code>javax.swing.text.rtf</code>
HTML	<code>javax.swing.text.html</code>

1.2.1 Experiment Methodology

The methodology was the following; A set of standard DSL application libraries was identified, and the source was scanned for specific *import* statements e.g. `java.util.regex`, which indicated that the standard package that implements regular expressions was used, thus regular expressions were used. If a package is detected in the source code, then the project will be tagged that it uses one (1) DSL. Consequently, if a project has DSL count four (4), then it means that four (4) different application libraries were detected during the source code scan. Table 1.3 lists the selected DSLs application libraries. Note that all these libraries are included as part of the Java Software Development Kit (SDK).

The initial goal of this experiment was simply to provide quantifiable results that are indicative regarding the usage of each DSL, thus only files containing Java code were taken into account. Build files or other resources that may contain DSLs were not included.

One final assumption was also made; if XML Path (XPATH) or Extensible Stylesheet Language and Transformations (XSLT) were found in the source code, then the project would be marked that it also uses XML, since those two languages are used for query and transformations on XML Document Object Model (DOM) trees.

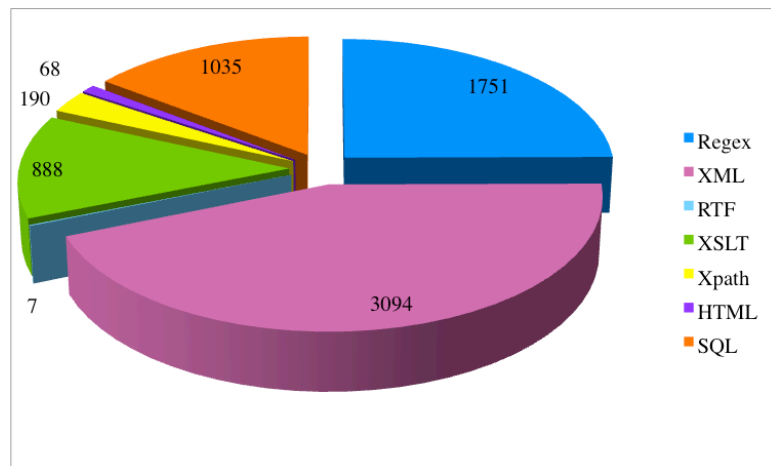


Figure 1.2: DSL language usage in Java projects

1.2.2 Results

The results were not surprising. Many researchers mentioned in the past that DSL usage is significant in the software development projects [22]. Figure 1.3 illustrates the number of projects using a specific number of different DSLs. More than one-third of the projects (4,655) are using one or more DSLs. Figure 1.2 indicates that the most popular DSL is XML, with a percentage of 75% usage in Java projects. Regular expressions follow with a 25%.

Another interesting aspect of the results is presented in Table 1.4, which lists popular DSL usage combinations. In the left column the DSL languages combination is presented, along with the right column, which contains the number occurrence of this combination.

The results of this experiment indicate that DSLs are used and that they are very popular in the Java software ecosystem. Note that this experiment was not exhaustive. If more application libraries were detected and peripheral files were taken into account, the DSL usage would have been higher. This was not the goal of this thesis though, and these data were only needed to solidify the motivation to further investigate methodologies that will make DSL usage even more efficient.

1.3 Thesis Outline

This thesis is structured as follows; this chapter provides a brief introduction, covers the relevant scientific areas, identifies and exposes the research question and the motivation behind it. Chapter 2 focuses on the three (3) different fields that are partly involved in this research, (1) *Programming Languages*, (2) *Domain-Specific Languages* and (3) *Metaprogramming*.

Chapter 3 discusses the theoretical aspects of this research, provides a definition for all the key concepts in GPL–DSL integration and covers the design of the J% programming language.

Chapter 4 analyses the implementation of the J% compiler framework. Along with practical experience, this chapter also examines the two DSL modules that are offered with the standard compiler implementation. These include a regular expression module and a SQL module that provides a more sophisticated and complex integration approach that fully utilises the capabilities of the J% framework.

Chapter 5 provides the evaluation of the proposed system, which is divided in two parts; the first part evaluates and criticises the proposed system against the requirements that were set in

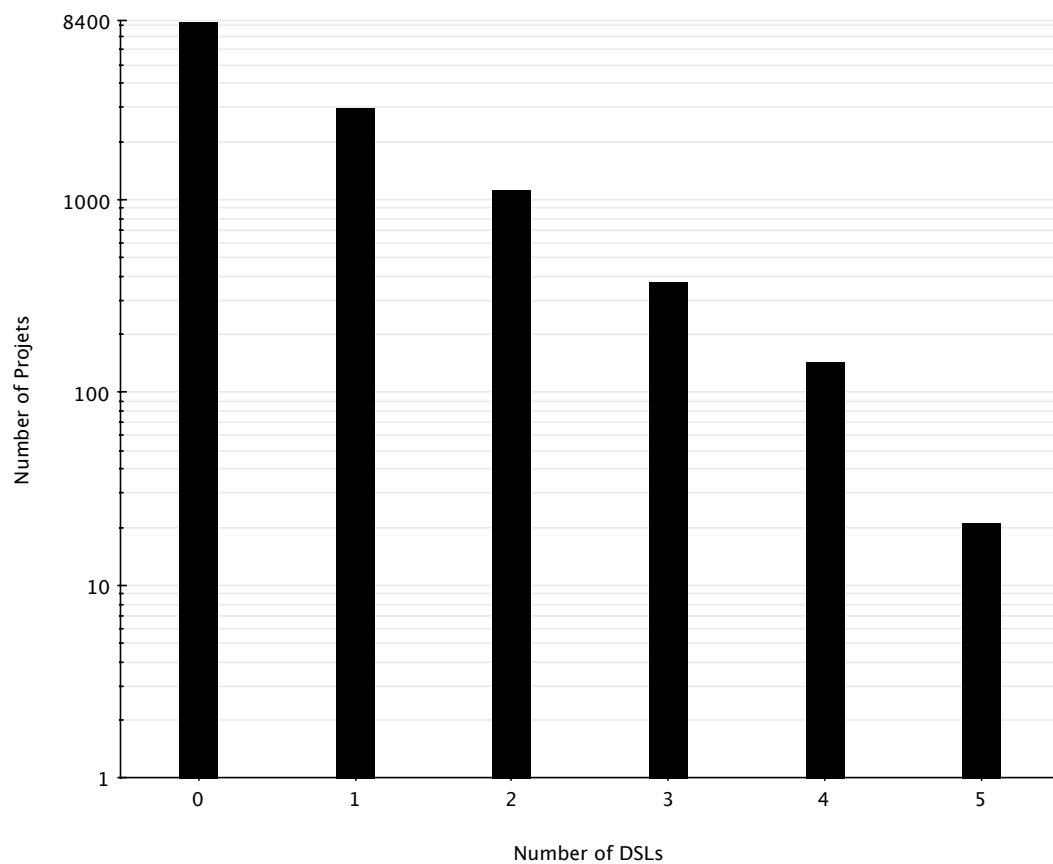


Figure 1.3: Number of DSLs' used in Java projects

Table 1.4: Popular DSL Usage Combinations

DSLs	Count
XML	1,561
Regex	909
SQL	493
XML, XSLT	475
Regex, XML, XSLT	158
Regex, XML	303
SQL, XML	162
Regex, SQL	116
Regex, SQL, XML, XSLT	80
Regex, SQL, XML	71
SQL, XML, XSLT	54
XML, XPath	50
XML, XPath, XSLT	39
Regex, XML, XPath, XSLT	38
HTML	23
Regex, XML, XPath	20
Regex, SQL, XML, XPath, XSLT	18
SQL, XML, XPath, XSLT	11
HTML, Regex, XML	10
HTML, XML	10

Chapter 3, and the second examines more practical implications with the usage of software size metrics.

Finally, Chapter 6 summarises the contribution of this research and highlights open issues that will guide future research.

1.4 Research Methodology

Pólya [207] described the basic steps needed to solve a problem and deal with the process of research, providing a solid foundation. Wegner [275] identified the research paradigms that guided the computer science until the late 70s. He observed that computer science is in part a *scientific*, *mathematical* and a *technological* discipline. It is dominated by *empirical*, *mathematical* and *engineering* research paradigms. According to Wegner [275], programming languages, even though they followed their own research paradigms [92], along with general computer science research paradigms, passed three major phases of development, the (1) *data-gathering* (1950-1960), the (2) *elaboration and abstraction* (1961-1969) and the (3) *technological* (1970-today) phase [275].

Categorisations for research in computer science have been abundant while the previous decades, and various proposals have been made [25, 64, 79, 108, 130, 210, 286]. Adrion [7] proposed a simple classification that divided the research in four categories: *Scientific*, *Engineering*, *Empirical*, and *Analytical*. This thesis describes the work done in a typical *Systems' PhD*. According to this categorisation, the *Engineering* approach is followed, which follows the process depicted in Figure 1.4.

The *engineering* process is straightforward. At first, information is gathered through observation, mostly of other systems that implement solutions to a specific problem. The proposition of an improved system follows, along with its realisation into a real prototype system. The prototype is measured, and the results are analysed. If the results show more room for improvement, then the process is repeated. The aforementioned *Engineering* process encapsulates perfectly the research methodology followed in this work.

Figure 1.5 illustrates in a hierarchical manner the classification terms, according to the Association for Computing Machinery (ACM) Computing Classification System (CCS) 2012 [176]. This thesis deals mainly within the boundaries of the general term of *Software and its engineering*. This includes two relevant subcategories; *Software notation and tools* and *Security and privacy*. The work in this thesis is more focused in the first subcategory, which includes the scientific areas of *General Programming Languages*, *Context specific languages*, *Formal language definitions*, and *Compilers*. The second subcategory include scientific areas that are covered partly and only as by-products of this research. Namely, these are, *Software and application security* and *Database and storage security*.

1.5 Contribution

This thesis presents J% [146], an extension of the Java programming language [110]. The main contribution of J% is the development of a framework by which arbitrary DSLs can be embedded in Java programs in a type-safe way that supports syntax-checking. Other relevant efforts either do not support arbitrary DSLs or do not provide type safety and syntax checking.

DSL Awareness The host language is aware of the DSL, which should be syntactically and semantically checked at compile-time. All errors are reported as compiler warnings and errors.

Static Typing The compilation process is type-safe. The J% compiler recognises DSL statements and provides the means to perform checks that ensure correct value-passing between the

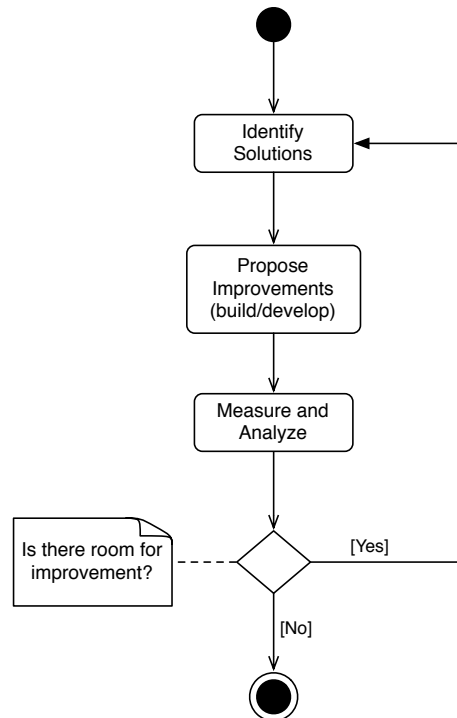


Figure 1.4: Engineering research process

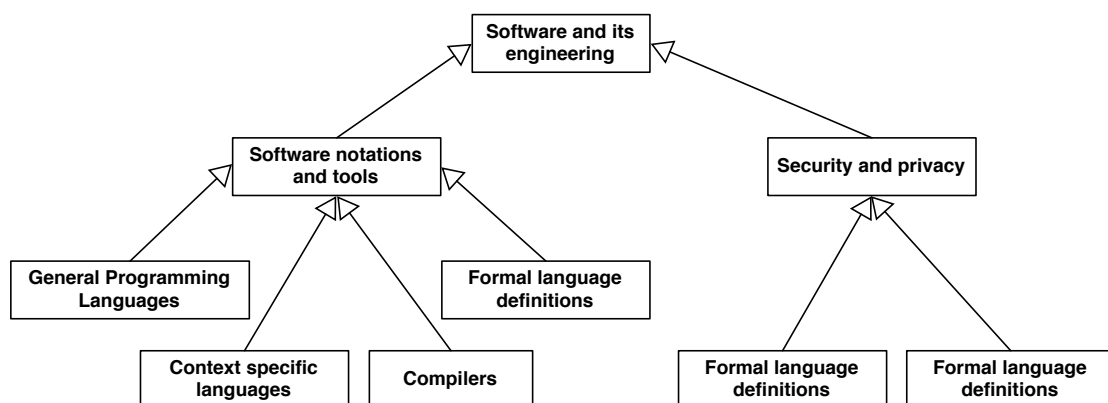


Figure 1.5: Scientific areas according to reference [176]

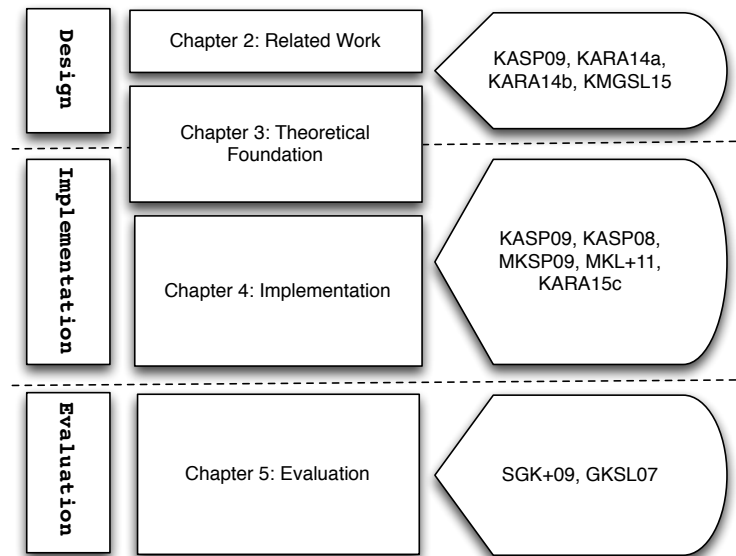


Figure 1.6: Published research work categorised (Appendix A)

host language (Java) and each DSL accordingly. This is achieved through language constructs that enable the compiler to check the DSL statements.

Domain-Specific Knowledge Each DSL retains its syntax, thus domain-specific knowledge can be harnessed through domain experts (See Section 2.2.2).

Extensibility Developers are able to create and support their own DSLs, in the form of compiler DSL modules.

Compile-time Configuration Each DSL statement can be separately configured with user-defined parameters at compile-time.

The above are the major contributions presented in this research work. Additionally, this research included findings on the area of DSLs, specifically for regular expressions, by defining new methods for optimised execution in Java environment [146] by using generative programming.

Also, research work was performed on software quality evaluation, where the CLMT tool was crafted to calculate well-known metrics like Lines of Code (LOC), the Chidamber and Kemerer (CK) metric suite [52] and the Instability metric [241]. CLMT was presented in reference [244] as part of this work and is described in Appendix C. CLMT is partially embedded in the J% compiler, offering a basic set of size and complexity metrics to J% users. It was also used to quantify DSL usage in Section 1.2.

Figure 1.6 relates the three basic chapters of this thesis with the published research work. The complete publication list is available in Appendix A.

1.6 J% in Action

Consider the case of a typical Java application that needs to access a relational database as *MySQL* or *Oracle*. The following code executes a standard SQL query that accepts one parameter, by using the JDBC Application Programming Interface (API).

```

public class SQLTest {
    public void sqlQuery(String id) {
        try {
            Connection connection = connectToDatabase();
            Statement stmt = connection.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM t where id = " + id);

            // Fetch each row from the result set
            [...]
        } catch (SQLException e) { /* error */ }
    }
}

```

The class `SQLTest` contains a method called `sqlQuery` that accepts an integer (`String`) parameter, named `id`. Typically, the parameter is concatenated with the SQL query. This method arises the following problematic issues:

1. The SQL query is passed as a `String` and the compiler does not know that this is an SQL query. A typical syntax error in the SQL statement could evade all tests at compile-time and even at runtime, and finally deployed with the final version of the application, potentially causing many problems to its users. To name a few, it is possible to lead to data corruption or create security risks.
2. The parameter (`id`) that is blindly concatenated to the SQL query is never checked or converted to valid a SQL data type. Many erroneous situations are never checked; the `id` variable could have invalid or incompatible value, or produce a malformed query. In addition, malicious users can exploit it to devise input validation attacks [189].

Many workarounds exist, like the use of *prepared statements*⁵, which ensure that all parameters are checked for their proper types and values at runtime.

If the previous code had been using *prepared statements*, it would be rewritten as follows:

```

public class SQLTest {
    public void sqlQuery(String id) {
        try {
            Connection connection = connectToDatabase();
            PreparedStatement stmt =
                connection.prepareStatement("SELECT * FROM t where id = ?");
            stmt.setInt(1, Integer.valueOf(id));
            ResultSet rs = stmt.executeQuery();

            [...]
        } catch (SQLException e) { /* error */ }
    }
}

```

The above code is better in many ways; `String` concatenation is no longer used, so it is not possible to produce a malformed SQL query, which will later be passed on to the database. In

⁵<http://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html>

addition, the parameter is checked at compile-time and must be a valid integer. If not, the Java compiler produces a compile-time error.

However, the problem still lies. The compiler remains ignorant of the SQL query and any syntax problem will result in a runtime error. J% deals with the problem from a different perspective. The language is extended to support DSLs internally, thus the same program would be written as follows:

```
public external SQLTest extends SQLQuery<SQLConf> {
  SELECT * from t where id = #[id]<int>
}
```

The code is simpler and straightforward. However, there are three things that are unknown to a Java developer; the `external` keyword, the `extends` statement with the generic SQL type and the `#[id]<int>` addition to the query. The `external` keyword indicated that this type includes DSL code.

The `SQLQuery` type defines that this class is an SQL query, which is specialised by the `SQLConf` class. For example, with the proper configuration, the compiler can be instructed to use the custom flavor of the SQL, which is used or check the provided query with the usage of a specified SQL schema.

The `#[id]<int>` statement is known as an *external reference* and informs the compile-time environment that this query accepts one integer (`<int>`) parameter, named `id`.

The product of the compilation process is Java-compatible code, which is later compiled by a standard compiler.

1.7 Typographical Conventions and Diagrams

This section lists the conventions used in this thesis [240]. The code listings use the `typewriter` font. To avoid listing boiler-plate code, several code lines are omitted by using the `[...]` symbol or its more simplified version `....`. Code examples usually are in the Java programming language. Comments in the code listings are defined by using the standard Java notation, which consists of the single-line (`//`) and the multi-line (`/* ... */`) symbols. The following listing contains the aforementioned conventions in action,

```
/*
Excerpt of class Foo
*/
public class Foo {
  [...]
  public boolean test(int i) {
    // the code goes here
  }

  // The method body is omitted
  public int test() { ... }
  [...]
}
```

The unix shell command prompt is displayed by the symbol `$` in the start of a line, and denotes shell commands.

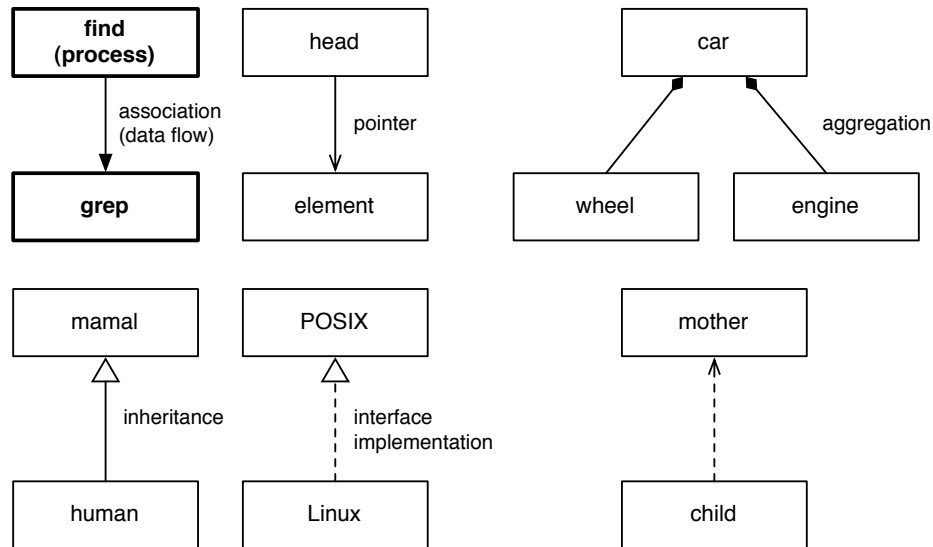


Figure 1.7: UML class diagram notation [240]

```

$ java -jar jmodv2.jar --module-list
---EXTERNAL modules---
Default module - Default External Module (dummy module), ver. 1.0
Regex - Regular expression library module, ver. 0.3
SQLModule - SQL Support Module, ver. 0.2
GetSet - Getter/Setter Generator, ver. 1.0

```

Almost all diagrams in this thesis are following the Unified Modeling Language (UML) notation [98, 160]. Diagrams that describe the design of software components are depicted as class diagrams. Activity diagrams were used for the description of algorithms and processes. Cheat sheets with the basic notation elements of UML class and activity diagrams are illustrated in Figures 1.7 and 1.8 respectively.

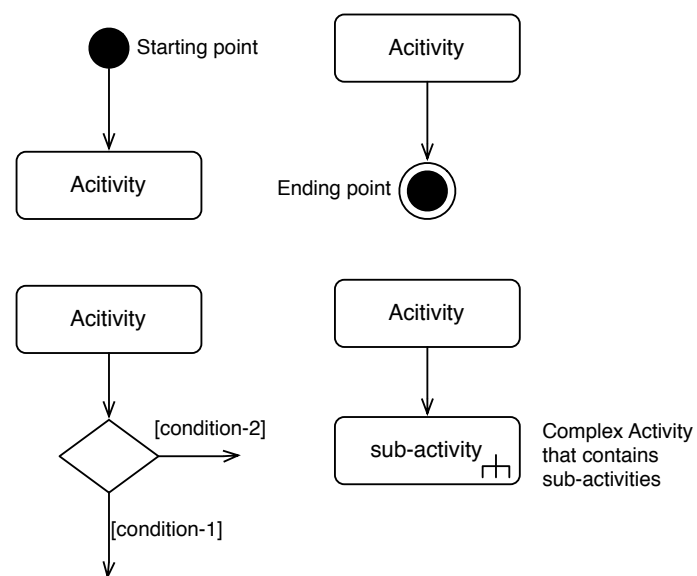


Figure 1.8: UML activity diagram notation [240]

Chapter 2

Current Research on Domain-Specific Languages

The knowledge and experience of the human Picard is part of us now. It has prepared us for all possible courses of action. Your resistance is hopeless ... Number One.

Locutus [to Riker], The Best of Both Worlds Part II, ST:TNG

This chapter presents the state-of-the-art research for DSL integration patterns with GPLs. Section 2.1 and 2.2 provide formal definitions regarding the concepts of DSLs and GPLs. Also, advantages and disadvantages of DSL usage in the software development process are discussed thoroughly.

Section 2.3 focuses on design and implementation patterns of DSLs. Complementary works are presented in Sections 2.3.2 and 2.5.7.

A series of DSL-GPL embedding and integration case studies is presented in Section 2.4. DSL integration efforts are divided into two main categories; one aspect that deals with the DSL design and implementation problem and a second that focuses on extensible notations using metaprogramming and similar methods and techniques, which presents DSL integration as a side-product. Approaches based on the DSL design and implementation patterns are further studied in Section 2.4. Metaprogramming and generative programming research methods listed in Section 2.5.

2.1 General-purpose Programming Languages

This section examines the research area of *Programming Languages* and provides formal definitions of its basic concepts.

Regarding the use of term *Programming Language*, Wirth [281] states:

“We believe that this term is rather unfortunate on the whole, because a programming language is not spoken, and therefore is not a language in the true sense of the word, Formalism or formal notation would have been more appropriate terms.”

2.1.1 Definitions

First, the definition of the term *Programming Language* should be provided. A formal approach by the Institute of Electrical & Electronics Engineers (IEEE) Computer Society proposed *Programming Language* [37] through the term *computer language*,

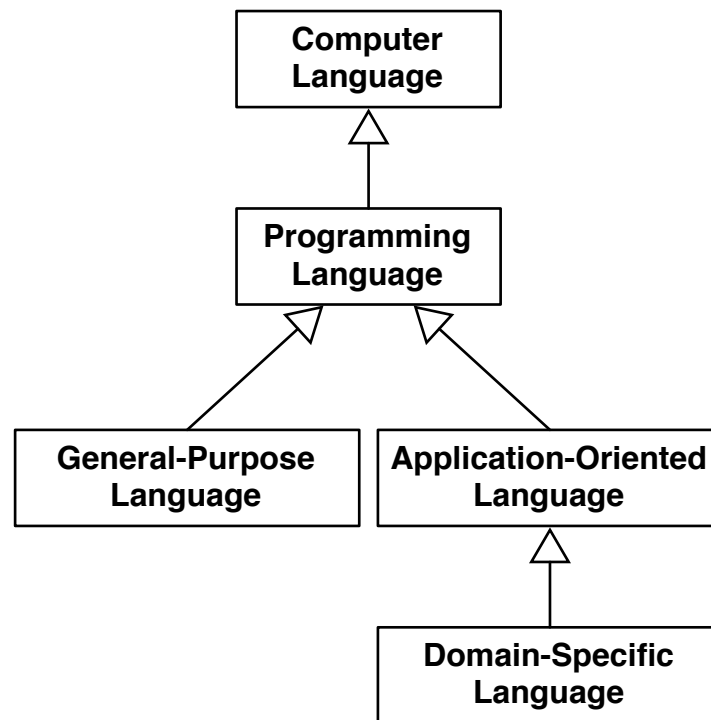


Figure 2.1: Language definitions hierarchy

“(1) A language designed to enable humans to communicate with computers and computer systems and (2) a language that is used to control, design, or define a computer or computer program.”

and as *programming language*,

“A computer language used to express computer programs.”

concluding to the definition of the *General-Purpose Language*,

“A programming language that provides a set of processing capabilities applicable to most information processing problems and that can be used on many kinds of computers.”

2.1.2 Programming Languages Categorisation

Programming Languages are divided into two categories [37, 221, 222, 224, 226, 236] (Figure 2.2):

Declarative (1) A non-procedural language that permits the user to declare a set of facts and to express queries or problems that use them. (2) A programming language that can be understood without reference to the behavior of any particular computer system.

Imperative Languages that are based on the change of state and assignment. They take sequential instructions and apply them to data of various kinds. Typical examples are C [134], Fortran [212], and Pascal [137].

Declarative languages are categorised into [37, 226]:

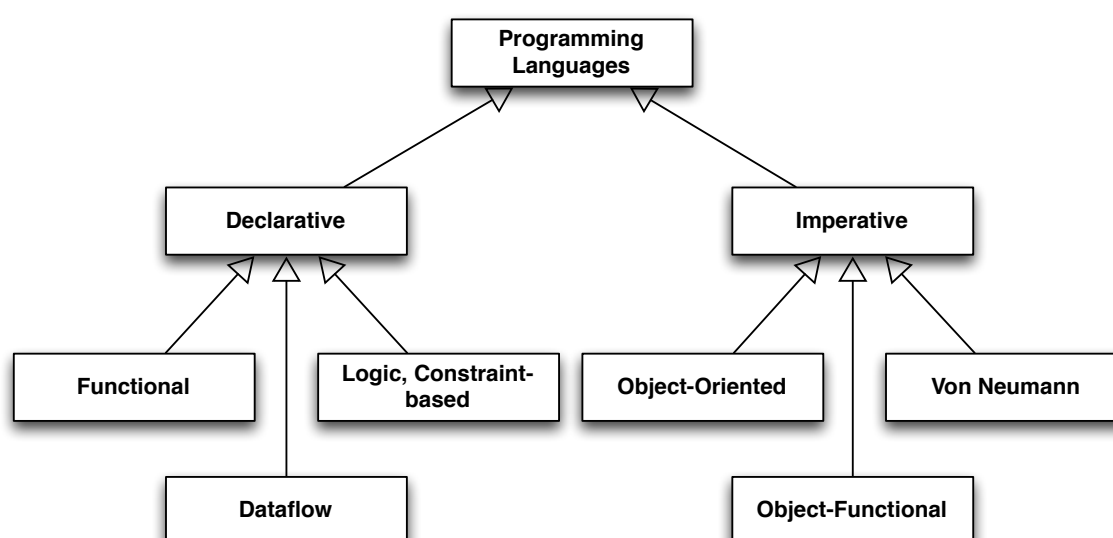


Figure 2.2: Programming languages categorisation

Functional This category employs a computational model based on the recursive definition of functions. Inspired from the *lambda calculus*, each program is considered a function that has inputs and outputs. Typical examples of functional languages are *Lisp* [5], *ML* [256] and *Haskell* [132].

Logic, Constraint-based Languages that follow the axioms of propositional logic. Sets of relationships are defined that must be satisfied, using goal-directed search. *Prolog* [34] is a typical representative programming language for this category.

Dataflow This model of computation is defined as the flow of information through primitive functional nodes, resulting to a parallel model of execution. *Id* [15, 217] and Value-oriented Algorithmic Language (VAL) [185] are classic dataflow languages.

Imperative languages are divided into [37, 226]:

Object-Oriented Languages are based on the Object-Oriented approach, use objects as their central points with encapsulated properties, messages, and inheritance [276]. Classic examples of Object-Oriented (OO) languages are Java [110] and C++ [247].

Von Neumann The most familiar category and practically the most popular. The basic means of computation is the modification of variables [226]. C [134], Pascal [137] and BASIC [230] are popular languages of this category.

Object-Functional With the appearance of Scala [173, 200], the *Object-Functional* method has emerged. This paradigm uses a mix of OO features, along with functional traits. This hybrid reduces code size and permits the creation of more elegant computer programs, enhancing the expressiveness of the language.

2.2 Domain-Specific Languages

DSLs are programming languages that focus on a specific problem domain. They are often called *micro-languages* or *little languages*. Usually they are *declarative* and dealt as *specification lan-*

guages [29, 97, 191, 258]. The first DSL ever invented dates back in 1957. It was the APT language, which aimed to enhance programming process of numerically controlled machines [219]. The second oldest was invented in 1959 and it is the infamous Backus-Naur Form (BNF) [18].

2.2.1 Definitions

The *IEEE Standard Glossary for Computer Languages* [37] provides a definition for *application-oriented languages*, which can act as a premature definition for DSLs:

“An application-oriented language is a programming language with facilities or notations applicable primarily to a single application area; for example, a language for computer-assisted instruction or hardware design.”

Deursen et al. [258] proposed the following definition,

“A domain-specific language (DSL) is a small, usually declarative, language that offers expressive power focused on a particular problem domain. In many cases, DSL programs are translated to calls to a common subroutine library and the DSL can be viewed as a means to hide the details of that library.”

He also proposed a secondary definition,

“A domain-specific language (DSL) is a programming language or executable specification language that offers, through appropriate notations and abstractions, expressive power focused on, and usually restricted to, a particular problem domain.”

2.2.2 Software Development Process and Domain-Specific Language Usage: Advantages and Disadvantages

This section enumerates the advantages and the disadvantages of DSL [97, 106, 187] usage in the software development process. They divide into two distinct categories; those that affect the *Software Development Process* and those that can be considered purely as *Technical*. Figure 2.3 summarises the advantages / disadvantages for each category.

Software Development Process

The *Software Development Process* category focuses on software development cost and development team productivity. It is the view that a manager of a software project should have, to evaluate possible DSL adoption in the software development process.

Advantages

- **Enhanced Productivity** DSLs enhance productivity, reliability, maintainability and portability [97, 128, 187, 237, 238, 242, 257].
- **Embodied Domain Knowledge** DSLs provide annotations that allow direct representation of the domain knowledge. Thus, all software design can be partially performed at the application domain level, enabling opportunities for better overall design [24, 106, 238, 262, 268].

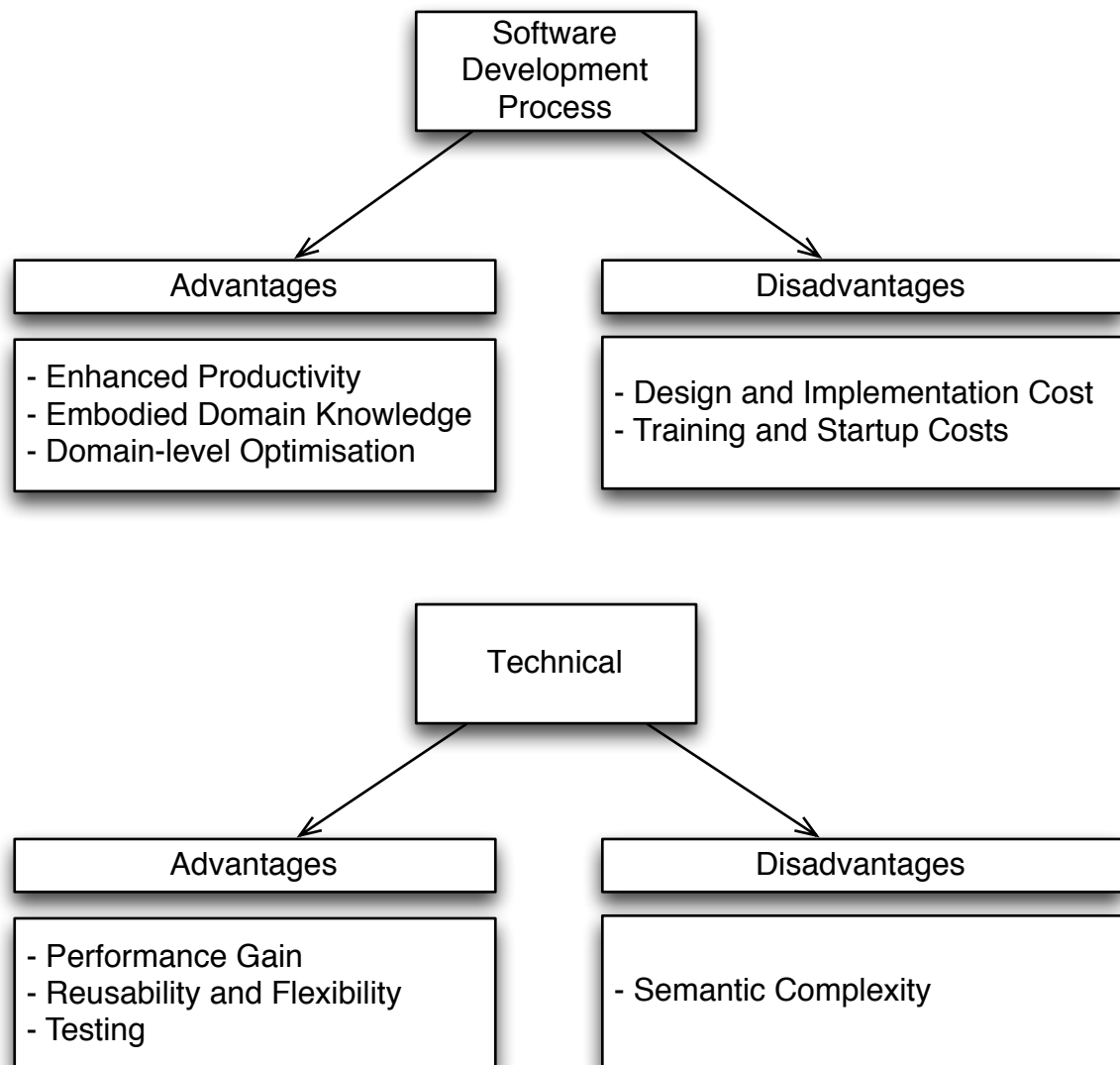


Figure 2.3: Advantages and disadvantages of DSL usage

- **Domain-level Optimization** Experts from each problem domain have the opportunity to provide validation of the implementation and introduce further domain-specific optimisations [97, 141, 216, 238].

Disadvantages

- **Design and Implementation cost** The cost of design and implementation of a DSL is usually high [12, 97, 106, 155, 187, 191].
- **Training and Start-up Costs** The developers must learn the semantics and the bindings of the DSL, to gain expertise and become productive [97, 106, 131, 223, 239].

Technical

The *Technical* category focuses on the technical / engineering issues and how the overall usage of DSLs can help the software development process from a more developer-centric perspective.

Advantages

- **Performance gain** DSL usage also provides a solid foundation for Domain-Specific Optimisation (DSO)s [106, 187, 238, 260]. This second level of optimization is created providing more opportunities for enhancing computer programs' performance in terms of code execution, security enhancements etc.
- **Re-usability and Flexibility** DSLs programs are concise and can be reused for different purposes [13, 57, 124].
- **Testing** The use of DSLs enables and encourages testability following domain-testing approaches [208, 284].

Disadvantages

- **Semantic Complexity** The difficulty of balancing between domain-specificity and GPL constructs [106, 238].

2.3 Design and Implementation of DSLs

The design and the implementation of DSLs and their integration patterns with GPLs is an active topic of research for several decades [12, 150, 187, 258]. Various tools have been implemented in order to provide an environment to design, implement, and use them [27, 89, 118, 158, 158, 203, 229, 249, 263].

The basic design process of designing and implementing a DSL, involves the following phases (Figure 2.4):

1. **Identify the problem domain** The preamble of the design phase is the identification of the problem domain. DSLs have very narrow scope and high expressiveness, so the correct definition of the problem is crucial for a successful design and implementation.
2. **Theoretically design semantics** The semantics of the proposed DSL should be theoretically designed.

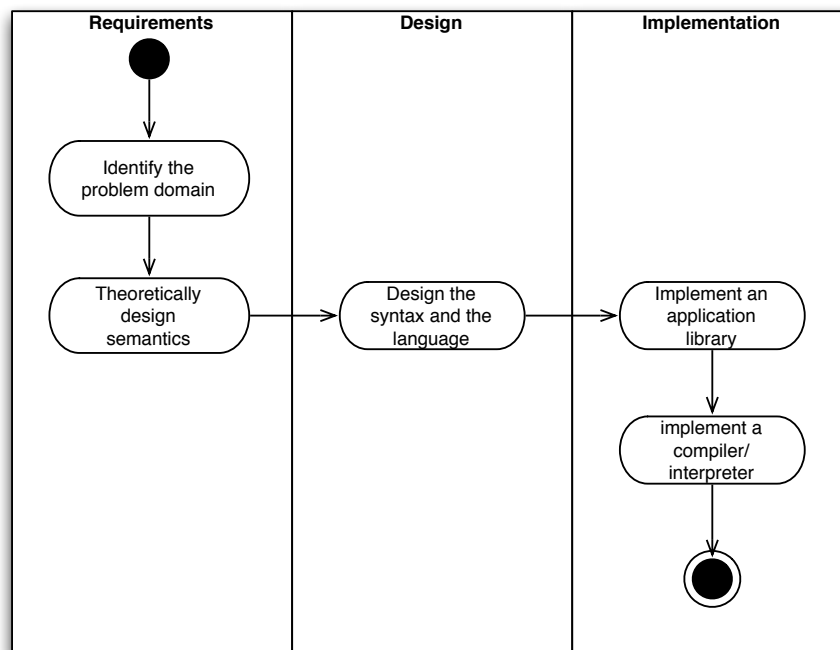


Figure 2.4: DSL Design and Implementation

- 3. Design the syntax and the language** Define the syntax and the core language characteristics. This task can be completed using lexical and syntax tools [139, 140, 149, 167].
- 4. Implement an application library** Construction of a library (dynamically or statically linked) that implements the DSLs' low-level functionality.
- 5. Design and implement a compiler / interpreter (optional)** Design and implementation of a compiler or interpreter according to usage. Many implementations stop in the previous step, providing only bindings with series of programming languages. For example, regular expressions are always part of bigger tools, and there is no meaning to use them stand-alone.

The above design and implementation process provides a straightforward way to implement any DSL, mostly following the *Implementation:Embedding* Pattern [187]. More sophisticated design efforts are usually using metaprogramming and generative programming techniques, which are discussed in Section 2.5.

2.3.1 Classification of DSL Patterns

Regarding DSL development and integration with GPLs, many relevant methodologies appear in the scientific literature [187, 238]. Those are categorised in four (4) patterns; *Decision*, *Analysis*, *implementation*, and *Design*. Figure 2.5 illustrates those patterns along with their sub-patterns [187].

In Section 2.4, popular programming languages are examined and categorised according to the design patterns that are listed in Table 2.1. The patterns listed are the most popular implementations. The following notation is used; the pattern name is defined as a classification / specialisation combination e.g. *Design: Language Exploitation (extension)* is referred as the *Language Exploitation (extension)* specialisation of the *Design* DSL integration pattern.

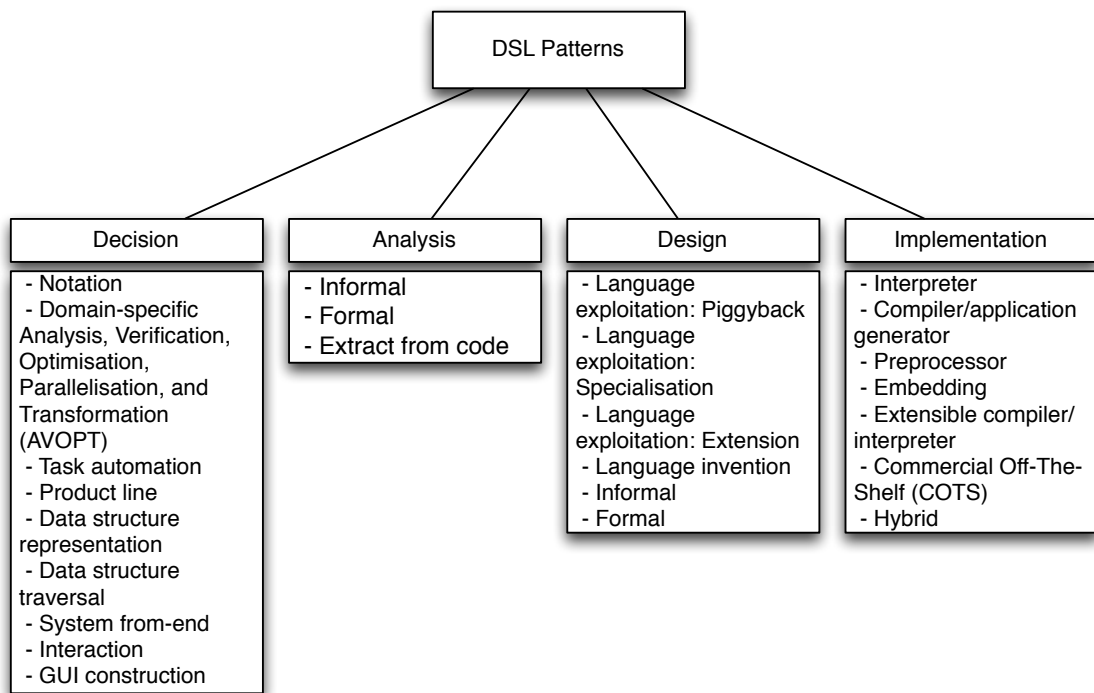


Figure 2.5: DSL Patterns as in reference [187]

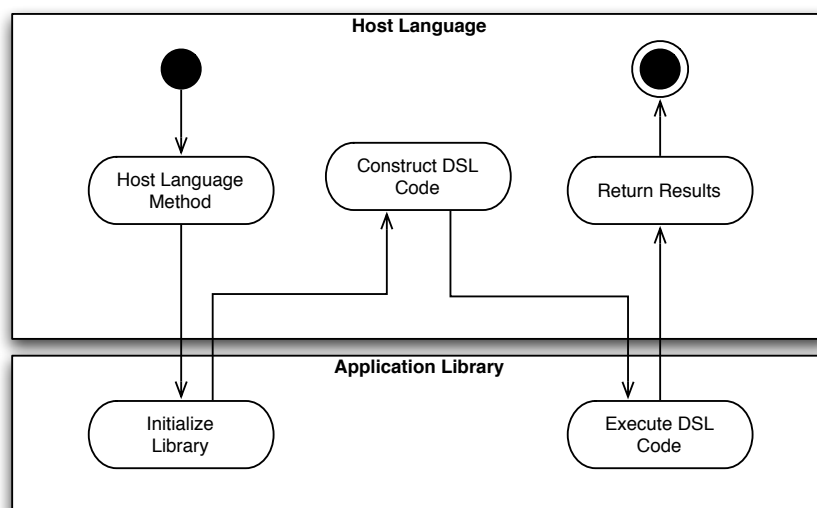


Figure 2.6: Activity Diagram for Implementation: Embedding Pattern

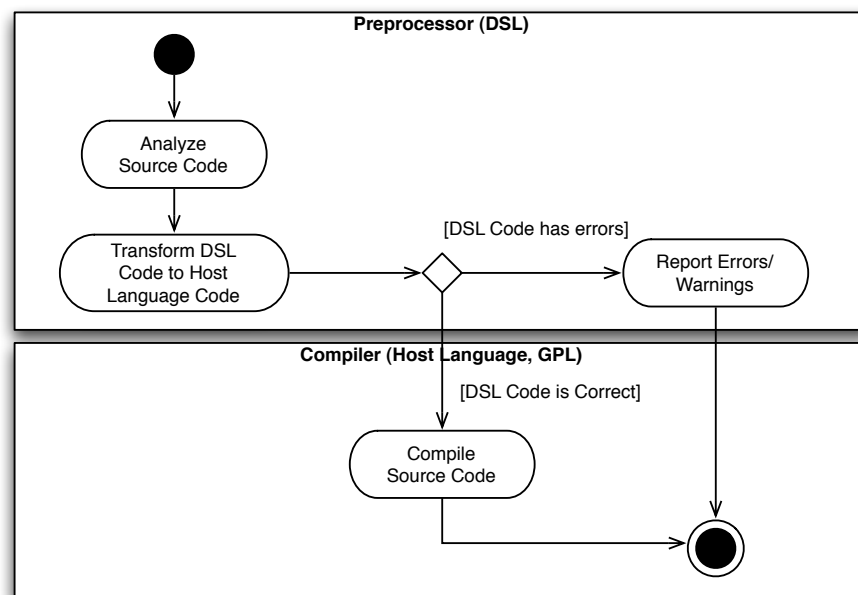


Figure 2.7: Activity Diagram for Implementation: Preprocessor Pattern

Table 2.1: Dominant design and implementation patterns for DSL integration

Pattern	Description
Design: Language Exploitation (extension)	The DSL extends an existing language with new data types, semantic elements, and syntax
Design: Language Exploitation (specialisation)	The DSL restricts an existing language for purposes of safety, static checking, and optimisations
Implementation: Extensible-Compiler/Interpreter	A GPL compiler / interpreter is extended with domain-specific optimization rules and domain-specific code generation
Implementation: Embedding	DSL constructs are embedded in an existing GPL (the host language) by defining new abstract data types and operators
Implementation: Preprocessor	The DSL syntax is translated to constructs in an existing, host language

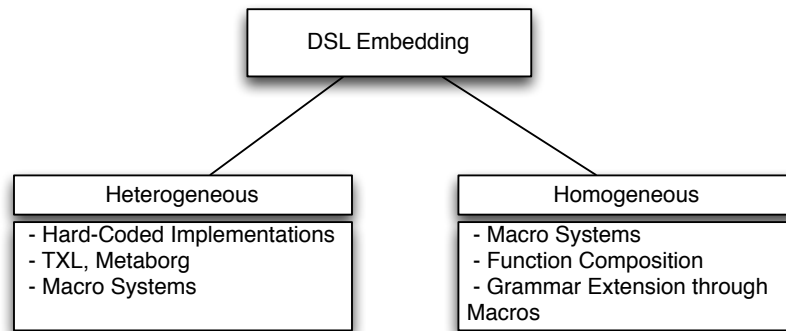


Figure 2.8: DSL embedding approaches according to Tratt [252]

2.3.2 Other Approaches

The main issues of DSL embedding relate with the integration and the proper interaction of DSL code with the host language, without having to forfeit the specific domain's established notation. Tratt [252] states that two approaches can be distinguished (Figure 2.8); *heterogeneous* embedded and *homogeneous* embedded DSLs.

Heterogeneous embedded DSLs require in general a pre-processor to analyse and transform the program code. After that, the compiler of the GPL compiles the code as usual. The pre-processor rewrites DSL expressions into executable GPL code.

Homogeneous DSLs are application libraries that provide domain-specific constructs. These DSLs are developed in the GPL in which they are embedded. This method is similar with the one described in Mernik's classification [187] as *Implementation: Embedding*. DSL support in the Scala programming language follows this approach.

In the 24th European Conference of Object-Oriented Programming (ECOOP), Renggli et al. [214] also presented three fundamental types that categorize language embedding. These are:

Pidgins A *pidgin* extends the semantics of the host language. The embedded language uses partially the syntax of the host language, along with new, but rather limited, constructs.

Creoles A *creole* introduces a new grammar and then transforms the code to the host language compatible code.

Argots An *argot* uses the existing host language syntax, but changes its semantics.

2.4 Case Studies of DSL Integrations

This section presents several DSL-GPL integration efforts, categorised according to the dominant design patterns [187] presented in Section 2.3.1. Figure 2.9 summarises those patterns along with a few representative implementations.

2.4.1 Design: Language Exploitation (extension)

Perl [273] introduces a series of operators to efficiently use regular expressions within the language syntax. For example, the `m//` is the matching operator for regular expressions. The following program examines if the input is a number and prints a message accordingly on success.

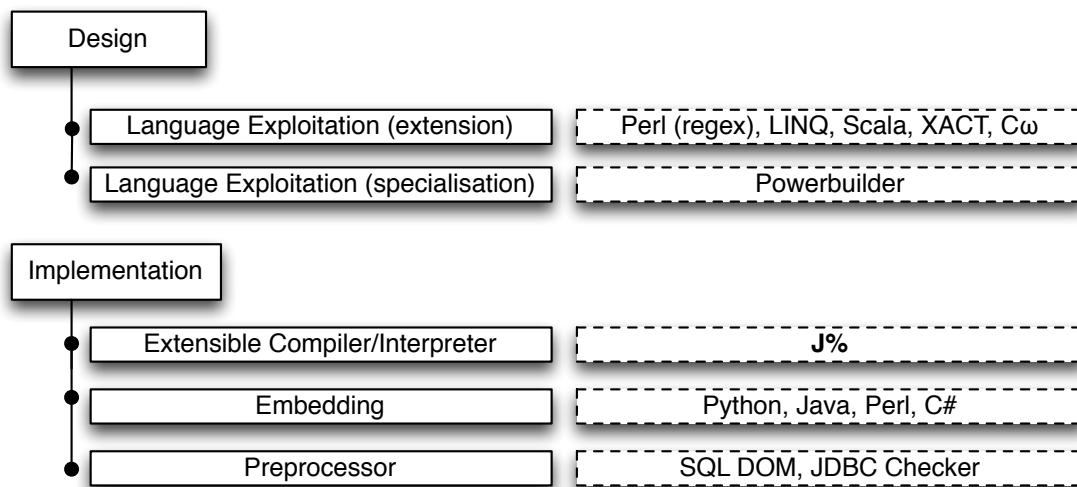


Figure 2.9: Popular DSL integrations categorised by design pattern

```
while(<>) {
  if(m/[0-9]{1,3}/){ print "number"; }
}
```

Language INtegrated Query (LINQ) is an integrated component to the .NET languages (C#, Visual Basic and Visual C++) that adds querying capabilities. In addition, it can be used to declaratively perform queries in collections of objects. The LINQ to SQL provider allows it to be used to query databases. It converts a LINQ query to a SQL query, which is later sent to the database server.

Haskell/DB [164] is a host language variant that has been extended to encapsulate SQL queries. This approach completely hides the DSL from the developer, hindering productivity and forbidding domain experts to become involved with the development process.

An interesting case study of DSL integration is the *Boo*¹ programming language [69]. Based on the .NET framework *Boo* provides a python inspired syntax and features *String Interpolation* and support for *regular expressions* with a similar syntax to the Perl approach. The following example illustrates the usage of regular expressions in Boo:

```
for line in StringReader(stdout):
  match line:
    case /^[0-9]+$/:
      print "Found a digit!"
```

*Felix*² uses specific types and syntax for regular expression pattern matching support. In the following example, the `regexp` type is used to define regular expressions that match numbers and identifiers.

```
include "std";
regexp digit = ["0123456789"];
regexp alpha = lower | upper | "_";
regexp id = alpha (alpha | digit) *;
```

¹<http://boo.codehaus.org/>

²<http://felix.sourceforge.net/>

XML Enhancements for Java (XJ) [121] integrates XML notation into its syntax, thus providing static type checking. It also takes into account additional data types from a given XML Schema. It is consistent with the XML standards and uses XML Schema Definition (XSD) for strong typing at compile-time. Programs are written directly in terms of XML and not with the usage of existing APIs, like DOM or Simple API for XML (SAX).

XACT [152] tries to determine possible dynamic generated XML statements at compile-time and provide error checking. It features optional type annotations using XML Schema declarations and performs static analysis to ensure type safety.

C ω [33] integrates both SQL and XML into its syntax, extending the C# programming language introducing numerous changes to the type system and C# syntax. The listing below illustrates a simple query in XML data, using an SQL-like statement.

```
select <book-with-prices>
    <title>{a.title}</title>
    <price-A>{a.price}</price-A>
</book-with-prices>
from book a in A.book, book bn in BN.book where a.title == bn.title
```

SQLJ [81] is a language extension of Java to efficiently support SQL. Maybe it is the only implementation that is standardized with an International Organization for Standardization (ISO) (ISO/IEC 9075-10) [133]. This integration is complete; it supports type checking and syntax check for Java and SQL at compile-time. The following listing exhibits the usage of an SQL `select` statement.

```
#sql [ctx] iter = {
    SELECT LASTNAME, FIRSTNAME, SALARY
    FROM DSN8710.EMP
    WHERE SALARY BETWEEN :min AND :max
};
```

Scala [173] supports XML natively through syntactic enhancements of the language. For example:

```
class AddressBook(a: Person*) {
    def toXHTML =
        <table cellpadding="2" cellspacing="0">
            <tr>
                <td>Name</td>
                <td>Age</td>
            </tr>
        </table>;
}
```

Scala is designed with language extension and metaprogramming in mind. The extensions that can be written are Scala code that is disguised as a hosted DSL through various syntactic tricks. For example, Scala syntax permits the omission of parentheses (``('` and `)``) and the dot (``.``) for a simple method call with one parameter. A typical example is the following `this.is(true)` could be written also as `this is true`. With the usage of syntactic idioms and clever definition of classes, objects (singleton classes) and methods, Scala developers can create extensions that provide domain-specific syntax, with certain limitations of course, but with no real language extension at all.

2.4.2 Design: Language Exploitation (specialization)

Powerscript is the core development language for the Rapid Application Development (RAD) tool *Powerbuilder*³. It is a specialisation of the BASIC programming language that supports integration with SQL. This is a classic example of *Creational: Language Specialization* pattern, where a GPL, BASIC in this instance, is restricted syntactically to a specialised development language for database applications. Powerscript provides syntax and type checking in the integrated SQL queries. The following example exhibits a parameterised SQL *insert* statement:

```
int Sal_var
Sal_var = 18900

INSERT INTO EMPLOYEE ( SALARY ) VALUES ( :Sal_var );
```

2.4.3 Implementation: Extensible Compiler/Interpreter

Research works that have been using this integration method are found in metaprogramming frameworks, which are thoroughly analysed in Section 2.5. Various research works in Scala are also adding metaprogramming and DSL embedding with the usage of this pattern.

[196] provides a general way to integrate a generic query language into Scala syntax, thus enabling support for a series of DSLs. The current method is very well designed, but it is tightly coupled to Scala language features.

Generative programming is used to apply DSOs in Scala programs using the Lightweight Modular Staging (LMS) [218]. The generated program is pure Scala, but with LMS higher-level abstractions were provided to the developer, making it easier to apply domain-specific knowledge. In addition, the components that implement them are highly re-usable.

[46] introduces macros for the Scala programming language. This library is merged to the main distribution of the language, and it provides a series of case studies, on how they can be used to embed external DSLs. The concept of Scala macros is simple; functions can be provided that will evaluate at compile-time. These functions have access to the program's Abstract Syntax Tree (AST) and can perform code generation, complex type checking, syntax validation, etc.

Dubochet in his thesis [74] describes two embedding techniques for the Scala programming language. The first allows embedding of domain-specific syntax in Scala code. The second introduces an implementation of structural types for the Scala programming language [75] and uses them to enforce domain-specific correctness properties on Scala programs.

The J% language extension [146], which is presented in this thesis, focus on any language integration with Java proposing a generic way to work with and follows this DSL implementation pattern.

2.4.4 Implementation: Embedding

Python, Java, Perl, C# and all the mainstream programming languages use the *Implementation: Embedding* as the standard method to host DSLs into their code. This pattern proposes that a DSL application library should be implemented. The DSL code is passed as a *String* to it and then executes it. Consider the following code, which opens a database file and then executes a simple *insert* statement.

```
import sqlite
```

³<http://en.wikipedia.org/wiki/PowerBuilder>

```

con = sqlite.connect('mydatabase.db')
cur = con.cursor()
insert_value = "some text"
cur.execute("INSERT INTO foo (o_id, desc) VALUES(NULL, '%s')" % (insert_value,))

```

Object-relational Mapping (ORM) approaches like Hibernate,⁴ Java Persistence API (JPA),⁵ Scala Language-Integrated Connection Kit (SLICK),⁶ and Squeryl⁷ provide a way to access a relational database, by introducing a programming layer, typically supported by a custom DSL, or by extending the host language, in order to automatically generate and execute the SQL code. This approach has several advantages; the database entities are closely coupled with the host language and the application is completely decoupled by any database backend. The main disadvantage of this approach is that the programmer needs to learn a new query language. In addition, this language is expressed via strings; thus it suffers from the same problems related with the use of plain JDBC.

Anorm⁸ is another Scala integration approach that enables database access within the Play! Web application framework. Anorm adopts the principle that SQL has more than enough features and language richness to cover all database access uses cases that an application could want; thus it offers a Scala library that enhances SQL integration, simplifying common usage patterns via a sophisticated API.

Java Object Oriented Query (JOOQ)⁹ is a Java application library and it offers a unique API that maps all database elements into Java code. This is realised with a code generation utility. This utility scans the target database and generates all the models and the code that represent the database as host language types. The generated code uses standard JDBC API calls. The advantage of this method is the direct mapping of each database table and field to a Java program element. For example, it is impossible for a programmer to mistype a table name or any other SQL identifier, since it is directly mapped into a Java class and the compiler. Hence, the Integrated Development Environment (IDE), will catch the error and present it to the user.

Similar approaches in C++ include Object Database (ODB)¹⁰, and Database Template Library (DTL).¹¹

Simple Oracle Call Interface (SOCI) [233] is a C++ library that provides advanced functionality when working with SQL. Technically, it is a C++ extension, but it provides a series of techniques, only by using features of the standard C++ language, to enable SQL embedding; among them, basic query support and basic ORM capabilities.

The following code illustrates an SQL query with ODB,

```

int id = ...;
string name;
int salary;

sql << "select name, salary from persons where id = " << id,
      into(name), into(salary);

```

⁴<http://www.hibernate.org/>

⁵<http://www.oracle.com/technetwork/articles/javaee/jpa-137156.html>

⁶<http://slick.typesafe.com/>

⁷<http://squeryl.org/>

⁸<http://www.playframework.com/modules/scala-0.9.1/anorm>

⁹<http://www.jooq.org>

¹⁰<http://codesynthesis.com/products/odb/>

¹¹http://dtemplatelib.sourceforge.net/dtl_introduction.htm

2.4.5 Implementation: Preprocessor

The *Implementation: Preprocessor* approach requires that the DSL syntax should be translated to the existing host language. Typical implementations include the preprocessors of the C and C++ programming languages, where the macros, which act as the DSL for this case, are translated into each language respectively.

JDBC Checker [111] acts as a preprocessor and searches Java code for JDBC calls and SQL statements to detect possible errors. Notably, the SQL statements are checked against the database schema. Annamaa et al. [14] uses Eclipse's compiler infrastructure to efficiently embed SQL queries in Java programs. The plug-in can detect common syntax SQL errors, including misspelled table or column names. To embed DSLs, Erdweg [85, 86, 87, 88] proposes SugarJ, a framework used to extend GPLs. The main contribution of this framework is that can be applied on many languages as host languages. The prototype implementation supports Java, Haskell and Prolog.

JSquash [228] is a tool that analyses Java code and several by-products of a Java application and trace the existence of SQL statements. Then, it automatically links the dynamic parts of SQL queries with the variables of the program. Typically, the tool aims to identify this linkage and replace the contents of these variables accordingly, when the database schema changes.

SQL DOM [182] acts as a preprocessor and translates an SQL database schema into C#. The generated collection of classes is used as an application library, thus ensuring type safety and syntax checking at compile-time.

Switch [116] is a compiler that can be used when developing in the Ruby programming language. Its goal is to provide an alternative for the Ruby on Rails active record library, focusing on query performance.

2.5 Metaprogramming and Language Transformation Tools

Metaprogramming permits programs to generate programs and is a vital part of modern languages and frameworks. Code generation techniques are used for various ends; including compile-time optimisations [127, 181, 271], software testing [53], and code quality improvement [277]. This section begins with the definitions of *metaprogramming*, *generative programming* and *generative metaprogramming*, and will continue our analysis with program transformation systems and finally present the metaprogramming facilities of C, C++, and Java.

Metaprogramming¹² [61, 99] is defined as “*the writing of computer programs that write or manipulate other programs (or themselves) as their data, or that do part of the work at compile time that would otherwise be done at runtime.*”

Generative Programming is defined in reference [65],

“*A software development paradigm based on modelling software system families such that, given a particular requirements specification, a highly customized and optimized intermediate or end-product can be automatically manufactured on demand from elementary, reusable implementation components by means of configuration knowledge. The generated products may also contain non-software artifacts, such as test plans, manuals, tutorials, maintenance and troubleshooting guidelines, and so on.*”

¹²<http://en.wikipedia.org/wiki/Metaprogramming>

Trujillo et al. [254] combined the above terms and defines also “generative metaprogramming”,

“Generative programming is about metaprograms that synthesize other programs. Metaprogramming is the concept that program synthesis is a computation. This work describes an approach to generate metaprograms, which when executed, will synthesize a target program of a product line. We name this process generative metaprogramming.”

2.5.1 Program Transformation Systems

Visser in references [264, 267] enumerates all the rule-based strategies that are used for program transformations. These strategies provide the basic elements that are used from program transformation systems. Visser states in reference [267] that rewrite rules are context-free can only access information through pattern matching the term to which the rule is applied and this can act as a straightjacket. We will see in Section 4.1 that our approach provides a solution that addresses this problem, by focusing on the program context and not only on “blind” rewrite rules. An interesting case study of program transformation is also presented in references [126] and [125]. Another survey of metaprogramming frameworks was published in Principles of Programming Languages (PoPL) (2006) [17]. One could also find these earlier works interesting [171, 232]. Turing eXtender Language (TXL) [60] is another program transformation system that was developed by J.R. Cordy.

Stratego [263, 266] is one of the most popular program transformation systems, with much usage in various research works [38, 39, 216, 232, 259, 265]. WebDSL [268] is a web framework that was developed using stratego and serves as an example, on how efficiently DSLs can be developed by using a rich program transformation system. Usually a program transformation system has a compiler that accepts as input the program with the transformation rules and the grammars. Upon compilation, the source is transformed and compiled by a standard GPL compiler i.e. Java.

J% cannot be compared directly with these systems, only utilise them as the basis for its design and its implementation. It offers a specific way for DSL integration with a specific syntax and features. Program transformation systems deal with that problem at syntax level, providing a way to extend grammars and provide new transformations. The aforementioned systems partially solve the problem exactly at that level, often disregarding types checking between the languages. In addition, these systems require the implementation of complex grammar rules for each DSL that they need to support.

2.5.2 Metaborg, Converge and Other Approaches

Metaborg [40, 216] is an approach based on Stratego/XT [266] and is used for programming language embeddings. With metaborg a method is presented that can embed language within language at the syntax level. The hosted language is translated through program transformations into the host language, using existing application libraries, or by specific code generation targeting optimised performance, security features and others. In this research, several case studies are presented, exhibiting various embeddings, even Java within Java. Metaborg method supports type checking between the host and the embedded language, but not in a structured and concise way, type checking rests in the hands of the implementer of each DSL embedding. Visser states in reference [267] that one of the problems of program transformation is the usage of pattern matching to extract information regarding the program and finally transform it. Recent research indicates that the type reflective metaprogramming systems appeared in references [102, 156, 157].

The following code [40] illustrates a modified Java syntax that introduces inline regular expression support and a regular expression match operator (`~?`), which is used as in the `if` statement.

The code is *source-to-source* translated into plain Java code and compiled by the standard compiler that is distributed with the SDK. The generated code uses the regular expression library, which is shipped with the standard Java Development Kit (JDK) package `java.util.regex`¹³.

```
regex ipline = [/
    ( ( [0-1]? \d{1,2} \. ) | ( 2[0-4] \d \. ) | ( 25[0-5] \. ) ){3}
    ( ( [0-1]? \d{1,2} ) | ( 2[0-4] \d ) | ( 25[0-5] ) )
/] ;

if( input ~? ipline )
    System.out.println("Input is an ip-number.");
else
    System.out.println("Input is NOT an ip-number.");
```

Tratt in reference [252] shows how any DSL could be embedded into Converge programming language, by using the concepts of *language blocks* and compile-time metaprogramming. He also presents a specialised mechanism to detect and report errors.

A similar approach is presented with the Helvetia [214] system; an extensible system of the Smalltalk language to easily integrate language extensions. This approach is based on the on the concept of *Language Boxes*, which are presented in reference [213]. The host language (Smalltalk) is extended through a series of specifications, and the program transformation is implemented through various hijacks in the compiler pipeline.

J% is similar in some ways with these approaches, by defining a common container, a *Language Box* [214] that provides the means for efficient DSL embedding. In addition, it presents a standard method for type checking between the host language and each DSL.

2.5.3 The C Preprocessor

The American National Standards Institute (ANSI) C [134] supports a simple method for metaprogramming, which is implemented with a preprocessor, and offers search and replace facility, along with various directives that provided basic flow control. The following snippet lists a macro that generates and defines four (4) functions, that turns on / off Light Emitted Diode (LED)s in a hardware component. Note that the code generates functions, which can be referenced and used by the application, thus exhibits the features of the preprocessor and how it can be used as a basic metaprogramming system.

```
#define LED_CONF(led, port, pin) \
static CC_INLINE void led_##led##_set_output() { P##port##DIR |= (1 << pin); } \
static CC_INLINE void led_##led##_on()          { P##port##_##pin = 1; } \
static CC_INLINE void led_##led##_off()         { P##port##_##pin = 0; } \
static CC_INLINE void led_##led##_toggle()      { P##port##_##pin ^= 1; }
```

Thus, the following code

```
LED_CONF(1, 0, 3)
```

```
int main(int argc, char** argv) { return 1; }
```

will be finally expanded to,

¹³<http://download-llnw.oracle.com/javase/6/docs/api/java/util/regex/package-summary.html>


```
static CC_INLINE void led_1_set_output() { PODIR |= (1 << 3); }
static CC_INLINE void led_1_on() { P0_3 = 1; }
static CC_INLINE void led_1_off() { P0_3 = 0; }
static CC_INLINE void led_1_toggle() { P0_3 ^= 1; }

int main(int argc, char** argv) { return 1; }
```

2.5.4 C++ and Template Metaprogramming

C++ templates [70, 206] is maybe the most famous and influential metaprogramming system [6], both in academia and the industry. It is considered standard feature of C++ and with the Standard Template Library (STL) that provides a series of containers and a collection API for C++. The templating system is also used for metaprogramming as presented by Alexandrescu [10]. The following listing contains a program written using the C++ template system.

```
#include <iostream>

template <typename T>
const T& max(const T& x, const T& y) {
    if(y < x)
        return x;
    return y;
}

int main() {
    std::cout << max(3, 7) << std::endl;

    return 0;
}
```

The above listing defines the method `max`, which is generalised using a template to support various types, such as `int` and `double`.

Templates are used also for compile-time optimizations and pre-calculations. For example, the following code can calculate factorial at compile-time with a standard template initialisation, as `Factorial<4>::value` for `4!`. This feature, if used properly can offer great calculation speed-ups for applications, while keeping the program in generic form, without defining constants.

```
template <int N> struct Factorial {
    static const int value = N * Factorial<N - 1>::value;
};

template <>
struct Factorial<0> {
    static const int value = 1;
};

template <>
struct Factorial<1> {
    static const int value = 1;
};
```

2.5.5 Java Annotations & Python Decorators

Annotations were introduced in Java syntax since version 1.5, as a standard method to provide compile-time plug-ins to the Java programming language. They are used mostly to generate code at compile time and through them a number of compiler plug-ins can be developed. Maybe the most famous annotation for Java programmers is the `@Override`, which indicates that the method is overridden from a parent class.

```
public class Example {
    @Override
    public String toString() { return "example override"; }
}
```

Type annotations¹⁴ [66] were introduced in JSR 308 and offer enhancements in Java's type system at compile-time. For example, `NonNull String str;` declares that reference `str` cannot be `null` and a compile-time error should emerge if so.

Python has a similar mechanism called decorators. These are used like annotations, and they are executed at runtime, since Python is a dynamic programming language. The following code exhibits decorator usage:

```
def hello_world_decorator(fn):
    def hw_inside(*args, **kwargs):
        print "Hello world from decorator!"
        return fn(*args, **kwargs)

    return hw_inside

@hello_world_decorator
def test_function():
    print "Hello world!"

test_function()
```

2.5.6 Extensible Compilers

Extensible Compilers can be describes as open tools that provide entry points to extend and change a host language. Java Annotation Processing Tool (APT) enables a compile time, read-only view of the Java program structure. ASTs can be transformed only using a private API, which is not supported by a standard. Dryad [148], JastAddJ [83], and Polyglot [199] are extensible Java compiler frameworks. Usually, language extensions are modular, and the extension code is transformed into Java. Most extensible compilers define the syntax changes using an external language definition. None of the systems offers IDE integration, and the generated code cannot be easily debugged. eXtension-Oriented Compiler (XOC) [62, 63] is an extensible C compiler. It uses a source-to-source translator that reads the input, analyses and transforms it, to eventually generate standard C code. An interesting definition is *Active Libraries* [261], which categorises language embedding approaches that can act at compile time, to define abstractions, and also provide the means to control them. eXTensible Compiler (XTC) [103, 114] is an extensible compiler that can be used to create and test different concepts in programming languages. Based on this work, various tools have been constructed, like universal macro system for programming languages [162], and mixed-environment debugging systems [161].

¹⁴<http://types.cs.washington.edu/jsr308/>

2.5.7 Language Boxes

In reference [213], Renggli et al. define *Language Boxes*, a language model which can support embedding of DSLs into a host programming language. They present a case study, which is implemented in Smalltalk and show how regular expressions can be efficiently supported with a small extension of the base language grammar. Practically, the grammar is extended, and the compiler can understand the embedded language through the special notation, and compile it into a host language program, or transform it depending the needs of the software project. For example, in Smalltalk a literal is defined by the following rule:

```
Literal ::= String / Number / Boolean
```

To support regular expressions, this rule should be extended as follows,

```
Literal ::= String / Number / Boolean / Regexp
```

where *Regexp* is defined as a rule which is that recognises a regular expressions defined by the boundary `<regex>/`. To achieve those goals, *Languages Boxes* use executable grammars to enable the language changes.

A similar approach is implemented in Haskell in reference [177]. A *language box*-like technique is implemented to embed DSL into Haskell syntax.

2.6 Discussion

The case studies presented, support only one or two DSLs, except metaprogramming systems like Metaborg and TXL, which act as methods of language embedding at syntax level. The syntactic flexibility enforces a practical overhead; significant effort is required to embed any DSL and the developer has to figure out a way to support many DSLs. In that manner, these systems could be used to implement J%, which is a complete solution that allows DSL embedding, without having to intervene to the grammar of the host language nor the DSL, each time that a new DSL needs to be supported.

Table 2.2 provides a summary for the implementations presented according to design patterns presented in reference [187]. [214] presents a research work that has significant similarities with J%, by utilising *language boxes*. All these approaches share a major disadvantage, in order to embed DSLs complex syntax notations should be specified. J% provides an efficient and elegant way to include all embedded DSLs by using a “*common container*” for all embedded languages, without demanding the definition of complex syntax notations (see Chapter 3).

For example, Perl efficiently integrates regular expressions by extending the language using specific operators (`m//`, `s//`), but for SQL it uses a standard application library (*Implementation: Embedding*) approach. Surprisingly, Perl does not perform syntax check on regular expressions and the programmer must use the `eval` function to provide error checking [55].

```
sub is_valid_pattern {
    my $pat = "[0-9*"; # erroneous regular expression
    return eval { "" =~ /$pat/; 1 } || 0;
}
```

The other case studies share the same characteristics. Only Scala provides a more flexible approach, in terms of language extension and provides incredibly efficient support for XML and regular expressions. Generative programming frameworks like Stratego [266] and TXL [60], provide

Table 2.2: Multiparadigm Languages Implementations

Implementation	GPL	DSL	DSL Pattern
Java	Java	regex, SQL	Implementation: Embedding
Python	Python	regex, SQL	Implementation: Embedding
ODB	C++	SQL	Implementation: Embedding
Squeryl	Scala	SQL	Implementation: Embedding
Slick	Scala	SQL	Implementation: Embedding
SOCI	C++	SQL	Implementation: Embedding
JPA	Java	SQL	Implementation: Embedding
DTL	C++	SQL	Implementation: Embedding
Hibernate	Java	SQL	Implementation: Embedding
Anorm	Scala	SQL	Implementation: Embedding
JOOQ	Java, Scala	SQL	Implementation: Embedding
Boo [69]	Boo	regex	Design: Language Exploitation (extension)
Haskell/DB [164]	Haskell	SQL	Design: Language Exploitation (extension)
JDBC Checker [111]	Java	SQL	Implementation: Preprocessor
Powerbuilder	Powerscript	SQL	Design: Language Exploitation (specialization)
XJ [121]	Java	XML	Design: Language Exploitation (extension)
XACT [152]	Java	XML	Design: Language Exploitation (extension)
C ω [33]	C#	SQL, XML	Design: Language Exploitation (extension)
Perl	Perl	regex	Design: Language Exploitation (extension)
Perl	Perl	SQL, XML	Implementation: Embedding
Switch [116]	Ruby	SQL	Implementation: Preprocessor
SugarJ [85]	Java	SQL	Implementation: Preprocessor
SQL Plug-in [14]	Java	SQL	Implementation: Preprocessor
JSquash [228]	Java	SQL	Implementation: Preprocessor
SQL DOM [182]	C#	SQL	Implementation: Preprocessor
J% [146]	Java	Any	Implementation: Extensible Compiler / Interpreter
LINQ [186]	C#	SQL, XML	Design: Language Exploitation (extension)
Felix	Felix	regex	Design: Language Exploitation (extension)
Scala [173]	Scala	XML	Design: Language Exploitation (extension)
Scala [173]	Scala	regex, SQL	Implementation: Embedding
Scala [173]	Scala	Any	Implementation: Extensible Compiler / Interpreter
Scala	Scala	ScalaQL	Design: Language Exploitation (extension)
SQLJ [81]	Java	SQL	Design: Language Exploitation (extension)
HELVETIA [214]	Smalltalk	Any	Implementation: Extensible Compiler / Interpreter

Table 2.3: DSL integrations score according to syntax checking, static typing and complexity

Project	Language	Syntax	Typing	Complexity
Java	Java	X	X	low
Python	Python	X	X	low
odb	C++	X	✓	low
Squeryl	Scala	X	✓	medium
Slick	Scala	X	✓	medium
soci	C++	X	X	medium
jpa	Java	X	✓	high
dtl	C++	X	✓	high
Perl	Perl	X	X	low
Hibernate	Java	X	✓	high
Anorm	Scala	X	✓	medium
jooq	Java, Scala	X	✓	high
Scala [173]	Scala	X	X	low
jdbc Checker [111]	Java	✓	✓	high
SugarJ [85]	Java	✓	✓	high
sql Detection Plugin [14]	Java	X	X	high
jsquash [228]	Java	✓	✓	high
sql dom [182]	C#	✓	✓	medium
Switch [116]	Ruby	✓	X	high
Haskell/db [164]	Haskell	X	✓	high
Cω [33]	C#	X	✓	high
siq [270]	Scala	✓	✓	high
linq [186]	C#	X	✓	low
sqlj [81]	Java	✓	✓	low
Powerbuilder	Powerscript	✓	✓	low
J% [146]	Java	✓	✓	medium

generic frameworks for source-to-source transformations and have been used in various research efforts.

J% is not a language development system like TXL [60], Jakarta Tool Suite (JTS) [27], and Khepera [89]. It provides an extensible way to embed DSLs into Java. So far, it facilitates existing Java DSL APIs, like JDBC [91] for SQL. But this is not the case, since a compiler module can translate any DSL code to the host language. The J% compiler could be implemented with any of these program transformation frameworks, since the modules are implemented with techniques that are supported by these tools.

Table 2.2 summarises the approaches and for every approach, provides information about two functional characteristics and one non-functional. These are:

Syntax (functional) The approach supports dsl syntax checking.

Typing (functional) Types are mapped between the dsl and the gpl and errors are detected at compile-time.

Complexity (non-functional) How complex is the approach for the average programmer that already knows the gpl. The complexity can be characterised as *low*, *medium*, or *high*. *Low* complexity denotes that a programmer with good knowledge of the base language, could use the dsl integration framework with reasonable effort and *high* means the exact opposite.

Chapter 3

Design Concepts of the J% Programming Language

*Two languages implementing the same idea must, on pain of death,
use different terms.*

Feldman's Law of Programming Terminology

In the previous chapter, DSL embedding methods and state-of-the-art research and practices were analysed. They were categorised according to classifications provided by various researchers [187, 213, 214]. All those approaches try to address the same problem, which is

How can Domain-Specific Languages efficiently be embedded in General-purpose Programming Languages?

It is difficult to define the term *efficiency*; to better formulate this research, a series of functional and non-functional requirements are specified and a series of research goals are introduced.

3.1 Design Strategy

This section sets the basic design goals of a generic system that embeds DSLs with GPLs. These goals are realised in the form of functional and non-functional requirements [234].

3.1.1 Requirements

In Chapter 2, we presented a series of case studies regarding DSL integration in GPLs, along with popular metaprogramming frameworks and several other facilities in various languages. All these approaches have partly common goals and use similar methods to address the issue. The list that follows contains a set of functional characteristics, which are derived from the research work we discussed earlier in this chapter.

DSL Syntax Validation The DSL should be syntactically checked at compile-time. Most case studies presented fulfilled this requirement, usually only for one DSL.

Static Typing The language embedding should respect each language's type system and ensure type safety. We must ensure at compile-time that the values exchanged between the GPL and the DSL are compatible, and the actual values are proper.

Extensibility A significant problem in all the ongoing research and industry systems is the lack of extensibility. The majority of the implementations support efficiently one or two DSLs with operators or any other extension as annotations, but this is it. They cannot support embedding of any other language in an efficient manner. The systems that support many DSLs, are tightly coupled with one specific host language.

In addition to the aforementioned functional requirements, a set of non-functional ones should also be defined. Those requirements are derived from existing works [97, 106, 187].

Semantic Complexity The proposed solution should not increase the semantic complexity of the host language. In practice, it is not possible to add functionality without some form of extension, but, on the other hand, the proposed extensions should be modest and simple. In short, the *Principle of Least Astonishment* [35] should be followed, which states,

“The Principle of Least Astonishment (POLA) states that, when two elements of an interface conflict, or are ambiguous, the behaviour should be that which will least surprise the user; in particular a programmer should try to think of the behaviour that will least surprise someone who uses the program, rather than that behaviour that is natural from knowing the inner workings of the program.”

Domain Models Maintenance Each DSL must maintain its syntax. One of the major advantages of using DSLs in software development is that the language illustrates the domain model, permitting the efficient development of applications on those domains.

Re-usability The proposed solution should encourage re-usability. This requirement is expressed with two facets; enforcing the re-usability of DSL code, which is repetitive and enhance the usage of existing DSL application libraries.

Domain-Specific Optimisations When the DSL code is analysed, it is an excellent opportunity to induce DSOs. These optimisation could lead to better performance in terms of execution speed, security enhancements, etc.

Figure 3.1 correlates the above requirements with the characteristic of quality model defined by ISO/IEC 9126 [94]. The requirements cover almost all the ISO's characteristics.

3.1.2 Key Elements for Efficient Language Mixing

According to the research work and the various implementations that were described in Chapter 2, we isolated the following key semantic and technical features that must be taken into consideration, to efficiently integrate GPLs with any DSL.

DSL Language Code Blocks Recent research approaches, such as those cited in references [213, 214, 253] show that a successful solution should provide a way to isolate the DSL code from the host language. The addition of special DSL-oriented operators must be avoided, unless it is crucial to provide refined support for a specific DSL. The integration of Perl with the regular expressions is the perfect example for the latter. These operators provide a very efficient way to use regular expression patterns, but this is the end of it. Other languages cannot be integrated without introducing syntax noise in the host language. Language code blocks constitute the perfect trade-off, since they permit native DSL syntax, without breaking language syntax.

		Functional			Non-Functional			
		DSL Syntax Validation	Static Typing	Extensible	Semantic Complexity	Maintain Domain Models	Resuability	Domain-Specific Optimisations
Functionality	Suitability							
	Accuracy							
	Interoperability							
	Security							
Reliability	Maturity							
	Fault Tolerance							
	Recoverability							
Usability	Understandability							
	Learnability							
	Operability							
	Attractiveness							
Efficiency	Time Behaviour							
	Resource Utilisation							
Maintainability	Analyzability							
	Changeability							
	Stability							
	Testability							
Portability	Adaptability							
	Instability							
	Co-Existence							
	Replaceability							

Figure 3.1: DSL embedding requirements correlated with the basic aspects of ISO/IEC 9126 quality model

Special Types Specialised types should be also be defined to aid the compilation process through compile-time hooks. These types can give the compiler information that will initiate specific code generation and validation mechanisms at compile-time. They should be able to be defined arbitrarily and not burden semantically the language constantly. Such approaches are used in implementations like *Anorm*¹ and *Felix*².

Configuration Hooks Compile-time configuration hooks should be defined to aid the compilation process and instruct the compiler on the parameters of the DSL handling. These hooks should follow the syntax of the host language and are expressed through its abstractions. They should not be defined as another DSL, such as Java annotations [110].

Type Mapping All types of the DSL that will be able to interact with the host language should also be able to be mapped by a type mapping mechanism. The compiler should have information regarding the compatibility of the types at compile-time and raise compiler warnings and errors for every incompatibility. The principles that Scala can use Java libraries can form an interesting example. Both languages are based on the JVM, which solves many problems, but many conversions need to be defined to bridge the two type systems. Typical type mapping examples exist in the JDBC and Open Database Connectivity (ODBC) specifications, which map normal Java and C types with SQL ones.

Type References Each DSL may be modified in order to define type references. A type reference should contain type information regarding the data values that are going to be passed inside the DSL runtime domain. These values should be converted into compatible DSL types and vice-versa to ensure type safety throughout the compilation process.

The above characteristics fit better to statically compiled languages. In dynamic languages, there will be several problems with the dynamic type inference and the definition of special types. This thesis focuses on J%, an extension of the Java language, which is based on the above concepts and serves as a case study of GPL and DSL language mixing in the static domain.

3.2 The J% Language

This section provides an overview of the J% language. J% adopts common Java syntax with a minimal set of extensions. Each DSL is also extended to support the integration with Java. J% follows the *Implementation: Extensible compiler / interpreter* pattern [187].

3.2.1 Basic Terminology

First, a set of basic terms must be defined.

External Module *External modules* can be developed as additional compiler components, which add support for additional DSLs. Each module can generate Java compatible code or compiled JVM bytecode. Each module definition requires the declaration of one or more external types and one configuration type. The type mapping between Java and DSL types must also be declared as a series of axiomatic and code generation rules.

External Type The keyword `external` marks them. They are special user-defined reference types and contain the DSL code. They are subtypes of references, as enumerations and interfaces.

¹<http://scala.playframework.org/documentation/scala-0.9/anorm>

²<http://felix.sourceforge.net/>

Table 3.1: J% terms compared to key concepts

J% Term	Generic Term
External Type	DSL Language Code Blocks
External Base Type	Special Types
Configuration Type	Configuration Hooks
External Reference	Type Mapping, Type References
External Module	Special Types

Each *external module* defines a set of external types and one configuration type, which are used for the compilation phase to guide the code generation. All *external types* inherit one common *external base type*.

Configuration Type The *configuration types* provide code generation and execution information for each external type, in other words for each DSL code block. These configurations are expressing compile-time and runtime options, and are defined by each external module implementor. All configuration types must extend the base *external configuration type*.

External Reference *External references* provide the bridge between the Java code and the DSL code. They act as place-holders and are replaced in the code generation phase by compatible DSL code. They are the only practical extension that is required for each DSL that is supported by the J% compiler.

Table 3.1 relates the basic J% concepts with the key concepts presented in Section 3.1.2.

3.2.2 Overview

This section describes the J% syntax. As an extension of the Java programming language, it is described by using the same conventions used by the Java language specification [110]. Appendix E contains the full J% grammar. It is the grammar distributed by the Java Language Specification [110], extended by the new grammar rules.

The following BNF conventions are used:

- $[x]$ denotes zero or one occurrences of x .
- $\{x\}$ denotes zero or more occurrences of x .
- $x \mid y$ means one of either x or y .
- All characters literals, are enclosed in double quotations e.g. "<" TypeArguments ">"

Figure 3.2 lists an excerpt of the J% grammar. The entry point rule is labelled *CompilationUnit*, which contains the declarations of *package*, *import statements* (ImportDeclaration), and *declared types* (TypeDeclaration). Each type can be either a *class*, *enumeration*, *interface* (ClassOrInterfaceDeclaration) or an *external type* (ExternalDeclaration). The rule *ExternalBody* contains the DSL code, which is enclosed by a pair of curly braces ($\{\}$).

```

CompilationUnit:
    [ [ Annotations ] "package" QualifiedIdentifier ";" ] { ImportDeclaration }
    { TypeDeclaration }

ImportDeclaration:
    "import" [ "static" ] Identifier { "." Identifier } [ ".*" ] ";"

TypeDeclaration:
    ClassOrInterfaceDeclaration
    ExternalDeclaration
    ";"

ExternalDeclaration:
    { Modifier } "external" Identifier "extends" Type ExternalBody

ExternalBody:
    "{" { AnyCharacter } "}"

```

Figure 3.2: Simplified J% grammar

3.2.3 External Types

An *external type*, is a custom user-defined reference type, which embodies the DSL code. Practically, an external type instructs the compiler that DSL language code exists, and configures it the appropriate external module for execution.

Only one *external type* can be declared per source file. Each *external type* is parameterised by a *configuration type*. *External types* are `final` and cannot be inherited. They accept one modifier, `public` or none for package access. If an invalid modifier is set to an external type, the compiler will raise a compilation error. All *external types* are subtypes of the *external base type*³. Figure 3.3 illustrates a typical *external type* hierarchy.

External types can be instantiated by name with the usage of the operator `new` like all Java reference types (Section 3.2.9). The external type has methods and attributes in addition to those, which are inherited by the external base type. They may not be decorated by annotations, and they can be documented by Javadoc style comments [110].

The following code excerpt declares an external type of a number-matching regular expression,

```

public external NumberRegex extends Regex<Conf> {
    [0-9]+
}

```

and then instantiates it,

```

NumberRegex nr = new NumberRegex();

```

3.2.4 External Declaration

An *external declaration* specifies a new named *external type*.

³`org.jmod.dsl.module.ExternalBaseType`

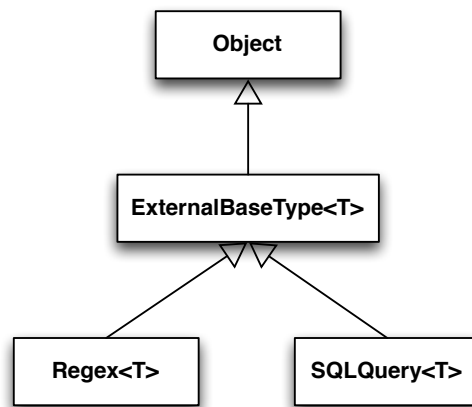


Figure 3.3: A typical external type hierarchy

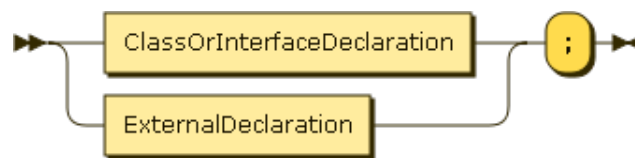


Figure 3.4: Syntax diagram for TypeDeclaration rule

```

TypeDeclaration:
  ClassOrInterfaceDeclaration
  ExternalDeclaration
  ";"

```

```

ExternalDeclaration:
  { Modifier } "external" Identifier "extends" Type ExternalBody

```

Figure 3.4 and 3.5 depict the above rules as syntax diagrams. Each type declaration rule contains a list of *modifiers*. The keyword “external” and the *Identifier* that represents the name of the external type.

3.2.5 Package Declaration and Importing of External Types

External types belong to a *package*, like any other typical Java type. If no package is declared, then the default package is used. External types also use `import` statements to include other types into their scope. Static imports are supported.

```

CompilationUnit:
  [ [ Annotations ] "package" QualifiedIdentifier ";" ] { ImportDeclaration }

```



Figure 3.5: Syntax diagram for ExternalDeclaration rule

```
{ TypeDeclaration }
```

ImportDeclaration:

```
"import" [ "static" ] Identifier { "." Identifier } [ ".*" ] ";"
```

The following example exhibits an import statement of the `Regex` class of the regular expressions external module.

```
import org.jmod.dsl.regex.Regex;
```

3.2.6 External Type Modifiers

ExternalDeclaration:

```
{ Modifier } "external" Identifier "extends" Type ExternalBody
```

Modifier: one of

```
Annotation "public" "protected" "private" "static" "abstract"
"final" "native" "synchronized" "transient" "volatile" "strictfp"
```

An external type declaration may include several of the standard modifiers. They cannot be decorated by *annotations*, cannot be declared *abstract*, are by default *final* and cannot be *private*, *protected*, *native*, *transient*, or *volatile*. An *external* type can be declared *static*. Misuse in the declaration of the external's modifiers results in a compile-time error. An example of external type declaration follows:

```
public external DSLExtType extends DSLBaseType<DSLConf> { ... }
```

3.2.7 External Body

The following rule is simple and acts as the container for each DSL code block. All characters inside the curly braces (`{}`) are considered DSL code.

ExternalBody:

```
"{" { AnyCharacter } "}"
```

3.2.8 Configuration Types

Configuration types are reference types (Java classes) that contain information regarding the evaluation of the DSL code in compile-time and the runtime. They are used only as type parameters, for the external base type. Consider the following example:

```
public external IPAddress extends Regex<RegexConfiguration> {
([0-9]{1,3}\.){3}[0-9]{1,3}
}
```

The configuration type `RegexConfiguration` provides with compile-time configuration the *external base type* `Regex`, thus setting the code generation context for the external type named `IPAddress`. All configuration types are subtypes of the basic configuration type⁴. The following code exhibits a simple configuration type for the SQL module.

⁴`org.jmod.dsl.module.ModuleConfiguration`

```

ExternalRef:
    "#" "[" ParameterName "]" "<" FieldType ">"

ParameterName:
    Identifier

FieldType:
    Identifier { "." Identifier } { "[" "]" }

```

Figure 3.6: External References Syntax

```

public class SimpleSQLConfiguration extends SQLConfiguration {
    public boolean SQLMOD_NS_AWARE = true;
    public String SQLMOD_NS_URI = "file:///examples/simplesql/schema.sql";
}

```

The above instructs the SQL module to check the SQL code against a specified database schema, which can be found in a text file, defined by the `SQLMOD_NS_URI` parameter. Since these types are classes, no special rules apply. The actual configuration information is passed with the usage of specified class fields. The standard Java semantics permit override in the initialisation of a field's value from their subclasses.

Part of the module configuration is also available in the runtime. The configuration parameters that are excluded, are defined by the external module.

3.2.9 External References

In J%, each DSL maintains its own syntax. For simple cases, where the DSL has no type system, such as regular expressions, the language remains unaltered. When the J% language needs to interact with the DSL an *external reference* must be defined. It acts as a contract that describes the convention between the two type systems and a placeholder that will be used by the external module properly to convert the passing value from the GPL to the DSL domain.

Figure 3.6 contains the BNF syntax for the *external references*. The main rule is named `ExternalRef` and it consists of two parts; the *parameter name* and the *field type*.

Parameter name (`ParameterName`) defines the name of the parameter that is going to be used in code generation phase. The *field type* (`FieldType`) defines the corresponding Java type, according to the naming convention that is commonly used by the Java programming language.

The number of *external references* is bound to the maximum formal parameters that a method can possibly have. This is defined in the Java Language Specification [110].

Consider the case of an SQL query that accepts an `int` as a parameter as input (`c_id`):

```

public external SimpleSQL extends SQLQuery<SQLConfiguration> {
    select * from customer where customerId = #[c_id]<int>
}

```

The expression `#[c_id]<int>` defines that the `customerId` expects an `int` Java type. The `[c_id]` is the parameter name and practically generates the following constructor for the `SimpleSQL` external type:

```

public SimpleSQL(int c_id) { ... }

```

Table 3.2 lists a series of external references along with the generated constructors.

Table 3.2: External References Examples

Parameters	Constructor
#[char_v]<char>	TypeName(char char_v)
#[p1]<int>, #[p2]<java.lang.Integer>	TypeName(int p1, Integer p2)
-No parameters-	TypeName()
#[sh]<short[]>, #[i]<int>	TypeName(short[] s, int i)

3.2.9.1 Example

Consider that we have the following code, a regular expression that matches an Internet Protocol (IP) Address, for example, `([0-9]{1,3}\.){3}[0-9]{1,3}`. A typical Java program that uses the standard `java.util.regex` application library, would be:

```
import java.util.regex.*;

public class IpAddress {
    private Pattern ipPattern;

    public IpAddress() {
        this.ipPattern = Pattern.compile("([0-9]{1,3}\\.){3}[0-9]{1,3}");
    }
}
```

The above listing initialises a `Pattern` class with the IP address regular expression, which embeds the DSL as a simple `String`. With this approach, the validity of the regular expression will be checked in the runtime. Upon error an exception will be thrown and the programmer can handle it accordingly.

The following listing illustrates the same functionality, this time written in the J% language.

```
import org.jmod.dsl.regex.Regex;
import org.jmod.dsl.regex.RegexConfiguration;

public external IpAddress extends Regex<RegexConfiguration> {
    ([0-9]{1,3}\\.){3}[0-9]{1,3}
}
```

The class `IpAddress` extends the `Regex` type, which is defined by the regular expression module support. In addition, the `Regex` *external type* should be parameterised by a type, referred as `RegexConfiguration` in this example. This is a formal way to parameterise the module at compile-time to activate specific code generation features.

3.3 The J% Type System

This section discusses the semantics and the modifications that were made to the existing Java type system [110] to support J%'s features. It is organised as follows; Section 3.3.3 describes the J% extensions. Section 3.3.4 discusses the type mapping mechanism and how type safety is ensured at compile-time.

The study of type systems is the study of programming languages from a type theoretic perspective. First, the the definition of a type system [204]:

“A type system is a tractable syntactic method for proving the absence of certain program behaviours by classifying phrases according to the kinds of values they compute.”

Initially, type systems were formalized in an attempt to address problems in the area of logical paradoxes in mathematics. Later on, they became standard tools in logic, especially in proof theory. Major discoveries in this area include the *ramified theory of types* [279], the *simple theory of types* [211], the *lambda-calculus* [56], the *constructive type theory* [179] and the *pure type systems* [21].

Research in this field includes two major aspects; the *practical*, which focuses on applications on programming languages and the *abstract*, which extends the theoretical basis of type systems.

3.3.1 Why do we need a Type System

Type systems are usually realized as type checkers in compilers and linkers of programming languages. They are categorized as static and dynamic. *Static* type systems imply static type checking, which is the ability of a programming language’s compiler to know all the data types that are used in a program, and guarantee that they have and will have correct values at compile time. *Dynamic* typing is exactly the opposite. The languages that are dynamically checked perform run-time checks to ensure consistent type use.

A programming language is called safe, when it protects its own abstractions. For example, a programming language that provides an abstraction for arrays is considered safe, when it performs boundary checking upon access, thus Java that does such checks can be categorised as statically checked and safe, while C and C++ are statically checked, but unsafe. Perl and Lisp are dynamically checked and safe [204].

The design of a programming language should always include the type system design [47, 120, 129, 166, 197, 204, 287]. Early design and theoretical validation of a type system ensures,

Language Safety If designed correctly, language safety can be achieved through type checking.

Either static or dynamic, with types we can ensure that the correct data are stored in the correct variable. Practically, language safety implies the ability of a programming language to protect its own abstractions in runtime and compile-time. For example, in Java, with each array access, the virtual machine checks if the access is legal, within the array bounds. If not, an `ArrayIndexOutOfBoundsException` exception is thrown and the programmer can handle it at runtime, preventing the program to perform unexpectedly. Thus, Java is considered safe. In C an illegal array access usually results in a program crash or in the worst case in data corruption by letting the program run outside its normal parameters.

Error Detection One of the most typical advantages of type checking is the discovery of programming errors at compile-time. The errors can then be fixed immediately, rather than discovered later upon execution and produce effects that cannot be explained easily. Usually, error detection includes heuristic checks, like implicit casting in types and auto-boxing, which can easily result in bad performance, etc.

Abstraction The existence of a type system enforces disciplined programming. In practice, type systems form the backbone of languages, and allow a more abstract style of design, completely independent from the implementation.

Table 3.3: Types System Properties on Popular Programming Languages [204]

	Static	Dynamic
Safe	Java, C#, Scala	Perl, Lisp
Unsafe	C, C++	Bash, awk

Documentation Types are useful when studying a computer program. Strict types give hints about the program behaviour and constitute a form of documentation for a programmer that cannot become outdated, since type checking occurs every time the compiler is executed.

Efficiency The first type systems were introduced to improve the efficiency of programming languages. For example, FORTRAN improved its efficiency by differentiating between integer and floating point arithmetic. This allowed the compiler to perform machine dependent optimizations, when handling specific types. Type systems can improve efficiency, if used correctly and each case's individuality is exploited [84].

In addition to those static and dynamic type systems, *Soft typing* combines characteristics from both static and dynamic aspects [48]. The key idea behind soft typing is that the static type checker can not reject programs that contain potential errors, but inserts explicit runtime checks converting dynamically typed programs into statically correct form. With soft typing the compiler is more lenient. Consider the following statement of the Java programming language.

```
Integer i = 3;
```

The above is translated to,

```
Integer i = new Integer(3);
```

which automatically converts the primitive type `int` to `java.lang.Integer`. This feature is called *Auto-boxing* and serves as an example of the above. This method was introduced in Java 1.5, shares some traits with soft typing to seamlessly convert primitives to their class representations e.g. `double` to `java.lang.Double`, without the programmer's intervention.

3.3.2 The Java Type System

The Java type system (Figure 3.7), defined in references [11, 72, 73, 110, 198, 248], introduces two categories of types: *primitives*, and *references*. The *primitive* types are:

int 4 bytes, signed (two's complement). Ranges from -2,147,483,648 to 2,147,483,647

boolean 1-bit. Can take only two values, `true` or `false`

char 2 bytes, unsigned, Unicode. Ranges from 0 to 65,535

short 2 bytes, signed (two's complement). Ranges from -32,768 to 32,767

long 8 bytes signed (two's complement). Ranges from -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807

byte 1 signed byte (two's complement). Ranges from -128 to 127

float 4 bytes, IEEE 754. Ranges from $(+/-)1.40129846432481707e-45$ to $(+/-)3.40282346638528860e+38$

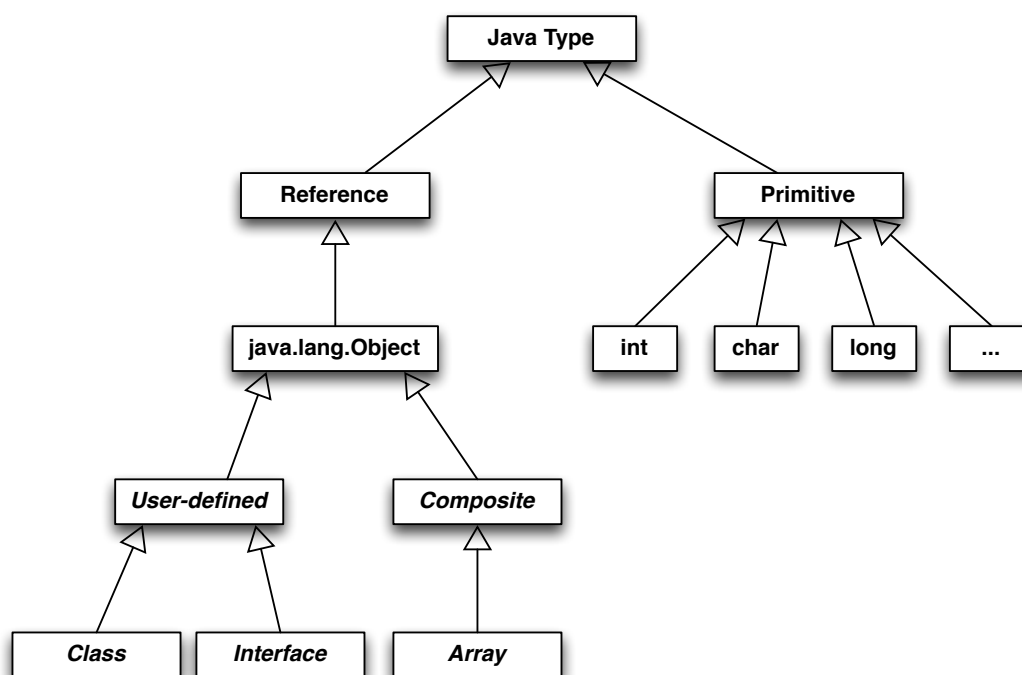


Figure 3.7: Overview of the Java Type System

```

<object_type>      ::= 'L'<fullclassname>;
<array_type>       ::= '['<optional_size><field_type>
<field_type>       ::= <base_type>|<object_type>|<array_type>
<base_type>        ::= 'B'|'C'|'D'|'F'|'I'|'J'|'S'|'Z'
<optional_size>    ::= '0'-'9' { '0'-'9' }
  
```

Figure 3.8: Internal Type Signature in Java

double 8 bytes IEEE 754. Ranges from $(+/-)4.94065645841246544e-324d$ to $(+/-)1.79769313486231570e+308d$

The *reference* types are subclasses of the `java.lang.Object` class. They are divided to *user-defined* and *composite*. *User-defined* types are typically *classes*, *interfaces* and *enumerations*. *Composite* types are in Java *arrays* of *primitives* or *reference* types. Table 3.4 provides details regarding the interpretation of the base types [169] by the JVM.

Figure 3.8 contains the BNF notation of *Type Signature* for Java. A *Type Signature* represents types and methods in the internal JVM notation. Table 3.4 contains the JVM type naming scheme and Table 3.5 lists typical examples of the type signature notation.

3.3.3 J% Type System Hierarchy

J% is a programming language that inherently supports *strong typing*, since it extends Java. This means that every type declaration, every variables' type is inferred, evaluated and checked at compile time. *Strong typing* is a key feature for a series of modern programming languages, since it provides the basis for early error detection and provides helpful recommendations during the compilation phase of a computer program. Figure 3.9 illustrates the type hierarchy for the J%

Table 3.4: Interpretation of base type characters

Signature	Type	Description
B	byte	Signed byte
C	char	Unicode character
D	double	double-precision floating point value
F	float	single-precision floating point value
I	int	integer
J	long	long integer
L<classname>;	reference	An instance of class <classname>
S	short	singed short
Z	boolean	Boolean type (true or false)
[	reference	one-dimensional array
V	reference	void (nothing)

Table 3.5: Examples of Java Type Signatures

Description	Signature
int (primitive)	I
java.lang.Integer	Ljava/lang/Integer;
Array of int, length 5	[5I
Array of String, length 10	[10Ljava/lang/String;
org.io.Buffer.flush()	Lorg/io/Buffer/flush(V)
org.io.Buffer.init(Buffer b)	Lorg/io/Buffer/init(Lorg/io/Buffer;)V
org.io.Buffer.append(char[] c)	Lorg/io/Buffer/append([C)Lorg/io/Buffer;
String.substring(int, int)	Ljava/lang/String/substring(II)Ljava/lang/String;

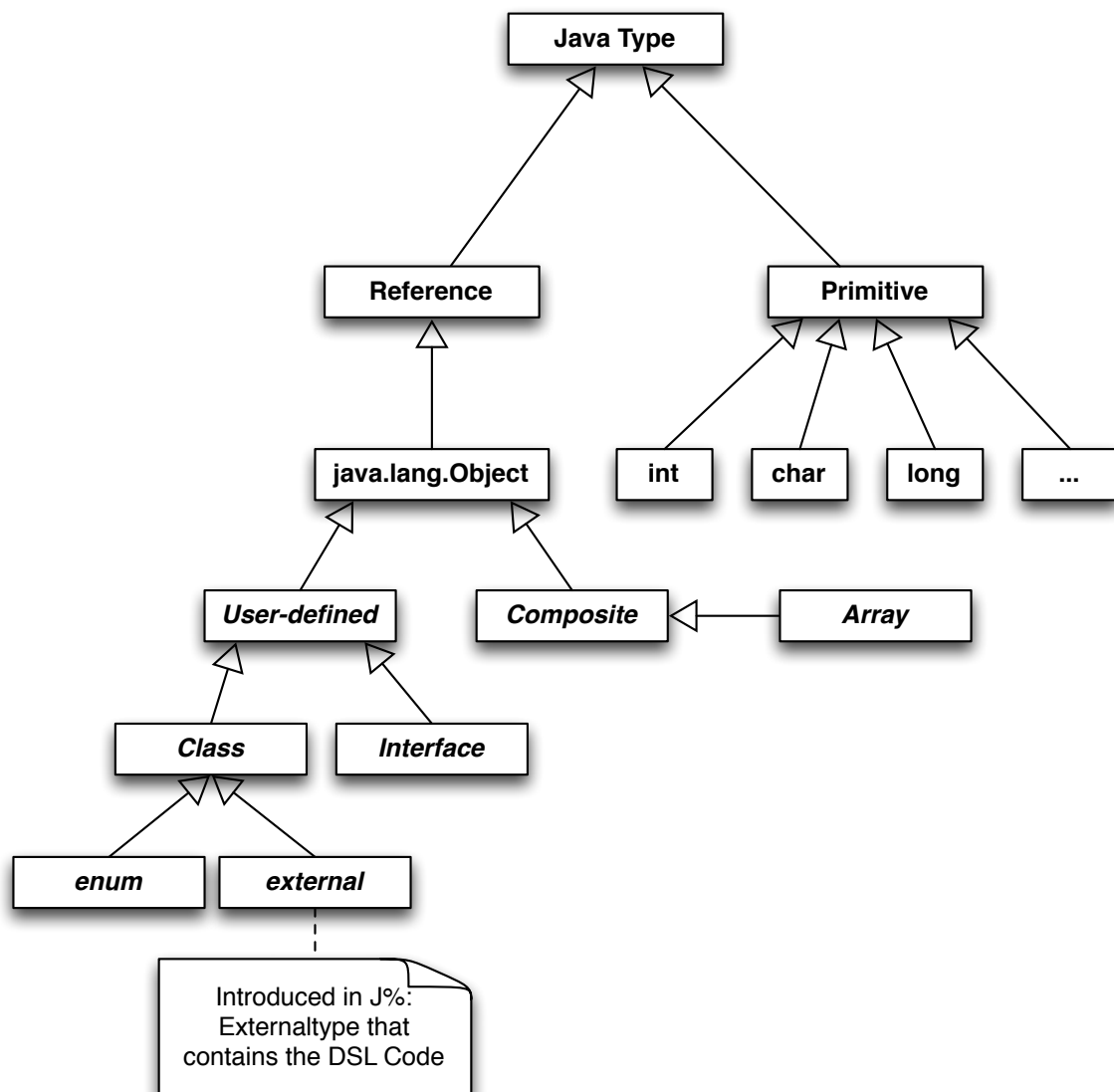


Figure 3.9: Overview of the J% type system extensions

programming language as a UML class diagram. J% type hierarchy follows the same principles as the Java programming language and has two basic categories for *Java types*; *primitives* and *references*.

The *primitive* types consist of eight (8) unique types, *int*, *short*, *double*, *char*, *float*, *byte*, *boolean* and *long*.

There are three (3) standard user-defined reference types, *class*, *enums* and *interfaces*. J% introduces another one, namely *Externals*. This is the only addition of J% in the Java type system. It is a special subtype of *class*, like enumerations, which is allowed to have special syntax at compile-time. Other than that, it interacts and behaves as a typical *class* reference type, as described in the Java Language Specification [110].

Table 3.6: `jsonsimple` list of mapped types

JSON	Java
string	<code>java.lang.String</code>
number	<code>java.lang.Number</code>
true and false	<code>java.lang.Boolean</code>
null	null
array	<code>java.util.List</code>
object	<code>java.util.Map</code>

3.3.4 Type Mapping

The type mapping mechanism defines the compatibility of types between the J% and each DSL language. A DSL module provides the type-mapping rules with Java type system. If an imported type is not valid according to the rules of the type mapping facility, then a compiler error is generated.

Type mappings between two languages have already been used in various of DSL embedding approaches. For example, JDBC [91] defines a mapping specification between SQL and the Java standard types. The JavaScript Object Notation (JSON) application library `jsonsimple` for Java uses the type mapping presented in Table 3.6.

Jint⁵ follows a similar type mapping approach between Javascript and C#.

```
script= @"
function square(x) {
    return x * x;
};

return square(number);
";

var result = new JintEngine()
    .SetParameter("number", 3)
    .Run(script));

Assert.AreEqual(9, result);
```

3.3.5 The J% Approach

To address the problem of type mapping in J%, two kind of relations are defined between the types of the languages:

Compatible Types A set of Java types that can be mapped to a set of DSL types and vice versa defines a map of *compatible types*.

Dominant Types Even if a type is compatible with other types, one type must be pointed out as *dominant*, and be preferred when the compiler needs to choose and resolve ambiguous situations. These types are defined as *dominant types*.

Each external module defines a series of *compatible* types that are used when the host language and the DSL need to exchange data. This mechanism must also provide default values for

⁵<http://jint.codeplex.com/>

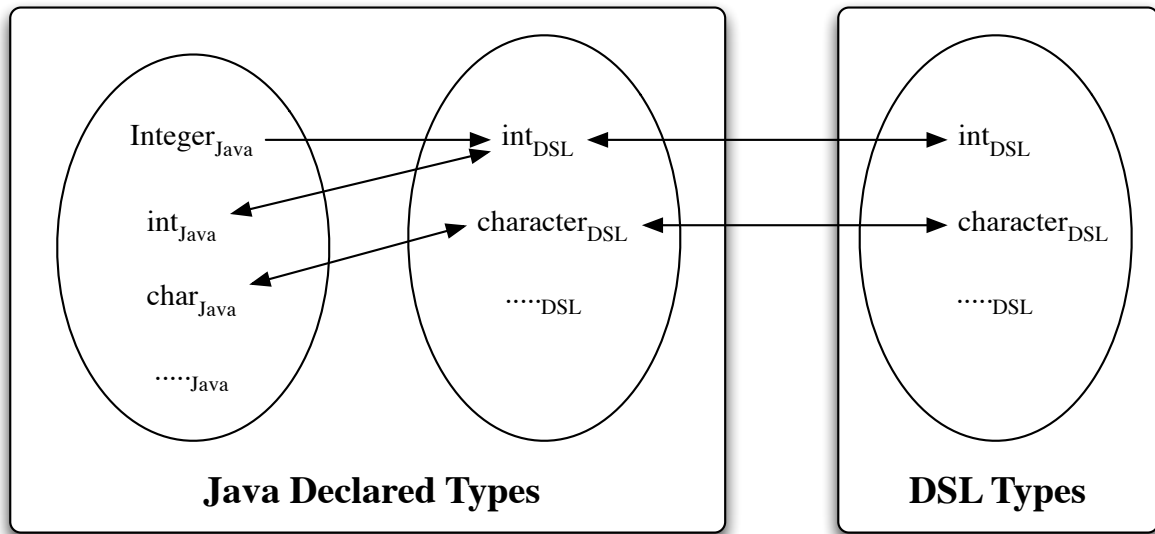


Figure 3.10: How type mapping works

each mapped type. To understand the type mapping mechanism, consider a hypothetical DSL, which has two types that need to be mapped to Java types, an *integer* (int_{DSL}) and a *long* ($long_{DSL}$) type. It is required to provide the following conventions:

- The *integer* DSL type should be mapped to the *int* Java primitive type,
- the *character* DSL type should also be mapped to the *char* Java primitive type,
- the *Integer* Java reference type (`java.lang.Integer`) should be compatible with the *integer* DSL type.

Figure 3.10 depicts the relations between the two type systems. The boxes contain the types declared inside the scope of J% (Java) types and the DSL. Practically, we declare types in Java type context, which have an one-to-one relation with the correspondent DSL types, then we associate them with the other Java types, which provide the mapping relations. The arrowheads express the type conventions that are used in this example. For the current mapping scheme we convert the *int* and *character* DSL types to the *int* and *char* Java primitive types. In addition, we declare that `java.lang.Integer` as compatible type with the aforementioned *int* DSL type.

3.4 Compilation Process

Figure 3.11 depicts the pipeline for the compilation process. It includes the following steps:

1. **Parse** The compiler accepts J% source files as its input. The files are parsed and syntax errors are reported,
2. **Analyse** Semantic analysis is performed and the *External* types are identified, along with their configurations. Symbol tables are populated,
3. **Execute DSL Modules** According to their configuration, each *DSL module* is executed for each *external* type. Type checking between the host language and each DSL is performed and

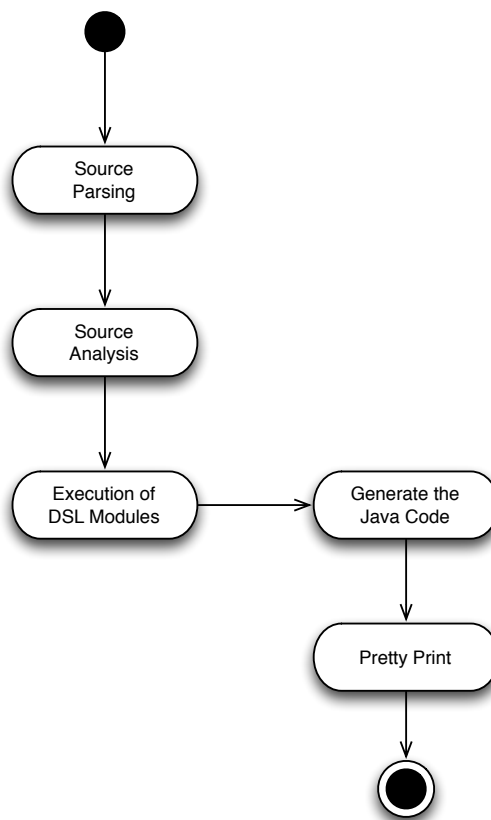


Figure 3.11: Pipeline of the compilation process

possible type incompatibilities are reported. In addition, each DSL code block is syntax checked and the errors are reported as compile-time errors,

4. **Code Generation** Java-compatible code is generated in this phase. The *DSL module* generates code for each *external* type. Java-compatible source files are left intact and simply copied,
5. **Pretty-print** The generated code is reformatted and syntactically checked with the usage of a *pretty-printer* filter [202].

After execution of the above steps, the standard *Java* compiler is invoked to compile the generated sources.

Chapter 4

Implementation of the J% Compiler System

Talk is cheap. Show me the code.
Linus Torvalds

J% follows a typical *language processing system* architecture, as described by Sommerville in reference [234]. The prototype implementation follows the *pipe and filter compiler architecture* pattern, which includes six basic components, which are illustrated in Figure 4.1. The *pipe and filter architecture* consists of the following components:

1. A *lexical analyser*, which takes as input the source language, and converts them to tokens and finally to ASTs.
2. A *symbol table*, which holds information regarding the types and the program declarations.
3. A *syntax analyser*, which checks the source files for syntax errors and reports them. In this case, this component also identifies files containing *external configuration* classes and the *external types*.
4. A *syntax tree*, which is practically a collection of ASTs for each source file.
5. A *semantic analyser*, which combines the information from the *syntax trees* and the symbol tables to find and report violations in semantics from the source files.
6. A *code generator*, which generates the output code, in our case Java compatible code.

4.1 The J% Compiler

The compilation process has two phases; in the first, all source files are analysed and the compiler gathers information regarding the user-defined types. In the second phase, the compiler uses all that information and compiles the code. Finally, it generates the Java output. All compile-time errors are reported during these two phases; these include Java syntax errors, DSL syntax errors and type mapping errors.

The process is illustrated in Figure 4.2 as a UML activity diagram. The J% compiler accepts the input source files (*.jmod and *.java), scans the input directories for input files, then marks each file as *External* or *Java*. *External* denotes that an external type is declared and *Java* that the code is pure Java code. Note, that external configuration types are also marked as Java, since

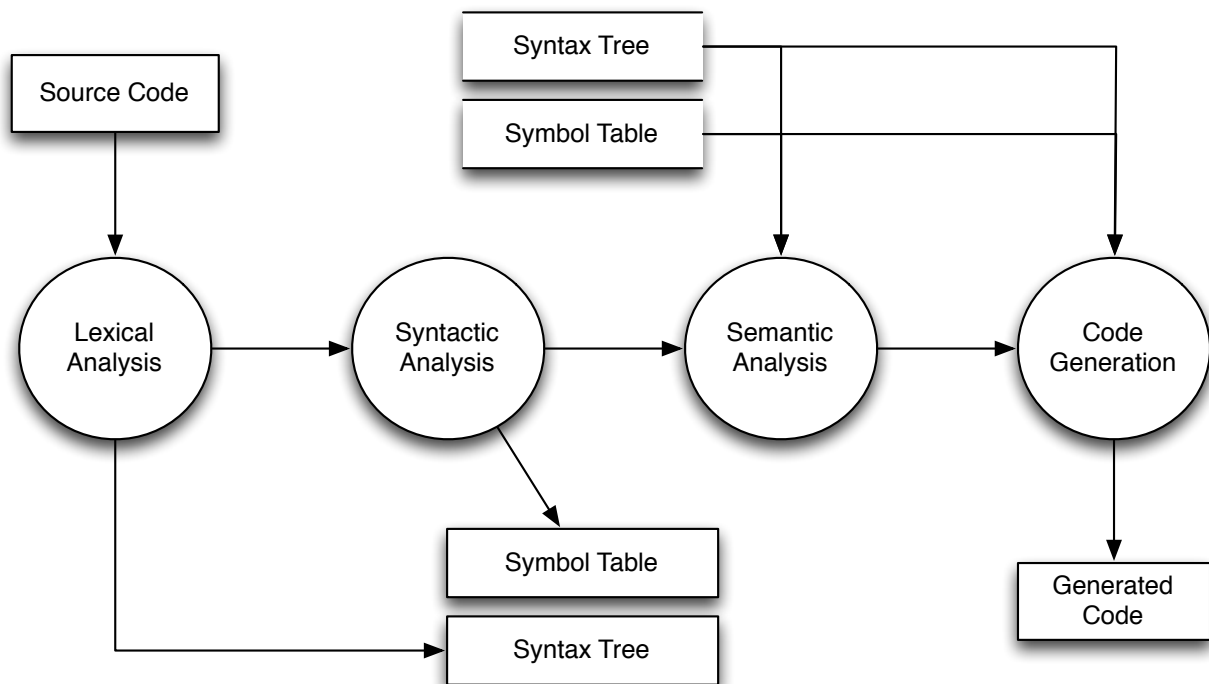


Figure 4.1: The J% Compiler as pipe and filter architecture [234]

they contain pure Java code. All this information populates directly the symbol table. The files are one-by-one compiled. The code generator is invoked and Java code is generated. Finally, the Java compiler (`javac`) is executed, which translates the generated code into executable JVM bytecode.

The compilation process per file is depicted in Figure 4.3. Each file can be compiled either by the *J% Compiler* or by an *External Module*. Since the compiler generates Java code, if a file is marked as Java in the first compilation phase, then it is copied to the output directory and no code generation takes place. If the file contains an external type, then the appropriate external module is initialised with the specified context. Upon that, all information regarding symbols, the configuration and all the specifics of the external code are available to the module for compilation. The module is responsible for the code generation. This process is repeated for all available source files.

All modern programming languages are distributed with a SDK that provides the basic tool set for programmers in their environment. It comprises of two parts; a set of programming libraries that perform common tasks and a set of compilation and debug tools. J% is a descendant of Java programming language. It adopts the Java development platform and extends it.

Figure 4.4 depicts the composition of the J% platform; the *J% core libraries*, the *Java platform libraries* and the underlying *operating system* dependency. The *J% core libraries* include the *external modules* for regular expressions and SQL (Section 4.4 and 4.5). In addition, the compiler prototype provides a metaprogramming package, which includes basic metaprogramming facilities for the external module developers. This is further analysed in Section 4.1.5. The J% components depend on mainstream application library implementations for the supported DSLs. For example, the SQL external module depends on the JDBC API [91].

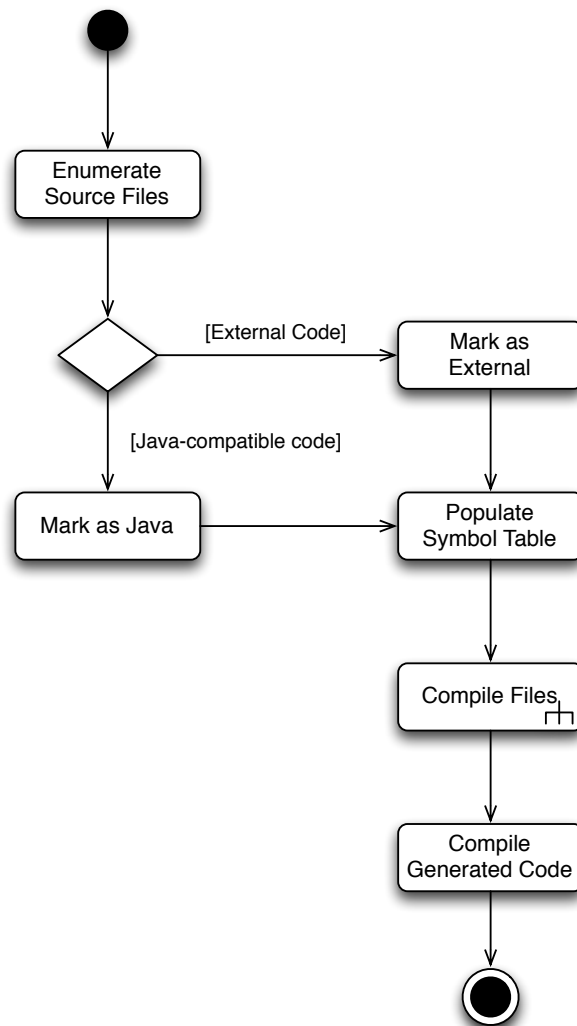


Figure 4.2: Overview of compilation process

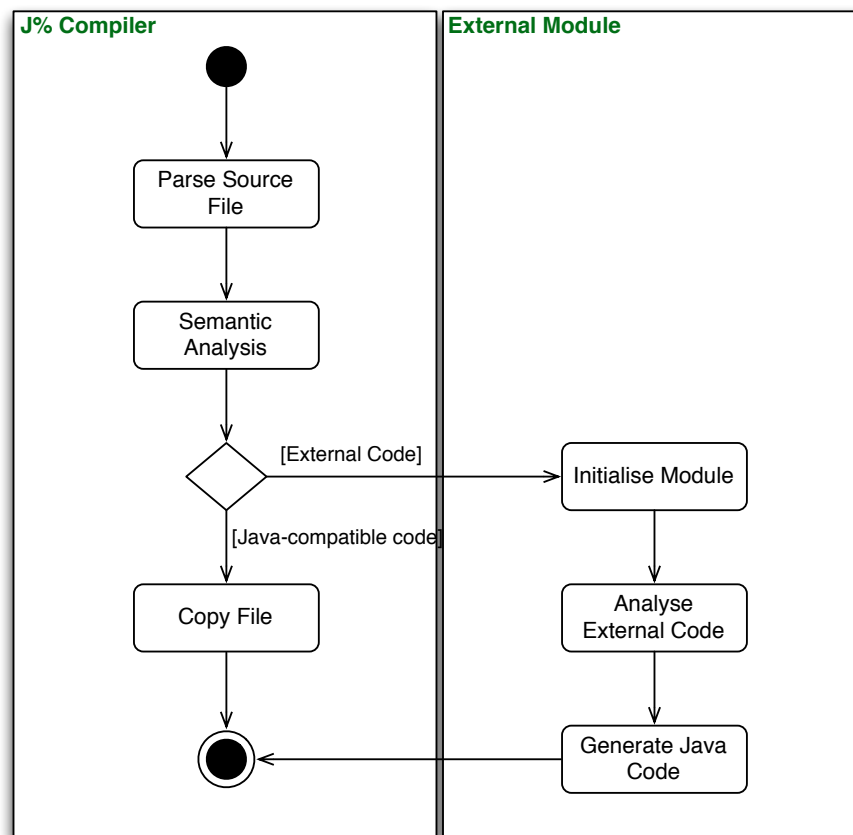


Figure 4.3: The compilation process per file

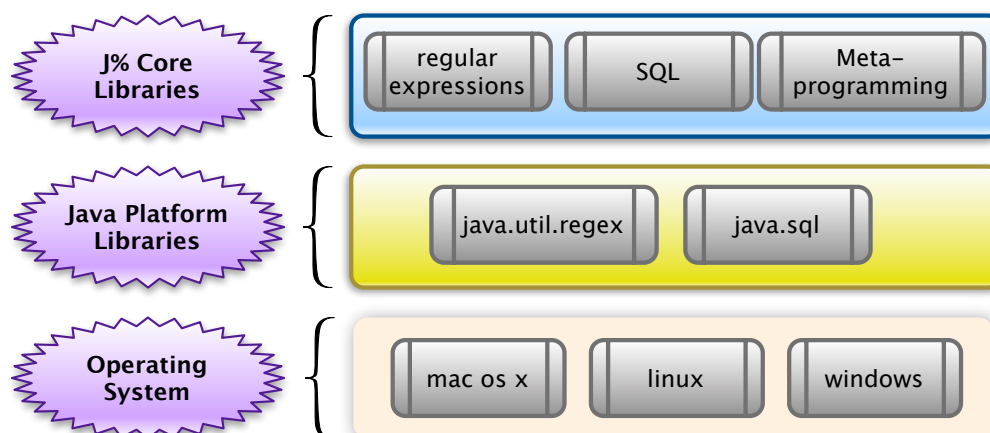


Figure 4.4: The J% Platform

Table 4.1: J% compiler size metrics

Metric	Value
File Count	97
Line Count	5,799
Lines of Code	5,799
Package Count	21
Number of Classes	86
Number of Interfaces	5
Number of Enumerations	10

Table 4.2: J% compiler standard options

Option	Description
-ml, --module-list	Prints the available External modules
-cl, --compiler-list	Prints the available compiler list (back-ends)
-pc, --print-classpath	Prints current classpath
-cc, --compiler-context	Prints current compiler context
-pextc, --print-external-context	Print each time the External context
-i, --input-dir	Input source directories (recursively add file(s) *.jmod)
-mc, --metrics	Calculate a basic set of size metrics
-h, --help	Displays help information
-xml, --output-xml	Compiler output is set in XML format
-st, --symbol-table	Export Symbol Table
-uexth, --user-external	Print user-defined external type list

4.1.1 Compiler Implementation

The J% compiler is implemented in Java. A JavaCC-based¹ generated parser was used. Each external module uses its own parsing engine for syntax checking. Type checking is performed against the compiler's symbol table.

Figure 4.5 depicts the basic elements of the J% compiler. The `JMod` class coordinates the compilation process. It is associated with the `CompilerContext` and the `ModuleList` class. The `CompilerContext` is a singleton class, which contains all the runtime information of the compiler, such as arguments, available external modules, and source files. The `ModuleList` contains information regarding the external modules and provides the means to instantiate them. Each module declares to the compiler two sets of types; the first contains the *External base types*, which act as the common types for the user-defined external types, and the second has the configuration types. Each external module is a specialisation of the abstract class `Module`, which is part of the module² package. All external modules must be registered in the `ModuleList`. The J% distribution contains out-of-the-box modules for regular expressions and SQL. The packages `org.jmod.dsl.regex` and `org.jmod.dsl.sql` provide the implementation for these. The `org.jmod.symbol` package contains the classes that implement the symbol table facility and perform the type checking. This package depends on the `org.jmod.compiler.source` package that contains the J% parser. Table 4.1 provides a series of size metrics for the J% compiler. Table 4.2 lists the compiler's command line options.

¹<https://javacc.dev.java.net/>

²`org.jmod.dsl.module`

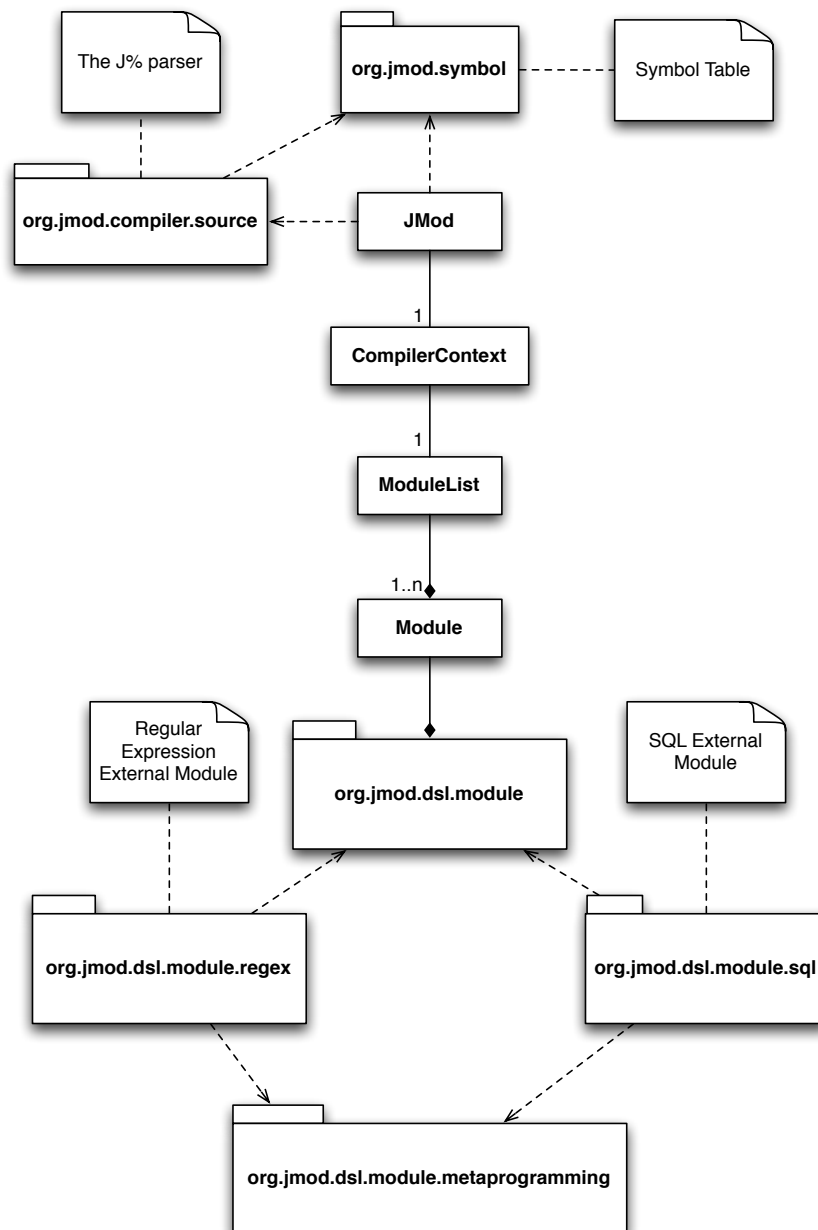


Figure 4.5: J% compiler architecture

4.1.2 External Modules

External modules provide the compile-time support for DSLs. They have a dual use; implement the syntax and type checking in *compile-time* and also provide, optionally, the execution environment at *runtime*.

Figure 4.7 depicts the external module architecture. The classes are divided in two groups; those that affect the *compile-time* part of the module, which are the `Module`, `ModuleException`, `ModuleList`, and `DefaultModule`, and those that affect the *runtime* part, which consist of the `ExternalBaseType`, `ExternalConfiguration`, and the `ModuleRuntime` interface.

The `Module` class is the entry point for each compiler module. It should be registered in the `ModuleList`, which is the registry for all the modules. The `DefaultModule` class is a dummy external module, and it is used internally by the compiler for error recovery. The following listing contains the declaration of the `Module` abstract class.

```
public abstract class Module {
    protected Module() { ... }
    public boolean isDefaultModule() { ... }
    public abstract Type getConfigurationType();
    public abstract Map<String, String> getDefaultConfiguration();
    public abstract boolean evaluate(CodeUnit cu, Map<String, String> ctx)
        throws ModuleException;
    public abstract String getName();
    public abstract String getDescription();
    public abstract String getVersion();
    public abstract String getAuthor();
    public abstract Type[] getExternalTypes();
    public static String exportContext(Map<String, String> map) { ... }
}
```

The `ExternalBaseType` is the base class for all the declared external types. The `ExternalType` class is parameterised by a `ModuleConfiguration` class, which contain the configuration for the specific external module invocation. Its declaration follows:

```
public class ExternalBaseType<T extends ExternalConfiguration> {
    protected T configuration;

    // Constructor
    protected ExternalBaseType(T configuration) { ... }
    public Map<String, String> getModuleRuntime() { ... }
}
```

`ExternalBaseType` has only one method, namely `getModuleRuntime()`, which returns the runtime configuration for the module. Currently, the standard J% Development kit supports two built-in external modules, one for regular expressions and one for SQL. These are implemented by the `RegexModule` and `SQLModule` classes accordingly (Figure 4.7), and their implementation details are analysed in Sections 4.4 and 4.5. Figure 4.6 lists the module list, provided by the `--module-list`, `-ml` command-line option.

4.1.2.1 Initialisation

When the compiler is executed, all external modules are identified, and all external types are added to the symbol table and tagged as external base type or configuration types. Each module can

```
$ java -jar jmodv2.jar -ml
---DSL modules---
Default module - Default External Module (dummy implementation)
Regex - Regular expression library module
SQLModule - SQL Support Module (based on standard JDBC calls)
```

Figure 4.6: J% standard modules export (--module-list option)

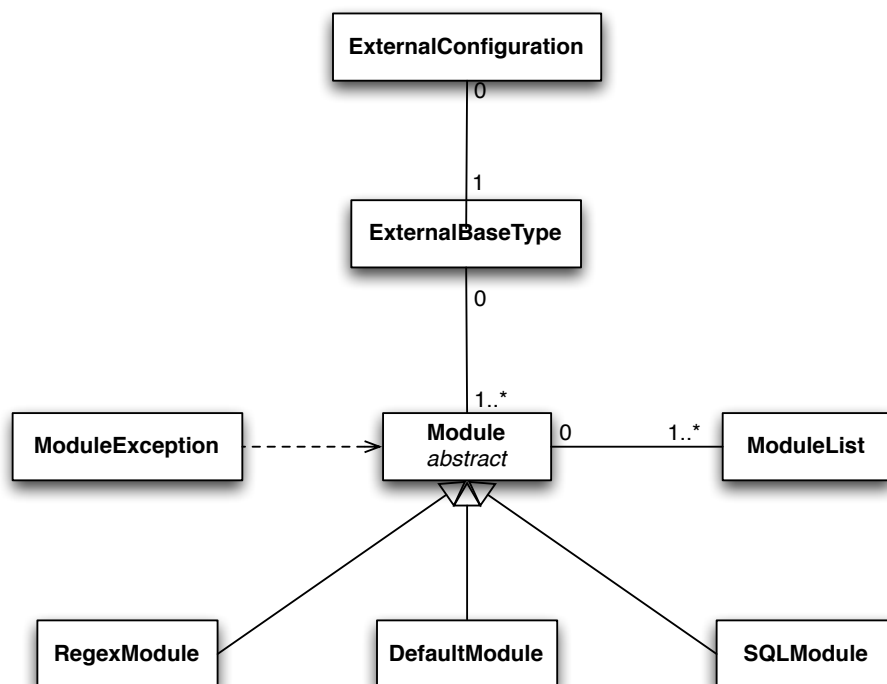


Figure 4.7: J% module package class diagram

Table 4.3: List of valid configuration field types and values

Type	Description	Examples
int	4 bytes, signed (two's complement)	1, 2, 3, ...
float	4 bytes, IEEE Std 754	0.5, 1.6, 1000.0, ...
String	Instances of the <code>java.lang.String</code>	"Example", "Hello, World!"
boolean	Boolean value	true or false

export multiple base types, but only one configuration type.

The following listing is an excerpt of the symbol table export, populated only with the types exported by the module and the user-defined external and configuration types. The compiler option for symbol table export is `--symbol-table, -st`.

```
$ java -jar jmodv2.jar -i examples/ -st
[...]
----- Symbol Table Dump -----
examples.simpleregex.SimpleRegex (External)
org.jmod.dsl.regex.Regex (External)
examples.simplesql.SimpleSQL (External)
org.jmod.dsl.sql.SQLQuery (External)
org.jmod.dsl.sql.SQLConfiguration (Java, Configuration)
org.jmod.dsl.regex.RegexConfiguration (Java, Configuration)
org.jmod.dsl.module.def.DefaultConfiguration (Java, Configuration)
```

4.1.2.2 Configuration Types

The external types are invoked within a specified compile-time context, provided by a *external configuration type*. Each configuration type must be able to be instantiated by a public class constructor that accepts no field parameters.

```
public abstract class ExternalConfiguration {
    protected abstract Map<String, String> getModuleConfiguration();
    public abstract Map<String, String> getRuntimeConfiguration();
}
```

The J% compiler identifies all the classes that are subclasses of the `ExternalConfiguration` class and uses them to identify all the user-defined configuration types. The configuration parameters are declared as a set of public class fields. These parameters can have only `int`, `String`, `float`, and `boolean` types as defined in the Java type system [110]. Table 4.3 summarises the types of the configuration fields.

Figure 4.8 depicts a configuration type hierarchy for an external module. The *ExternalConfiguration* is the parent type and contains two methods; namely the `getModuleConfiguration` and the `getRuntimeConfiguration`. Since the runtime configuration may be different (optional) from the *compile-time* configuration, these two functions provide the configuration parameters accordingly. At this point, this information is exposed as key-value pairs of strings (`Map<String, String>`).

For the case of SQL, the module may be configured to provide the database schema at compile-time. The runtime part of the configuration should not export this information. The access level for each of the configuration parameters is defined by the module creator. By default, all parameters are considered public both in compile-time and runtime.

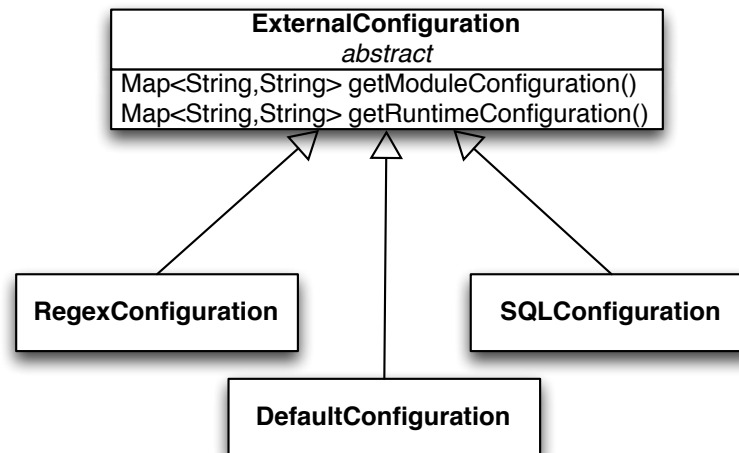


Figure 4.8: External modules configuration hierarchy

J% allows the creation of complex configuration hierarchies to allow easy customisations for each module invocation, using the Java feature of *class field overriding*. The class field overriding is supported fully by the Java programming language specification [110] and allows for each subclass to re-initialise a field with a new value.

Figure 4.9 illustrates such a hierarchy. Consider that the class `CustomConfiguration` is the exported configuration type for an external module. It defines two parameters; namely the `integerParam`, which is an integer, with the initial value of 1 and the `path`, which is a `String`, initialised with the value `/home/bkarak`. To provide two specialisations, we define two new configuration types, `OtherPathConfiguration` and the `OtherIntegerConfiguration`. Their declarations allow to redefine part of the original configuration field values and ultimately provide different configuration contexts. The configuration mechanism adopts the *Convention over Configuration* design paradigm³, and provides by default values on the configuration types parameters that define the typical module compile-time behaviour.

4.1.3 External References

To support type mapping, J% demands a minimalistic syntax extension for each DSL. This is common for all DSLs and called *external references*. These are implemented as placeholders inside the DSL and their use is defined strictly by the developer of the external module. When an external reference is declared, it typically means that the external type expects a value from the Java domain that needs conversion. Thus, they direct the constructor signature for the external type and guide the external module type checker, in terms of type checking. The type mapping facility ensures type compatibility; thus all types used must have registered behaviour, or else a compile-time error will occur.

Each external reference declaration consists of two parts, the *name* and the *Java type*. Consider the following SQL query:

```
select * from customer where
cust_id = #[cust_id]<int> and cust_name = #[cust_name]<java.lang.String>
```

³http://en.wikipedia.org/wiki/Convention_over_configuration

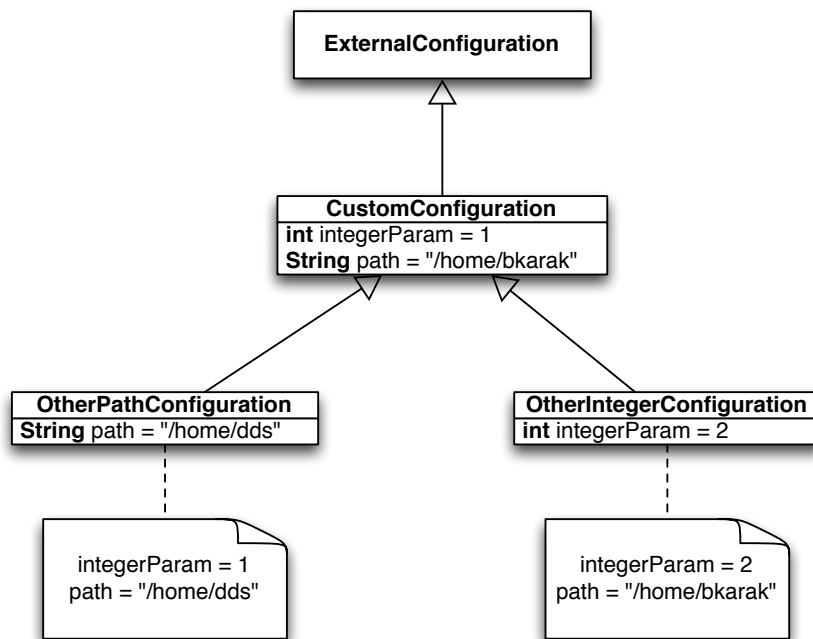


Figure 4.9: Example of configuration hierarchies

The above code excerpt is a standard select query with two external references. In this example, the first parameter is an `int` value and the second a `java.lang.String`. Since these parameters are used to generate the signature of the constructor, one should expect an external type with a constructor of two parameters, an `int` and a `String`. It would be generated like:

```
CustomerQuery(int cust_id, String cust_name) { ... }
```

The constructor convention is followed by all external types, which are defined by the programmers in the language. A corner case, includes multiple usage of an external reference with the same name in the DSL code block:

```
select * from customer
where tax = #[tax]<int> or tax = #[tax]<int> + 1
```

Even though the reference is declared twice, the external module will create a constructor with one `int` parameter. Both external references will be replaced with the appropriate value. If two external references have the same *name*, but different types, a compile-time error will occur.

```
CustomQuery(int tax) { ... }
```

4.1.4 The Type Mapping Mechanism

The type mapping mechanism is realised through a Java interface named `TypeMapping`. Each external module should provide an implementation for it. It defines three methods:

```
public interface TypeMapping {
    public boolean isCompatible(String javaType, String dslType);
}
```

```

    public String javaToDSL(String javaType);

    public String dslToJava(String dslType);
}

```

The method `isCompatible` checks if two types are compatible. The first parameter is from the Java domain (*javaType*) and the second the DSL domain (*dslType*). If the types are compatible (Section 3.3.5), the method returns true. In addition to the above method, `javaToDSL` and `DSLToJava` methods return the *dominant* type for each Java and DSL type accordingly.

The types are defined by a `String`. For the Java types the class name is used, e.g. for the `String` type, the name `java.lang.String` should be used. The DSL types textual representation is defined by each language separately.

Each module may implement the `TypeMapping` interface accordingly. The J% compiler framework provides a default implementation for the `TypeMapping` interface. The `DefaultMapping` class implements the standard interface methods and two methods that permit the addition of mapping rules:

```

public class DefaultMapping implements TypeMapping {
    public DefaultMapping() { ... }

    public void addJavaToDSL(String javaType, String dslType, boolean isDominant) { ... }
    public void addDSLToJava(String dslType, String javaType, boolean isDominant) { ... }

    @Override
    public boolean isCompatible(String javaType, String dslType) { ... }

    @Override
    public String javaToDSL(String javaType) { ... }

    @Override
    public String dslToJava(String dslType) { ... }
}

```

The `addJavaToDSL` method adds a type mapping rule that relates a Java type with a DSL type. The third parameter defines if the type is *dominant* or not. The `addDSLToJava` method does exactly the opposite and provides mapping rules that relates a DSL type with a Java type. Type mapping mechanism should be initialised with each *external module* instantiation. The following code provides a series of type mapping declaration examples.

```

// Initialise the Mapping class
DefaultMapping dm = new DefaultMapping();

// Relate String with char as compatible type
dm.addJavaToDSL("java.lang.String", "char", false);

// Relate String with varchar as dominant type
dm.addJavaToDSL("java.lang.String", "varchar", true);

// Relate varchar with String as compatible type
dm.addDSLToJava("varchar", "java.lang.String", true);

```

```
// should print "true"
System.out.println(dm.isCompatible("java.lang.String", "char"));

// should print "varchar"
System.out.println(dm.javaToDSL("java.lang.String"));

// should print "null"
System.out.println(dm.javaToDSL("varchar"));
```

4.1.5 Code Generation

The code generation process currently generates Java compatible code, but there are no practical restrictions on the output of the compilation process. The process is initialised with the invocation of the method `evaluate` from the appropriate external module (Section 4.1.2). Its specification follows:

```
public abstract class Module {
    [...]
    public abstract boolean evaluate(CodeUnit cu, Map<String, String> ctx)
        throws ModuleException;
    [...]
}
```

This method accepts two parameters; the `CodeUnit` which is processed, and its configuration context. There is no limitation on the behaviour of this method, except that it has to report if the compilation was successful or not. Consider the following example:

```
public boolean evaluate(CodeUnit cu, Map<String, String> context)
    throws ModuleException {
    DSLParser dslp = cu.getDSLParser();
    BaseVelocityWriter bvwm =
        new BaseVelocityWriter("org/jmod/dsl/getset/getset.vm", getName());

    bvwm.add("GS_PACKAGE", cu.getPackage());
    bvwm.add("GS_CLASSNAME", dslp.getExternalType());
    bvwm.add("GS_CLASS_CONFIGURATION", dslp.getConfigurationType());
    bvwm.add("GS_EXTREFS", dslp.getExternalCode().getExternalReferences());
    bvwm.add("GS_GEN_GETTER", context.get("GS_GEN_GETTER"));
    bvwm.add("GS_GEN_SETTER", context.get("GS_GEN_SETTER"));

    return bvwm.save(cu.getSourceFile().getCanonicalOutputFile());
}
```

The method `evaluate` implements the following functionality; it initialises the *BaseVelocityWriter* helper class, then it populates the metaprogramming environment and then generates the code. Metaprogramming facilities can be used for the module development. The J% prototype includes basic support for the Apache Velocity templating engine [95]. It is realised with the implementation of the *BaseVelocityWriter* class. This class serves as a high-level wrapper; it simplifies the library's usage within the compiler context for the module developers.

Table 4.4: Method list of the Compiler class

Method	Usage
<code>compile(Map<K, V>, String[], Writer)</code>	Batch compile of source files with standard parameters. All logging information is redirected to a <code>Writer</code> stream
<code>boolean validate(File f)</code>	Quick syntax validation of a source file
<code>boolean isJava(File f)</code>	Checks if the file contains Java-compatible code
<code>boolean isExternal(File f)</code>	Checks if the file is an External declaration
<code>CompilerContext getCompilerContext()</code>	Returns the <code>CompilerContext</code> instance
<code>ExternalModule[] getModuleList()</code>	Returns the list to the available External modules

```

public class BaseVelocityWriter {
    public BaseVelocityWriter(String tf, String modName) { ... }

    public void add(String key, Object value) { ... }

    public StringWriter prepare() { ... }

    public boolean save(File of) { ... }
}

```

The class is initialised by its constructor that accepts the velocity template as `String`, and the module name (`modName`). The `add` method is used to populate the context for the velocity template. The method `prepare` generates the code and returns it as a `StringWriter` object. The `save` method stores it directly to a specified file. Both the regular expression and the SQL modules use this mechanism for code generation.

On top of that, the compiler supports the in-place replacement of each external reference in the DSL code. Thus, it is trivial to inject Java code inside the DSL code, following specific conversion rules for the values.

Using a templating engine for *external module* development is not mandatory. Simple DSLs like the one described by Spinellis as the *world's smallest DSL*⁴ can be easily supported without using sophisticated parsing engines.

4.2 The J% Compiler Interface

In addition to the command line interface, the J% compiler provides an API that permits to invoke the compiler programmatically if needed. The class diagram in Figure 4.10 illustrates its basic concepts. All the classes that implement the API functionality reside in the package `org.jmod.compiler.api`

The `Compiler` class provides a wrapper for the compiler's internal functionality. The API calls are listed in Table 4.4. Currently, it aims to expose the functionality for syntax check and methods for batch compilation. The method `getModuleList` returns a list with the available *External* modules 4.2.2 and the method `getCompilerContext` returns the singleton instance [101] of the `CompilerContext` class.

⁴<http://www.spinellis.gr/blog/20090203/index.html>

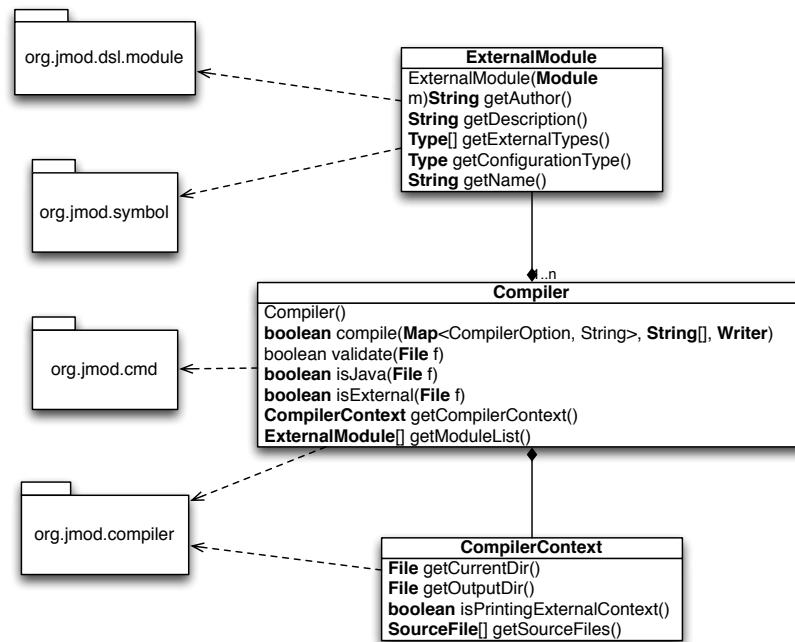


Figure 4.10: Class diagram of the compiler API

4.2.1 The CompilerContext Class

The `CompilerContext` class provides the compiler's runtime information. Table 4.5 summarises its methods. The following listing contains a sample dump of the `CompilerContext` class.

```

Compiler Context Dump
-----
Current Directory: /jmodv2/dist
Output Directory: /jmodv2/dist/work
Generate code only: true
---EXTERNAL modules---
Default module - Default External Module (dummy implementation)
Regex - Regular expression library module
SQLModule - SQL Support Module (based on standard JDBC calls)
--- Source Directories ---
/Users/bkarak/devel/bkarak-svn/trunk/jmod-implementation/jmodv2/dist/examples
--- Source files (Directories included)---
.../jmodv2/dist/examples/SimpleRegex/Main.jmod
.../jmodv2/dist/examples/SimpleRegex/SimpleRegex.jmod
.../jmodv2/dist/examples/SimpleRegex/SimpleRegexConfiguration.jmod
.../jmodv2/dist/examples/simplesql/SimpleSQL.jmod
.../jmodv2/dist/examples/simplesql/SimpleSQLConfiguration.jmod
.../jmodv2/dist/examples/simplesql/SQLMain.jmod
  
```

Table 4.5: Methods list for the CompilerContext class

Method	Usage
File getCurrentDir()	Returns the current working directory as a <code>java.io.File</code> instance
File getOutputDir()	Returns the output directory as a <code>java.io.File</code> instance
boolean isPrintingExternalContext()	Returns <code>true</code> if the compiler is set to dump the compiler context upon compilation
SourceFile[] getSourceFiles()	Returns a list of the source files

Table 4.6: Methods list for the ExternalModule class

Method	Usage
String getAuthor()	Returns a string containing information regarding the module's author
String getDescription()	Returns a string that contains the module's description
Type[] getExternalTypes()	Returns a list of the declared external types
Type getConfigurationType()	Returns the configuration type
String getName()	Returns a short name that identifies this module and is used as the module's identifier by the compiler

4.2.2 The ExternalModule Class

This class contains information for the available external modules. The information includes the author's name, a description and the name of the module. Table 4.6 summarises the methods of this class. The following code exhibits an *External* module enumeration.

```
Compiler c = new Compiler();
ExternalModule[] em = c.getModuleList();

for ( ExternalModule e : em ) {
    System.out.println(e.getName() + " - " + e.getDescription());
}
```

4.2.3 Using the Compiler's API

Assuming that we have a project that has two files, named `Main.jmod` and `Test.jmod`, which belong in the default package and physically are stored in the directory `/project/src/`. To programmatically compile these two files, and generate the output code to the directory `/project/output/`, we need to write the following code.

```
Compiler compiler = new Compiler();
Map<CompilerOption, String> options = new HashMap<CompilerOption, String>();
String[] files = new String[0];

options.put(CompilerOption.OPT_INPUT_DIR, "/project/src");
options.put(CompilerOption.OPT_OUTPUT_DIR, "project/output");
```

```
compiler.compile(options, files, System.out);
```

First, the `Compiler` needs to be initialised. The compilation options are defined in an enum type named `org.jmod.cmd.CompilerOption`. The `compile` method is invoked, with three parameters; the compiler options, the source file arguments, which are none for this case and the `Writer` object. In this example, the compiler is instructed to write the log messages to the standard output (`System.out`).

4.3 Extending J%

J% supports DSL integration by proposing a new reference type marked as external, which isolates the DSL code and performs compile-time checks, in terms of syntax and type safety. This section analyses the process that one needs to follow, in order to add support for a new DSL language by creating a basic external module. The process consists of the following steps:

1. Define the basic module class (Section 4.3.1),
2. Define the module's external types (Section 4.3.2),
3. Define the configuration type (Section 4.3.3),
4. Define the mapping between the languages.

4.3.1 Create the Basic Module

To create a new module, the abstract class `Module` should be extended. The class structure is listed below:

```
public abstract class Module {

    protected Module() { ... }

    @Override public String toString() { ... }
    public boolean isDefaultModule() { ... }
    public static String exportContext(Map<String, String> map) { ... }

    // Abstract methods
    public abstract Type getConfigurationType();
    public abstract Map<String, String> getDefaultConfiguration();
    public abstract boolean evaluate(CodeUnit cu, Map<String, String> context)
        throws ModuleException;
    public abstract String getName();
    public abstract String getDescription();
    public abstract String getVersion();
    public abstract String getAuthor();
    public abstract Type[] getExternalTypes();
    public abstract TypeMapping getTypeMap();
}
```


The developer needs to implement the above abstract methods. Most of them require the implementation of trivial functionality, since they provide information regarding the modules' name, its author, etc. The method `evaluate` implements the code generation process. The following code presents a realisation of a dummy module that has no functionality.

```
public class DefaultModule extends Module {
    public DefaultModule() {}

    @Override
    public boolean evaluate(CodeUnit cu, Map<String, String> context)
        throws ModuleException {
        return true;
    }

    @Override
    public String getAuthor() {
        return "Vassilios Karakoidas (bkarak@aueb.gr)";
    }

    @Override
    public String getDescription() {
        return "Default External Module (placeholder implementation only)";
    }
}

    @Override
    public abstract TypeMapping getTypeMap() {
        return new DefaultTypeMapping();
    }

    @Override public String getName() { return "Default module"; }
    @Override public String getVersion() { return "1.0"; }
    @Override public Type[] getExternalTypes() { return (new Type[] {}); }

    @Override public Type getConfigurationType() {
        return (new Type("org.jmod.dsl.module.def", "DefaultConfiguration"));
    }

    @Override
    public Map<String, String> getDefaultConfiguration() {
        return (new DefaultConfiguration()).getModuleConfiguration();
    }
}
```

The `DefaultModule` class declares a default constructor with no parameters that permits instantiation by the compiler. Since the module implements no functionality, the `evaluate` method simply returns `true`. To register the new external type and the new configuration type to the symbol table, instances of the API's `Type` class are used.

4.3.2 Define the External Base Types

The next step involves the declaration of the external base types. These provide the basic runtime functionality for the external types, since each user-defined external type is a specialisation for one of them. The code below defines a hypothetical external base type that supports regular expressions using the standard regular expression library.

```
public class Regex<T extends RegexConfiguration> extends ExternalBaseType<T> {
    private String regex;

    public Regex(String regex, T configuration) {
        super(configuration);
        this.regex = regex;
    }

    public Pattern getPattern() {
        return Pattern.compile(regex);
    }
}
```

The only convention that must be followed by the base type, is the proper initialisation of the parent constructor. Other than that, each developer is free to implement whatever functionality within the encapsulated type and define the API that will support the functionality at runtime.

Upon the compiler initialization, each external module is queried for its external and its configuration type. The following code excerpt exhibits the declaration of new types inside the hypothetical regular expressions module.

```
public class RegexModule extends Module {
    [ ... ]
    @Override public Type[] getExternalTypes() {
        return ( new Type[] { new Type("org.jmod.dsl.module.regex", "Regex") } );
    }

    @Override public Type getConfigurationType() {
        return (new Type("org.jmod.dsl.module.regex", "RegexConfiguration"));
    }
}
```

4.3.3 Define the Configuration Types

The configuration types support the compilation process, by providing each call of an external module with its execution context. The following code is an example of a configuration type, for the regular expression module.

```
public class RegexConfiguration extends ExternalConfiguration {
    protected String REGEX_ENGINE = "firej"; // firej, jdk
    protected String REGEX_OUTPUT = "java"; // class, java

    // just ensure it is public
    public RegexConfiguration() { }
```

```

@Override
protected Map<String, String> getModuleConfiguration() {
    Map<String, String> result = new HashMap<String, String>();

    result.put("REGEX_ENGINE", REGEX_ENGINE);
    result.put("REGEX_OUTPUT", REGEX_OUTPUT);

    return result;
}

@Override
public Map<String, String> getRuntimeConfiguration() {
    Map<String, String> r = getModuleConfiguration();

    r.remove("REGEX_OUTPUT");

    return r;
}
}

```

Each configuration type must have a public constructor that requires no parameters. The above code defines a configuration type, which has two fields; namely `REGEX_ENGINE` and `REGEX_OUTPUT`. These are initialised with two default values that can be overridden if necessary, and create a hierarchy of configurations. Each configuration should implement two methods, `getModuleConfiguration` and `getRuntimeConfiguration`. The above code exhibits a case, where the module developer declares the `REGEX_OUTPUT` configuration field private, by choosing not to expose it to the `getRuntimeConfiguration` method.

4.3.4 Type Mapping

Section 4.1.4 describes in detail how type mapping is implemented in J% and how to use it. Each module must implement the `TypeMapping` interface, or use the `DefaultTypeMapping` helper class. The following code, exhibits how to “disable” the type mapping mechanism, for a DSL that does not need this facility.

```

[...]
/* extending the abstract Module class */
@Override
public TypeMapping getTypeMap() {
    return new DefaultTypeMapping() {
        @Override
        public boolean isCompatible(String javaType, String dslType) {
            return (javaType.compareTo(dslType) == 0);
        }

        @Override
        public String javaToDSL(String javaType) {
            return javaType;
        }
    }
}

```

```

    @Override
    public String dslToJava(String dslType) {
        return dslType;
    }
};
}

```

4.4 Regular Expression Module

The theoretical background of regular expressions lies within automaton theory and formal languages. Regular expressions belong to a type 3 grammar of the Chomsky hierarchy. This hierarchy, described by Chomsky in 1956 [54], provides a categorization of grammars that describe formal languages.

In 1943 Warren McCulloch and Walter Pitts described the human neural system using automata [184]. The mathematician Stephen Kleene described the proposed models with a mathematical notation named *regular sets* [153]. Later Brzozowski [45] provided mathematical definitions for the Kleene regular expressions formalism, and introduced ways to convert regular expressions into state diagrams. In the late 1960s, Ken Thompson proposed a compiler that translated a regular expression into the assembly language of the IBM 7094 processor [250]. Later on, he implemented the *regular sets* in his text editor named *qed*, and afterwards on *ed*, which became part of the UNIX distribution [16]. From then on, regular expressions became widely used in almost all UNIX variants.

Ehrenfeucht and Zeiger [80] provided the academic community with four metrics that measured the complexity of regular expressions. In 1980, Ernst Leiss [165] presented an algorithm for constructing a finite automaton from a given regular expression, and two years later Floyd and Ullman [93] proposed an approach for compiling regular expressions into integrated circuits. Sensory Networks [4] constructed a content classification accelerator that implements a regular expression engine in hardware.

Regular expressions have also become a standard feature in many programming languages. Programming languages like *Java*, *Ruby* and *Python* contain regular expression engines as part of their API and Perl also defines a set of operators and integrates regular expressions into its syntax [272]. In addition, Perl permits a grammar definition with syntactic rules expressed with regular expressions [76]. SQL has been extended to use regular expressions patterns in its syntax [82, 245]. Most modern UNIX variants provide regular expression facilities built into the C library (*libc*).

In 1992, Wu and Manber [282] introduced bit-parallel searching. Navarro and Raffinot introduced in 2005 two new techniques for regular expression searching that produce faster results [195]. Their idea is based in bit-parallel matching [19, 282] and Glushkov's Non-Deterministic Finite Automata (NFA) construction algorithm [109]. This approach is implemented in the *NRGrep* tool [193]. Navarro [194] also introduced a way of searching text patterns on compressed data.

Regular expressions are mainly used in a variety of software projects as a standard way to perform pattern matching and text validation. Lex [167] introduced a method to write lexical analysers based on regular expressions and automatically generate their code. Nowadays, lex is considered a classic lexical analysis tool [8, 139].

A typical regular expression engine is defined by two main characteristics: (1) *engine type* and (2) *syntax standard*. According to their *engine type*, regular expression implementations can be divided into two main categories [8, 100]:

Table 4.7: Time & Space complexities for NFA and DFA automaton

Automaton Type	Space	Time	Common Implementations
NFA	$O(r)$	$O(r \times n)$	<i>grep, awk, perl, j2sdk</i>
DFA	$O(2^r)$	$O(n)$	<i>tcl, automaton, FIRE/J</i>

NFA (Non-deterministic Finite Automaton) The engine represents the regular expressions as a non-deterministic finite automaton. NFA-based libraries are slower, but allow syntactically richer regular expressions. NFA based engines are categorized into Traditional NFA and Portable Operating System Interface for Unix (POSIX) NFA engines. NFA engines have typically slower execution time than Deterministic Finite Automata (DFA) [100]. NFA-based engines are known as “regular expression-directed”, indicating that the engine dictates the matching process.

DFA (Deterministic Finite Automaton) DFA-based implementations construct a deterministic automaton to represent the regular expression. DFA-based engines are faster and predictable, returning always the longest leftmost match. They are more difficult to implement and lack certain features, such as backtracking. DFA-based engines are known as “text-directed”.

A finite automaton is a graph where nodes represent states and edges represent transitions among them. Each transition has rules that act as prerequisites while traversing the automaton, depending on the input data. Each engine type has different requirements in space and time depending on the regular expression and the input sequence length. If we define as r the regular expression length and n the character input sequence size, the algorithmic complexities of NFA and DFA based engines are shown in Table 4.7 [8]. There are also implementations that combine characteristics from these two categories. These are known as *hybrid implementations* [100].

Regular expressions come in three dominant syntax standards. We emphasize here that the syntax does not imply a corresponding engine type. These are:

Traditional UNIX regular expressions The traditional unix regular expression was the first syntax that was defined and implemented. It has now been replaced by the POSIX standard, although most unix applications and utilities, such as *sed* and *grep*, provide support for it. Notably, its syntax does not provide the union operator (``|'`).

POSIX (extended) modern regular expressions POSIX extended regular expressions to provide an extension to the aforementioned syntax [115]. Three new operators are introduced: (1) *union* (``|'`), (2) *zero or one matching* (``?''`) and (3) *one or more matching* (``+''`). Table 4.8 illustrates the most common operators of traditional unix and POSIX standards. POSIX also provides a portable way to define character classes. e.g. `[:: lower ::]` is `[a - z]`. These classes provided a quick and compact way to solve internationalization problems. For example, the Greek language features an alphabet different than the Latin, so in order to write a generic regular expression that matches non-capital alphanumeric characters, an abstract form of character representation is needed to transparently match text according to the system’s locale. The Unicode technical committee also proposed a technical standard on Unicode support in regular expression engines [68].

Perl compatible regular expressions Perl Compatible Regular Expressions (PCRE) extended the syntax to provide functions for *back-referencing*, *assertions* and *conditional sub-expressions* [272].

Table 4.8: Typical regular expressions operators

Operator	Description	Example	TR-Unix	POSIX
c	character match	a	✓	✓
.	Any character match	a.*a	✓	✓
[]	Character group definition	[a-z]	✓	✓
[^]	Negated group definition	[^a-z]	✓	✓
c*	zero or more character match	a*	✓	✓
c+	one or more character match	a+	✗	✓
c?	zero or one character match	a?	✗	✓
^	start of line	^aaab	✓	✓
\$	end of line	aaab\$	✓	✓
c{x,y}	x minimum to y maximum occurrences of character c	a{1,2}	✓	✓
r c	character r or c match	a b	✗	✓

Table 4.9: Examples of regular expression

Regular Expression	Example
[1-9][0-9]*	1, 2, 3, 4, ...
([a-f])	a, b, c, d, e or f
a+	a, aa, aaa, aaaa, ...
(m n kn r)ight	"might", "night", "knight" and "right"
(([0-9]+)? (([0-9A-F]+)?))	1, 1A, AA, BF, FF, ...

Table 4.9 contains examples of common regular expressions. The first example illustrates a regular expression that matches decimal numbers. Note that the parentheses (`((', '))`) have a dual use, it can be used both to define groups and to encapsulate sub-expressions. The fifth regular expression provides an example of its usage. An interesting example is the fourth, which depicts how easily we can match with a simple regular expression similar words like *might*, *night*, *knight* or *right*.

4.4.1 Module Architecture

Figure 4.11 contains the UML class diagram for the regular expression module. It exports only the *external type*, named `Regex`. The main module entry point is the `RegexModule` class. The declared *external configuration type* is named `RegexConfiguration`.

The module performs syntax check for each regular expression and reports the errors at compile-time. By default, it is backed-up by the JDK regular expression engine, which implements an NFA-based automaton. By setting the configuration parameter `REGEX_ENGINE` to "firej", then the module generates code that uses the FIRE/J engine [146]. This is implemented with the *AbstractRegex* class, which simply follows the *Strategy* design pattern [101] to easily provide support for both engines.

Table 4.10 provides the values of size metrics for the source code of this module. The implementation of the `Regex` module involved only 6 files and 314 lines of Java code. The lines of code for this module are quite low, since J% encourages the re-usability of the regular expression libraries and reuse of existing *APIs*.

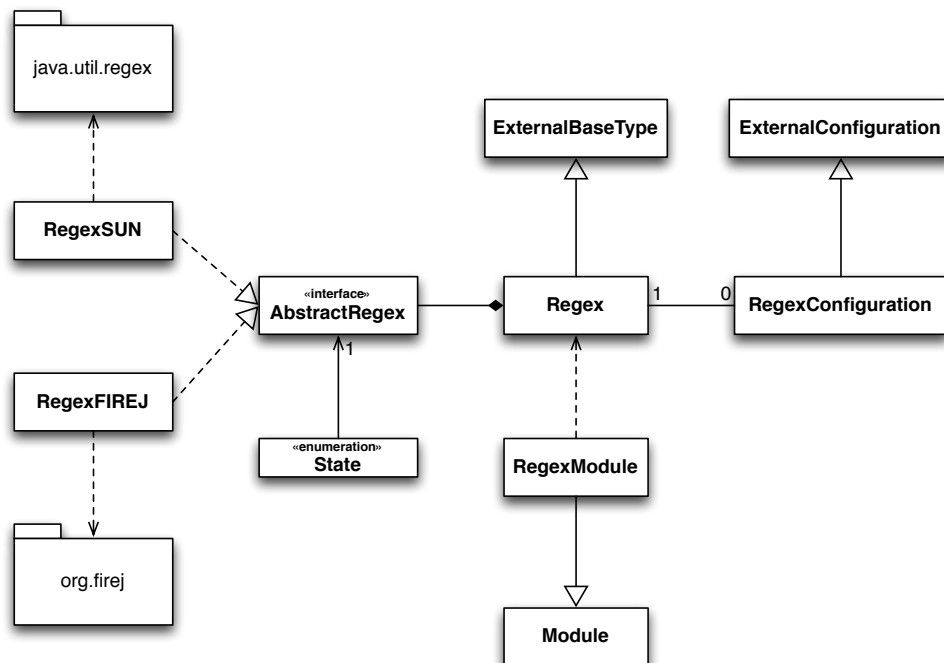


Figure 4.11: Regular expression module class diagram

Table 4.10: Regular expression module size metrics

Metric	Value
File Count	6
Line Count	341
Lines of Code	314
Comments Lines of Code	27
Package Count	2
Number of Classes	7
Number of Interfaces	1
Number of Enumerations	1

Table 4.11: Regex type public methods

Method	Description
<code>void setBuffer(CharSequence)</code>	Resets the data buffer and replaces it with the provided <code>CharSequence</code> , then sets the provided <code>CharSequence</code> as input.
<code>boolean matches(CharSequence)</code>	Resets the data buffer and replaces it with the provided <code>CharSequence</code> , then attempts to match the entire <code>CharSequence</code> against the regular expression pattern.
<code>boolean matches()</code>	Attempts to match the entire input buffer against the regular expression pattern.
<code>State find(CharSequence)</code>	Resets the data buffer and replaces it with the provided <code>CharSequence</code> , then attempts to find a partial match in the input buffer of the regular expression pattern.
<code>State find()</code>	Attempts to find a partial match in the input buffer of the regular expression pattern.
<code>String replace(String)</code>	Replace the first partial match in the input buffer with the provided constant string.
<code>String replaceAll(String)</code>	Replace all the matches in the input buffer with the provided constant string.
<code>Match group()</code>	Returns an instance containing the match sequence.
<code>Match group(int)</code>	Returns an instance containing a match subsequence, provided a valid index.

Table 4.12: Regex module configuration parameters

Name	Type	Description	Values
<code>REGEX_ENGINE</code>	<code>String</code>	Select regular expression backend engine	“jdk” or “firej”
<code>REGEX_OUTPUT</code>	<code>String</code>	Code generation output	“java” or “class”

4.4.2 External Type and Configuration Parameters

The regular expression module declares only one external type, named `Regex`. Table 4.11 summarises the major API calls provided by the type. The features of the API provided, is similar with the standard library `java.util.regex`, which is shipped with the JDK. The methods `matches` and `find` are used to match the entire or partial character buffer sequence against the regular expression pattern.

The external configuration type this module provided is named `RegexConfiguration`. It introduces two parameters; `REGEX_ENGINE` that selects regular expression back-end engine and `REGEX_OUTPUT` that instructs FIRE/J to generate tailor-made Java compatible code or just executable bytecode (Java class file). `REGEX_OUTPUT` is active only if FIRE/J is selected. Since the external configurations follow the *convention over configuration* [280], by default the `REGEX_ENGINE` is set to “jdk” and the `REGEX_OUTPUT` to “java”.

Figure 4.12 illustrates a typical hierarchy of configurations. The `OptimisedRegex` configuration enables FIRE/J as back-end library by setting the value “firej”.

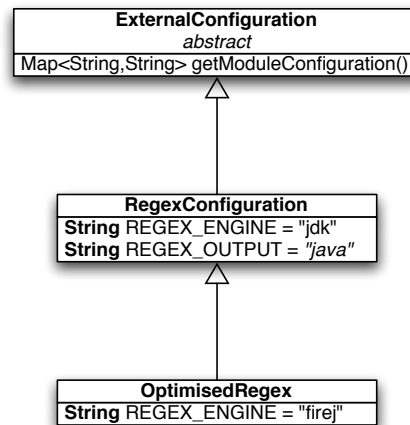


Figure 4.12: Regular expression module configuration hierarchy

4.4.3 Code Generation

The code generation is implemented with the usage of the template engine velocity [95]. Consider the following code, a regular expression that matches an IP Address (IPv4).

```

package examples.simpleregex;

import org.jmod.dsl.regex.Regex;
import org.jmod.dsl.regex.RegexConfiguration;

public external IPAddress extends Regex<RegexConfiguration> {
    ([0-9]{1,3}\.){3}[0-9]{1,3}
}
  
```

will be transformed into the following Java compatible code,

```

package examples.simpleregex;

import org.jmod.dsl.regex.Regex;
import org.jmod.dsl.regex.RegexConfiguration;

public class IPAddress extends Regex<RegexConfiguration> {
    private final String _regex = "([0-9]{1,3}\.){3}[0-9]{1,3}";

    public IPAddress() {
        super(_regex, new org.jmod.dsl.regex.RegexConfiguration());
    }
}
  
```

The *velocity* template that is used for the code generation is presented in the Appendix F.

4.4.4 Domain-Specific Optimisation

J% regular expression module supports DSOs through the utilisation of the FIRE/J application library [146]. When used, FIRE/J generates a tailor-made class for each regular expression, offer-

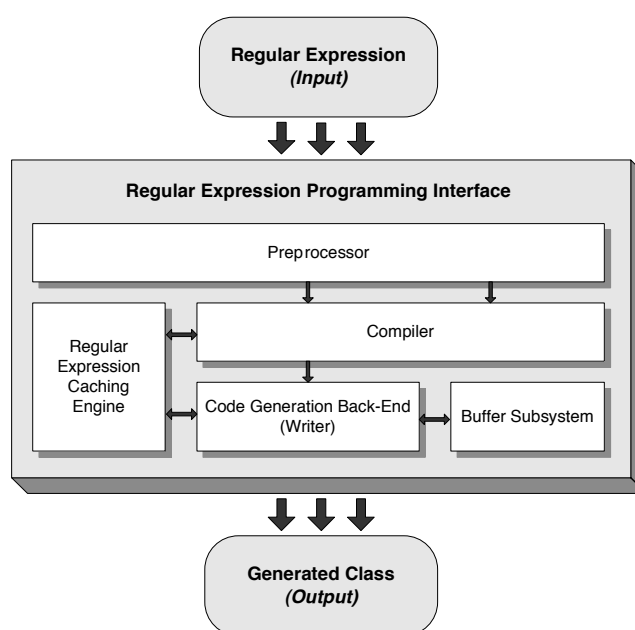


Figure 4.13: Architecture of the FIRE/J Engine

ing great optimisations in terms of performance. The optimisation technique and extensive performance tests are included in the Appendix B.

Architecture

FIRE/J is developed as a testbed for direct code generation from a regular expression into Java source code and JVM bytecodes. It is compatible with the J2SDK 1.5.x Regular Expression API. Currently, the FIRE/J library implements the POSIX standard.

Regular expressions can be considered as a DSL that deals with the text-parsing and manipulation. All regular expression libraries in Java are designed using the *piggyback* pattern, as Spinellis categorized in reference [238], where a DSL is being designed on top of an existing host language. FIRE/J is designed using the *source-to-source transformation* creational pattern, where a DSL is compiled into the host language code, which in our case are JVM *instructions*.

Figure 4.13 illustrates the architecture of our implementation. FIRE/J accepts as input a regular expression, and generates a compiled Java class as output. Internally, it consists of five distinct components; (1) a *Preprocessor*, (2) a *Compiler*, (3) *Code Generation Back-ends*, (4) a *Regular Expression Caching Engine*, and (5) the *Buffer Subsystem*.

The *preprocessor* translates regular expressions macros, such as character classes, into a simpler form. For example, the character class “ $\backslash p\{Alpha\}$ ” is automatically replaced by its equivalent $[a - zA - Z]$. This component is also used for internationalization. If the locale is correctly set, then the aforementioned character class will be replaced with the appropriate locale specific range e.g. in Greek, $\backslash p\{Alpha\}$ should be $[\alpha - \omega A - \Omega]$. The *preprocessor* can also be extended to work at the regular expression syntax level e.g. the expression $ab + a$ can be translated into its equivalent $abb * a$. The *compiler* constructs a DFA from the simplified regular expression.

Code generation back-ends translate the generated automaton into specific code. In the FIRE/J distribution, we provide two types of back-ends: an experimental that writes Java code and the production one that directly generates JVM bytecodes.

Code generation and runtime class assembling proved to be an expensive procedure. To cope

with the problem we implemented a caching layer. The *regular expression caching engine* provides a transparent mechanism to store the processed regular expressions and retrieve them when necessary. The way it works is similar to the *compile once* (``o'``) switch in Perl's operator ``m/'`` [272] and the *CompileToAssembly* method in the standard .net regular expression library. The engine provides caching for both the *compiler* and the *code generation back-end* modules. The *regular expression caching engine* is a singleton object [101] and has only one instance per virtual machine. Consequently cached objects, such as instances of compiled regular expression can be accessed by any application in the virtual machine using our regular expression library, and benefit by the precompiled regular expressions. This feature has significant performance improvement in server-side applications, like web servers and servlet/Java Server Pages (JSP) engines, such as Tomcat, where server and applications share the same JVM.

The *buffer subsystem* provides a standard means of accessing the input data that will be parsed. In order to improve performance, it is tightly integrated with the generated Java class. This layer is optional; therefore a module developer could override it, and implement his/her own buffering strategy.

FIRE/J generates code in order to provide performance optimization in the execution of each regular expression and it could be compared with other code generators, such as *lex* [167], *jlex* [30], *flex* or their Java-based clone *jflex*. FIRE/J shares some common characteristics with such tools:

Code Generation All the above tools generate code according to a regular expression based syntax.

Automaton Based FIRE/J and lexical analysers generate code that represents a finite automaton.

It has also several differences:

Scope FIRE/J as a regular expression engine has a very narrow scope. Lex variants on the other hand are used for lexical analysis. Lex could have a FIRE/J engine embedded as a building block.

Target Code Lex generates high-level code (Java and C) and depends on the compiler to generate the executable code, while FIRE/J generates JVM bytecodes.

Run-time vs. Compile-time FIRE/J is generating the executable code at run-time, while lex generates source code that is statically linked in a bigger program, or compiled and executed separately.

Code Generation

State machines are typically implemented either as a large `switch` statement, where each case represents a different case, or through code blocks linked together via `goto` statements. The `switch`-based method is easier to implement with integer-based lookup tables, but incurs the overhead of an additional level of indirection—the mapping from the integer identifier of the state to the corresponding machine address containing the code. To avoid this overhead and get around the fact that Java lacks a `goto` statement, FIRE/J generates code as portable machine instructions. As an aid to debugging and to measure the improvement of the JVM code generation technique, we also implemented a back-end that writes `switch`-based code in Java.

The Java generator compiles the given DFA directly into Java source code. Then, the Java compiler is invoked. Algorithm 1 shows the basic structure of the generated code, which follows

Algorithm 1 Generated code structure for a goto-less language

```

1: procedure match(char[] input, DFA[] states):Boolean
2:    $s \leftarrow 1$  ▷ The state counter gets the starting state
3:   for  $i \leftarrow 0 \dots \text{input.length}$  do ▷ For all the characters in the input buffer
4:     switch ( $s$ ):
5:       case 1: ▷ For state 1
6:          $s \leftarrow \text{check\_state}(\text{states}[1], \text{input}[i])$  ▷ Check the input character
7:         if  $s = -1$  then ▷ if input character is not accepted return false match
8:           return false
9:         end if
10:        break ▷ Else continue with the next character
11:        ...
12:      case  $n$ :
13:         $s \leftarrow \text{check\_state}(\text{states}[n], \text{input}[i])$ 
14:        if  $s = -1$  then
15:          return false
16:        end if
17:        break
18:      end for
19:      return isTerminalState( $s$ ) ▷ If the state is terminal return true, else return false
20: end procedure

```

the same principles described by Thompson [250] and adopted by code generators such as *lex* [167] and *jlex* [30].

The algorithm receives as input a character sequence and the DFA states array, and returns a Boolean value indicating if a full match has been achieved. The generated code is a big loop with a *switch-case* statement. Starting from the initial state, all the characters in the input buffer are being checked against the transition rules of the DFA states. If a character does not satisfy the selected state, then the match fails and the procedure exits. When the input buffer runs out of character data, the last visited state is checked. If the state is terminal then the procedure returns *true*; otherwise it returns *false*. The structure of this algorithm is similar to the code generated by the lexical analyser *lex*. In our implementation we have inlined all the methods, such as *check_state* and *isTerminalState*, in order to avoid the method call overhead [183].

The basic structure of the bytecode writer is listed as Algorithm 2. The code imposes the same logic as the one described in the previous algorithm. Their main difference lies in the state selection procedure. When using low-level JVM instructions, the *goto* instruction can be used. By using it, each state jumps directly to the appropriate memory address, instead of executing the *switch* statement used in Algorithm 1.

If we define as n the number of states of the automaton, and m the size of the character sequence for the Algorithm 1, a total of $n \times m$ of steps are needed to perform a full match, if the switch is naively compiled as a linear sequence of test and branch instructions. Consequently, the complexity of Algorithm 1 is $O(n \times m)$. For Algorithm 2, the required steps would be $O(m)$, because each state jumps directly to the next or fails.

The following code fragment is generated by the module, for the regular expression $[0-9] [0-9]^*$. Two files are generated, one that contains the regular expression type and the tailor-made class.

```

package examples.simpleregex;

import org.jmod.dsl.regex.Regex;

```

Algorithm 2 Generated code structure with goto

```

1: procedure match(char[] input, DFA[] states):Boolean
2:    $s \leftarrow 1$                                 ▷ The state counter gets the starting state
3:    $c \leftarrow 0$                                 ▷ Set the input buffer counter to
4:   goto  $s$                                        ▷ Jump to initial state
5:
6:   State_1:
7:   if  $c = \text{input.length}$  then                ▷ if we reach the end of the buffer, return true if Terminal
8:     return isTerminalState( $s$ )
9:   end if
10:   $s \leftarrow \text{check\_state}(\text{states}[1], \text{input}[c])$                                 ▷ Get next state
11:   $c \leftarrow c + 1$ 
12:  if  $s = -1$  then                                ▷ if input character is not accepted return false match
13:    return false
14:  end if
15:  goto  $s$                                        ▷ Jump to next state
16:  ... ..
17:  State_n:                                       ▷ Same code for the remaining states
18:  if  $c = \text{input.length}$  then
19:    return isTerminalState( $s$ )
20:  end if
21:   $s \leftarrow \text{check\_state}(\text{states}[n], \text{input}[c])$ 
22:   $c \leftarrow c + 1$ 
23:  if  $s = -1$  then
24:    return false
25:  end if
26:  goto  $s$ 
27: end procedure

```

```
import examples.simpleregex.SimpleRegexConfiguration;

public class SimpleRegex extends Regex<SimpleRegexConfiguration> {
    private final String _regex = "[0-9][0-9]*";

    public SimpleRegex() {
        super(_regex, new SimpleRegexConfiguration());
    }
}
```

The name of the tailor-made class (`ttflcfek`) is automatically generated by the FIRE/J engine and it is not visible to the programmer.

```
import org.firej.writers.abstractRegex.CharArrayRegex;
import org.firej.writers.*;

public final class ttflcfek extends CharArrayRegex {
    public ttflcfek() {
        super("[0-9][0-9]*");
    }

    public int exec(int offset) {
        int len = length;
        int state = 1;
        char[] arr = arrayBuffer;
        int returnValue = -1;

        for (int i = offset; i < len; i++) {
            int c = (int)arr[i];
            switch(state) {
                case 0:
                    if ( (c >= 48) && (c <= 57) ) {
                        state = 0;
                        returnValue = i;
                        break;
                    }
                    return returnValue;
                case 1:
                    if ( (c >= 48) && (c <= 57) ) {
                        state = 0;
                        returnValue = i;
                        break;
                    }
                    return returnValue;
            }
        }
        return returnValue;
    }
}
```

4.5 SQL Module

SQL stands for Structured Query Language and is the de facto standard DSL for database queries. Although SQL is an ANSI standard [135], there are many different dialects of the SQL language, usually accompanying each Relational DataBase Management System (RDBMS). In general terms, SQL offers the following features:

- execute queries and retrieve data from a database,
- insert, update, delete records in a database,
- create new databases and tables,
- create stored procedures in a database,
- create views in a database,
- set permissions on tables, procedures, and views.

The prototype implementation of the J% compiler includes a module that supports integration SQL. This case is more challenging than regular expressions, since it also supports type mapping, along with syntax checking. The syntax analysis is based on a custom SQL parser, and the generated code uses standard JDBC calls. We follow the type mapping scheme proposed by the JDBC specification [91].

4.5.1 Design & Features

Consider the case of a typical Java application that needs access to a relational database. A typical application should contain a class like the one listed below:

```
public class SQLTest {
    public void sqlQuery(int id) {
        try {
            Statement stmt = connection.createStatement();
            ResultSet rs =
                stmt.executeQuery("SELECT * FROM t where id = " + id);
[...]
```

```
        } catch (SQLException e) { /* error */ }
    }
}
```

The class `SQLTest` contains a method called `sqlQuery` that accepts an integer (`int`) parameter, labelled `id`. The parameter is concatenated with an SQL query, which is later executed. This is a simple and common scenario that all database application developers have encountered.

What is the problem with above code; The following issues are very typical after thorough examination of the code:

1. The SQL query is passed as a `String` (`java.lang.String`) and no-one, except the developer, knows that this is actually an SQL query at compile-time.
2. The parameter (`id`) that is concatenated to the SQL query is never checked in any way. It may have an invalid value, or simply produce a malformed query.

These deficiencies can lead to many problems, since they can be detected by common software engineering testing practices at runtime only. It is very likely to lead to data corruption or create security risks if they are detected and exploited by malicious users.

J% SQL deals with the aforementioned deficiencies with the introduction of the following features:

Query Validation The SQL queries are syntactically checked at first and if the feature is enabled, they can also be validated against a specified database schema, which is written in SQL. In addition, the SQL queries can also be executed in a live database environment at compile-time.

JDBC Compatibility The SQL module utilises the existing JDBC framework, in an attempt to avoid the alienation of the developers with new APIs. The type mapping also follows the standard JDBC conventions [91].

Transparent Usage of Prepared Statements The generated code utilises prepared statements and all queries are parameterised correctly.

Extensive Query Configuration Each query can be separately configured with different code generation and testing features. This is implemented with the utilisation of the *external configuration* concept, which is introduced by J%.

Security The J% SQL module supports signature-based intrusion detection mechanism that guards applications against SQL injection attacks [163, 189]. Specifically, the module can be used to automatically create all the signatures required by the SDriver/SQL secure JDBC library. Typically, a signature based approach requires a training phase, which is manually executed, gathers and generate all SQL code signatures.

4.5.2 Module Architecture

Figure 4.14 presents the module architecture as a UML class diagram. The SQL module declares only one type, named `SQLQuery`. Its implementation is based on standard JDBC calls, in order to be easily integrated into existing applications. Internally it uses prepared statements, to safely initialise each SQL query with the appropriated data. Its declaration follows:

```
public abstract class SQLQuery<T extends SQLConfiguration>
    extends ExternalBaseType<T> {
    public String getSQLStatement();

    public abstract PreparedStatement getStatement(Connection c)
        throws SQLException;
}
```

The above code declares the `SQLQuery` external base type, which extends the *ExternalBaseType* class. The type must be parameterised by the `SQLConfiguration` type, or one of its subtypes. Its public functionality includes two methods; `getSQLStatement`, which returns the SQL query as a string and `getStatement` that returns an instance of a `PreparedStatement`, given an initialised and open JDBC Connection.

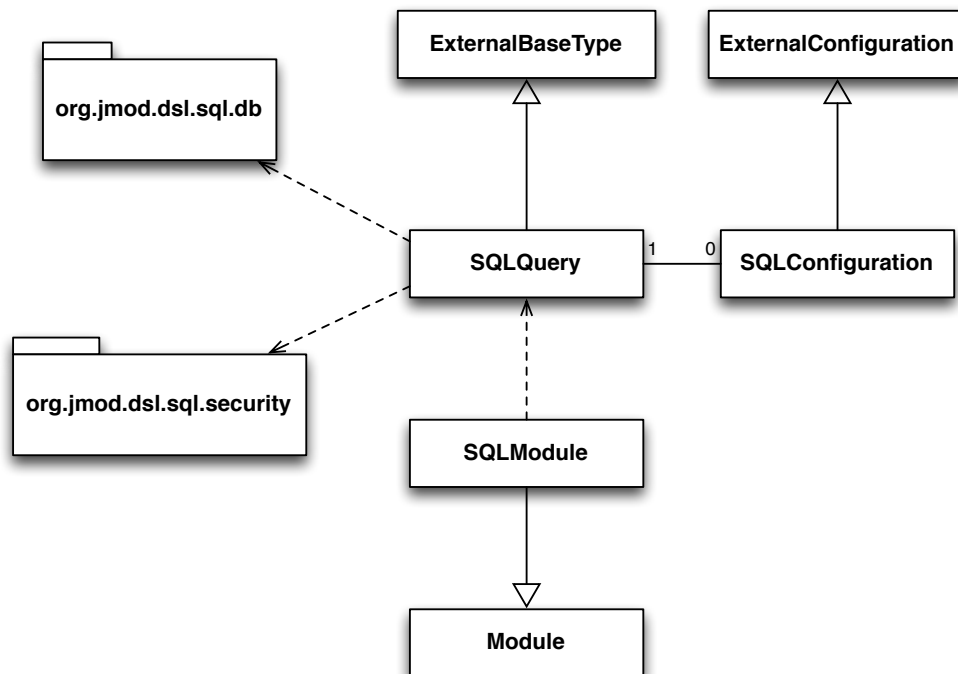


Figure 4.14: SQL module architecture

Table 4.13: SQL Module Size Metrics

Metric	Value
File Count	23
Line Count	2,785
Lines of Code	2,417
Comments Lines of Code	368
Package Count	5
Number of Classes	22
Number of Interfaces	0
Number of Enumerations	1

Table 4.14: Configuration Parameters

Name	Description
SQLMOD_NS_AWARE	Enable/Disable SQL check with a database schema
SQLMOD_NS_URI	Location of the Database Schema
SQLMOD_LIVE_TEST	Enable/Disable Live Database Testing
SQLMOD_JDBC_DRIVER	The JDBC driver classpath
SQLMOD_DB_URL	JDBC-compatible database URL
SQLMOD_DB_LOGIN	Live database test: user login
SQLMOD_DB_PASSWORD	Live database test: user password

Table 4.15: Configuration Parameters Requirements

Name	Required By
SQLMOD_NS_AWARE	
SQLMOD_NS_URI	SQLMOD_NS_AWARE
SQLMOD_LIVE_TEST	
SQLMOD_DB_URL	SQLMOD_LIVE_TEST
SQLMOD_JDBC_DRIVER	SQLMOD_LIVE_TEST
SQLMOD_DB_LOGIN	SQLMOD_LIVE_TEST
SQLMOD_DB_PASSWORD	SQLMOD_LIVE_TEST

4.5.3 Configuration Parameters

Table 4.14 summarises all available configuration options of J%'s SQL module. The configuration parameters can be divided in two categories, those that are used for off-line syntax check against the database schema and the rest that are used only if live database testing is enabled.

There are interdependencies between the parameters and if a dependency check fails e.g. live database testing (SQLMOD_LIVE_TEST) is enabled without the definition of the database Uniform Resource Location (URL) (SQLMOD_DB_URL), then the compiler will report it as a compile-time error and the compilation process will fail. Table 4.15 lists the requirements of each configuration parameter.

Figure 4.15 depicts a typical configuration hierarchy. SQLConfiguration is the basic configuration type of the SQL module. TestingConf and LiveTestingConf are two subtypes which override and set specific compile-time options.

4.5.4 SQL Types and Type Conversions

All the possible dominant and compatible relations between the Java and the SQL are defined exhaustively. Table 4.16 lists all the compatible and dominant types for the SQL module. Dominant relations are marked with italics.

4.5.5 Code Generation

A typical J% application that communicates with a relational database and needs to execute an SQL query, should include a custom *external type*. The source code typically looks like the following fragment,

```
package examples.simplesql;
```

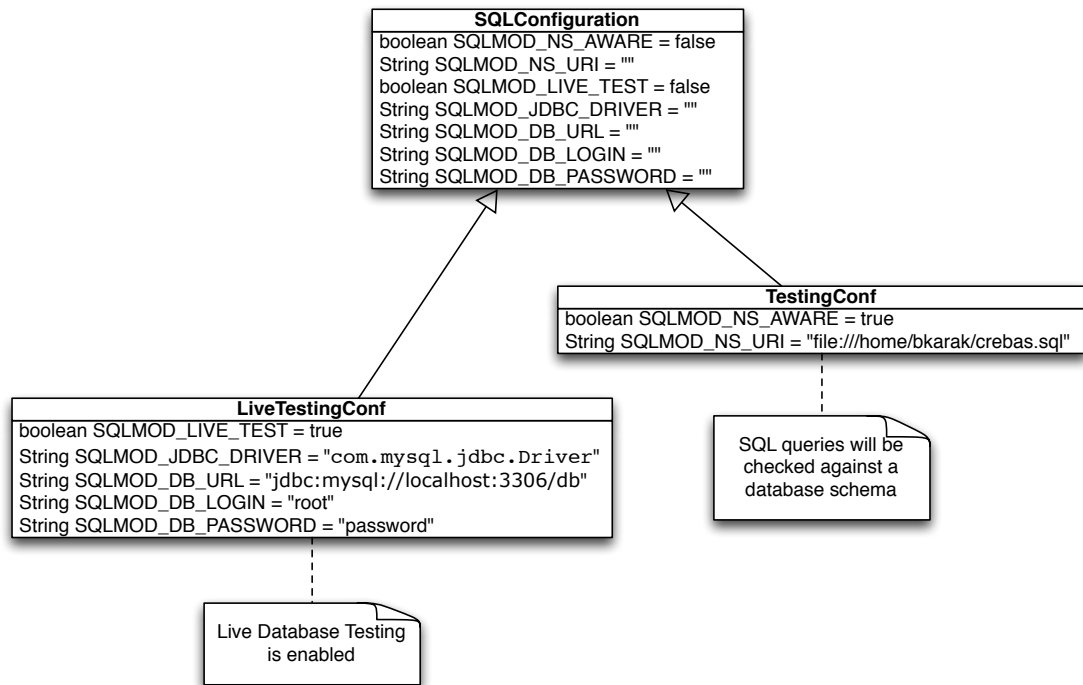


Figure 4.15: A typical configuration type hierarchy

Table 4.16: Type mapping definitions for the MySQL database dialect

Compatible	Dominant Types		Compatible
Java	Java	SQL	SQL
java.lang.Byte	byte	tinyint	-
java.lang.Short	short	smallint	-
java.lang.Integer	int	int	integer, mediumint, year
java.lang.Long	long	bigint	-
java.lang.Float	float	float	-
java.lang.Double	double	double	-
-	java.lang.BigDecimal	decimal	numeric
java.lang.Boolean	boolean	bit	bit varying
-	java.lang.String	varchar	character, char, tinytext, text
-	byte[]	BLOB	tinyblob, mediumblob, longblob, longtext
-	java.util.Date	date	-
-	java.util.Timestamp	timestamp	time

```
import org.jmod.dsl.sql.SQLQuery;
import examples.simplesql.SimpleConf;

public external SelectExample extends SQLQuery<SimpleConf> {
  select * from sqlexample where sqle_primary = #[prim]<int>
}
```

The `SelectExample` external type inherits the `SQLQuery` external base type, defined in the SQL module. The query is written in SQL. There are two things, which are not common knowledge to a typical database Java developer; the external type declaration statement with the parameterised `SQLQuery` type and the `#[prim]<int>` addition to the query. The `#[prim]<int>` is the declaration statement of an external reference. It declares that the SQL query needs an integer parameter to work and it is implemented by a formal parameter in the generated type's constructor.

The declaration of the external configuration type, namely `SimpleConf`, follows:

```
package examples.simplesql;

import org.jmod.dsl.sql.SQLConfiguration;

public class SimpleConf extends SQLConfiguration {
  public boolean SQLMOD_NS_AWARE = true;
  public String  SQLMOD_NS_URI = "file://./schema.sql";
}
```

The `SimpleConf` type parameter configures the compile-time part of `SQLQuery`. In this example it instructs the compiler to check the query with the provided schema of the database. It is a typical Java class and requires no special notation. A part of this configuration is passed into the runtime environment, enabling the developer to retrieve information regarding the code generation process. For example, the option `SQLMOD_NS_AWARE` should remain, indicating that the query has been checked against the specified database namespace. On the other hand, the `SQLMOD_NS_URI` should be omitted, since the path to the SQL schema is specific to the environment, which the source code is compiled. The configuration type can be different per *external type*, without any kind of overhead.

The following listing contains the generated Java-compatible code of the above external type. It follows the guidelines declared in the specification of the `SQLQuery` API. According to it, the constructor is generated and accepts one `int` formal parameter, according to the external reference declaration. Note that the parameter's name and its internal name is `prim`, which is the name that was used in the external reference declaration.

The abstract method `getStatement` is also implemented. The generated code uses prepare statements from the JDBC API with profound benefits, such as an extra layer of type safety at compile-time and transparent performance optimisation at runtime.

```
package examples.simplesql;

import org.jmod.dsl.sql.SQLQuery;
import examples.simplesql.SimpleConf;

import java.sql.*;
```

```

public class SelectExample extends SQLQuery<SimpleConf> {
    private int prim;

    public SelectExample(int prim) {
        super(new examples.simplesql.SimpleConf());
        this.sqlStatement = "select max(sqle_primary) " +
            "from sqlexample where sqle_primary = ?"
        this.prim = prim;
    }

    public PreparedStatement getStatement(Connection c)
        throws SQLException {
        String sql_query = "select max(sqle_primary) " +
            "from sqlexample where sqle_primary = ?";
        PreparedStatement pstmt = c.prepareStatement(sql_query);
        pstmt.setInt(1, prim);

        return pstmt;
    }
}

```

4.5.6 Domain-Specific Optimisation

The SQL module uses the existing JDBC specification and does not require from programmers to learn new APIs. The generated code utilises *prepared statements*, therefore securing the application against SQL injection attacks.

4.6 Cassandra CQL Module

Apache Cassandra is an open-source database that provides scalability and high availability without compromising performance.⁵ One significant advantage that Cassandra offers is the linear scalability (Figure 4.16). Note that the number of operations is fictional and is related with the Central Processing Unit (CPU) and the memory of the actual instance. The setup of a Cassandra cluster does not contain any master node, thus it offers no single point of failure. To query and retrieve the stored data, the database offers a query language named Cassandra Query Language (CQL), which supports similar syntax with SQL. J% offers a module tha provides basic support for CQL.

4.6.1 Design and Features

To access a Cassandra cluster you need the IP address of a participating node. To connect the Java driver is used, which is provided by DataStax⁶. Consider the following program that connects, then executes a simple insert query:

```

String name = "jmod-user";
String pass = "p@ssw0rd";

```

⁵<http://cassandra.apache.org/>

⁶<http://www.datastax.com/drivers/java/2.1/index.html>

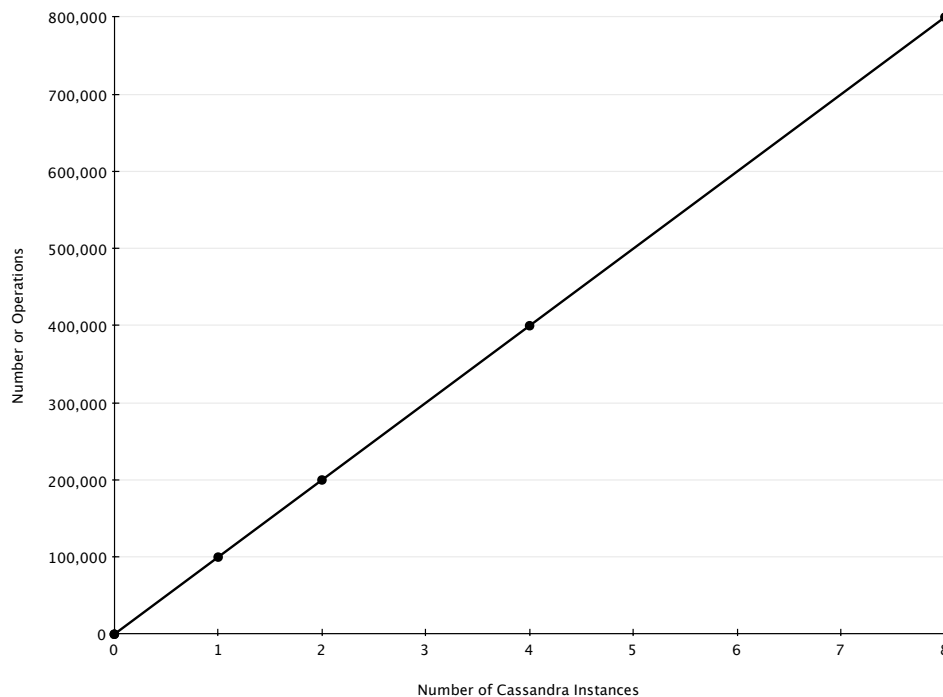


Figure 4.16: Cassandra supports linear scaling.

```
String fullName = "Jay Mod";

Cluster.Builder clusterBuilder = Cluster.builder()
    .addContactPoints("127.0.0.1").withPort(9042)
    .withCredentials("user", "pass");
Session session = clusterBuilder.build().connect();
session.execute("INSERT INTO users (KEY, password, name)" +
    "VALUES ('" + name + "', '" + pass + "', " +
    " '" + fullName + "');"");
```

The J% module aims to support this process by validating the syntax of the query and by introducing static typing. On the aforementioned example, the Java variables are simply concatenated in the query string, thus making it vulnerable to errors by malformed values and code injection attacks.

4.6.2 Module Architecture

Figure 4.17 illustrates the module's architecture. It follows the same architectural pattern with the SQL module. The base class is the `CQLQuery`, which is the only *external type* declared by the module. It is configured by the `CQLConfiguration` type, which is the *external configuration* type. The `CQLQuery` is dependent to the application library of CQL, which is provided by Datastax⁷. The structure of the **CQLQuery!** *external type* follows:

```
public abstract class CQLQuery<T extends CQLConfiguration>
```

⁷<http://www.datastax.com/drivers/java/2.1/index.html>

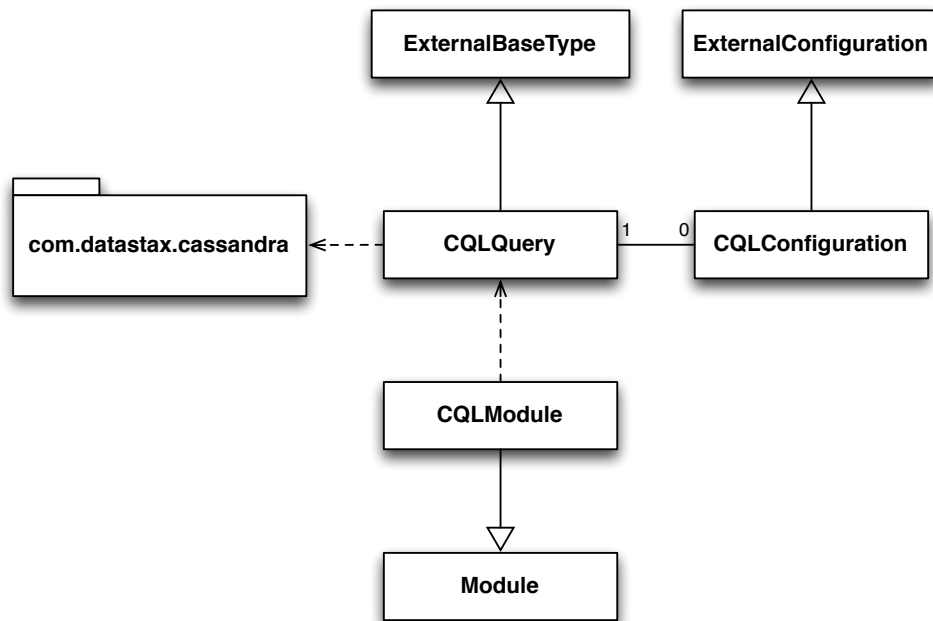


Figure 4.17: UML class diagram for the CQL module

```

        extends ExternalBaseType<T> {
    public abstract String getCQL();
}

```

The API defines only the method `getCQL`, which returns the generated CQL query.

4.6.3 Configuration Parameters

The module is not accepting any compile-time and runtime options, since it is designed to provide basic support for the CQL and does not enhance the standard application library. The *external configuration* type that is defined is illustrated in Figure 4.17. The actual implementation follows:

```

package org.jmod.dsl.cql;

import org.jmod.dsl.module.configuration.ExternalConfiguration;

public class CQLConfiguration extends ExternalConfiguration {}

```

4.6.4 CQL Types and Type Conversions

The module adopts the type mapping supported by the Java driver⁸. Table 4.17 list the mapping rules between the Java and the CQL types.

CQL language also supports composite types, like *set*, *list*, *map*, and *tuple*. These types can be parameterised by using any of the types already listed in Table 4.17. For example, a set of integers in CQL is declared as *list<int>* and is mapped to *java.util.List<Integer>*. The basic mapping rules are listed in Table 4.18.

⁸http://www.datastax.com/documentation/developer/java-driver/2.1/java-driver/reference/javaClass2Cql3Datatypes_r.html

Table 4.17: Type mapping. Dominant types are marked in *italic* (CQL module)

Java	CQL	Description
long	bigint	64-bit signed long
java.nio.ByteBuffer	blob	Arbitrary bytes
boolean	boolean	true or false
long	counter	Counter column (64-bit long)
java.math.BigDecimal	decimal	Variable-precision decimal
double	double	64-bit IEEE-754 floating point
float	float	32-bit IEEE-754 floating point
java.net.InetAddress	inet	Internet Address
int, <i>Integer</i>	int	32-bit signed int
java.util.Date	timestamp	A timestamp
java.util.UUID	<i>uuid</i> , timeuuid	Type 1 or type 4 UUID
java.lang.String	<i>varchar</i> , text, ascii	UTF8 encoded string
java.math.BigInteger	varint	Arbitrary-precision integer

Table 4.18: Type mapping for composite types (CQL module)

Java	CQL
java.util.List<T>	list
java.util.Map<K,V>	map
java.util.Set<T>	set
com.datastax.driver.core.TupleType	tuple

4.6.5 Code Generation

The code generation follows the same strategy with the SQL module. Consider the code we already presented in Section 4.6.1. The module provides abstractions for the concept of the CQL query, thus to refactor the code to use J%, we have to declare the following external type.

```
public external InsertUser extends CQLQuery<CQLConfiguration> {
    INSERT INTO users (KEY, password, name)
    VALUES ([name]<java.lang.String>,
            #[password]<java.lang.String>,
            #[fullName]<java.lang.String>);
}
```

This will finally generate the following class:

```
public class InsertUser extends CQLQuery<CQLConfiguration> {
    private String _cqlQuery = "INSERT INTO users (KEY, password, name) " +
                                "VALUES(%s, %s, %s)";

    public InsertUser(String name,
                     String password,
                     String fullName) {
        _cqlQuery = String.format(_cqlQuery, name, password, fullName);
    }

    public String getCQL() { return _cqlQuery; }
```



```
}
```

Finally, the original program should be modified as follows:

```
String name = "jmod-user";
String pass = "p@ssw0rd";
String fullName = "Jay Mod";

Cluster.Builder clusterBuilder = Cluster.builder()
    .addContactPoints("127.0.0.1").withPort(9042)
    .withCredentials("user", "pass");
Session session = clusterBuilder.build().connect();
InsertUser insertUser = new InsertUser(name, pass, fullName);
session.execute(insertUser.getCQL());
```

Chapter 5

Evaluation

Nobody ever did, or ever will, escape the consequences of his choices

Alfred A. Montapert

5.1 Evaluating Against the Design Requirements

Evaluating a system like J% is a difficult and complex task. Chapter 3 emphasises on the theoretical concepts and the design of the system by providing a set of requirements (Section 3.1.1). In this section the system's evaluation against those requirements is provided. Figure 5.1 illustrates the hierarchy separately for the *functional* and *non-functional* requirements [234].

5.1.1 Functional Requirements

The evaluation against those requirements is a relatively easy task, because each requirement either supported by the system or not, is presented as a standalone feature.

DSL Syntax Validation The DSL is checked syntactically at compile-time. How this is supported by J% is analysed in Chapter 3 and 4. The language is capable of performing DSL syntax check, for every DSL, which is included, without any limitations. *DSL Language blocks* are used for isolating the DSL code and the module API for retrieving and analysing it at

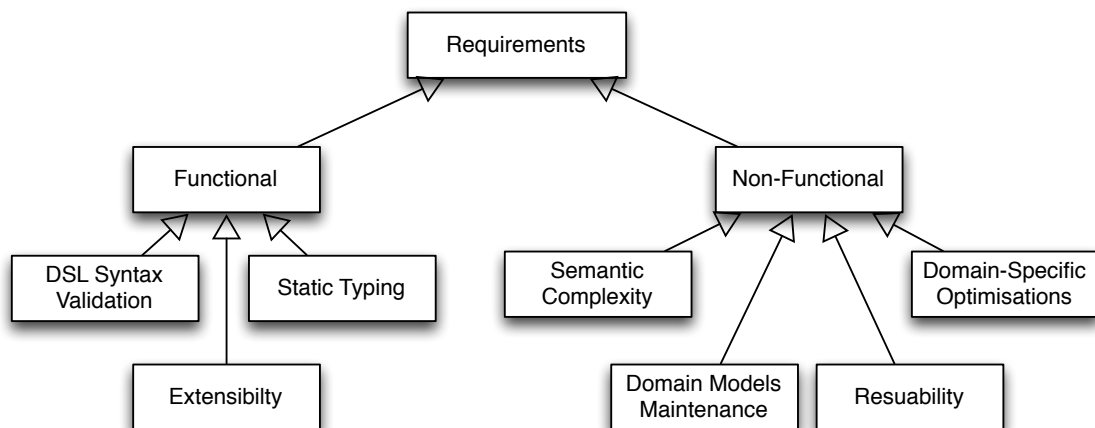


Figure 5.1: Function and non-functional requirements

compile-time by an external module. Validation errors are reported at compile-time as standard compiler messages.

Static Typing The DSL integration respects each language's type system and ensures that required values are properly exchanged between the GPL and the DSLs at compile-time. J% addresses that requirement with the usage of external references and the type mapping facility (Section 3.3). An external reference indicates that a value from the Java domain should pass to the DSL domain, with a specific type convention. This mechanism in coordination with the type mapping provides compile-time information to an external module, in order to correctly convert the values between the two languages. Each external module should define compatible types for its supported DSL by using the type mapping system (Section 3.3.4). All external references are being checked, and incompatible type usage is reported at compile-time.

Extensibility One common problem of all the ongoing research and the industry systems is the lack of extensibility. J% provides a standard way to be extended and for adding support to an infinite number of DSLs, without having to introduce new operators or write complex grammatical rules. To implement an external module a few lines of code are usually enough. The exact method of work for J% extensions is described in Section 4.3. Section 4.4 exhibits the simplicity of the framework, through the integration of regular expressions by providing standard support for both NFA-based (standard JDK library) and DFA-based library (FIRE/J [146]) with only 800 lines of code.

5.1.2 Non-functional Requirements

Non-functional requirements refer to more abstract properties of the design and it is difficult to prove that are actually supported.

Semantic Complexity The proposed solution should not increase the semantic complexity of the Java programming language. Of course, it is not possible to add a functionality without some form of extension. To enable DSL support, J% introduces a syntax, which is similar to Java syntax. The DSL code is encapsulated inside an external type, which follows similar notation with a standard Java class or interface. External type configuration is implemented through type parametrisation. Consequently, the semantic complexity of J% is not high, especially for a programmer that is already familiar with Java. Each DSL is also extended by the addition of the external references. The declaration syntax for these references follows a unique syntax, mostly to avoid syntax clashing with possible DSLs that could be supported in the future. This feature is alienating the J% developers since it has no connection with Java or its related technologies. External references also require the constructor signature for the external type to be in a specified form, a feature that is difficult for a programmer to remember.

Domain Models Maintenance It is crucial that each DSL maintains its syntax. One of the major advantages using DSLs in software development is that the language abstractions matches those of the domain model, permitting the efficient development of applications. For example, SQL is designed for expressing database queries. If anyone tries to model those queries by the definition of an API, the results will be surprisingly complex. Typical examples of such approaches are the ORM, such as Hibernate [28], which in order to write less code, it ends up recreating a new DSL of its own, an SQL-like language that performs queries and filtering on hibernate encapsulated Java classes, which model the relational data in the application. J% does not alter the DSL syntax, except from the external references.

Reusability J% encourages re-usability. This is expressed via two facets; first by enforcing the re-usability of DSL code in programs and second by enhancing the usage of existing DSL application libraries when developing an external module. In J% each external type is declared like any other reference type. This encourages re-usability, since those types can be reused by the application. Application libraries that support various DSLs exist on the Internet and usually under an open source license. Many language systems that support multiple DSLs, demand that each DSL should be rewritten from scratch. J% was designed with that problem in mind and its goal was to provide a pattern to harness all those libraries. During J%'s implementation, several DSLs were integrated, like regular expressions, SQL and velocity. All those module are using external application libraries to support their functionality (Section 4.4 and Section 4.5).

Domain-Specific Optimisations Since the DSL code is analysed, the next step is to induce domain-level optimizations [145]. Compile-time optimisations is not a new concept [277]. J% offers an indirect mechanism to introduce such optimisations. Thus J% does not enforce domain-specific optimisations on all DSLs, but provides fertile ground to develop them when implementing an external module. The two basic DSL module implementations that are presented in this thesis, implement domain-specific optimisations and serve as case studies, which are demonstrating this functionality.

Regular Expressions The regular expression module, which is analysed in detail in Section 4.4, performs compile-time code generation for each regular expression using the FIRE/J library [146], thus enhancing its execution speed. The optimisation is optional and is enabled by a compile-time configuration option.

SQL The SQL module also offers a significant optimisation in terms of performance and security, since it uses prepared statements only that offer significant improvements in terms of execution time. In addition, all queries are safe from SQL-injection attacks.

5.2 Empirical Evaluation

The empirical evaluation divided in four small experiments; first, it was analysed what it means for a developer to port a JDBC-based Java code to J% code. The second experiment involved how the DSL errors are identified by the system and presented to the programmer. The third included the porting of five Java projects that are using SQL, to examine what impact the J% code has on a project in terms of common size metrics, such as LOC. The final experiment measured the overall compilation overhead against the standard Java compiler. The results are indicative due to the small scale of the experiments. To evaluate the impact of a language extension usually requires thousands of lines from many developers to be written and analysed. Still, some early indications on the effectiveness of our approach can be derived.

5.2.1 Porting Applications to J%

Porting Java applications to use J%'s SQL module is straightforward. When a Java source file contains an SQL query, the code is extracted, and a new external type is created. After that, we replace the standard JDBC calls with the API offered by the SQL module. Consider the file `add_user.java` of the *Examj*¹ project, which contains the following `insert` statement:

¹https://github.com/bkarak/jmod-ports/blob/master/examj/java/src/add_user.java

```
int add_usr = myStatement.executeUpdate(
"INSERT INTO examj.users(name,surname,email,username,password)
VALUES('" + name + "', '"
      + surname + "', '"
      + email + "', '"
      + username
      + "', md5('"+ String.valueOf(usrpasword) + "'))");
```

In J% it should be rewritten as follows:

```
InsertUserQuery iuq = new InsertUserQuery(name, surname, email,
      username, md5(String.valueOf(usrpasword)));
myStatement = iuq.getStatement(connection);
int add_usr = myStatement.executeUpdate();
```

External type InsertUserQuery should also be defined:

```
package examj;

import org.jmod.dsl.sql.SQuery;
import org.jmod.dsl.sql.SQLConfiguration;

public external InsertUserQuery extends
      SQLQuery<SQLConfiguration> {
INSERT INTO examj.users(name,surname,email,username,password)
VALUES([name]<String>,
      #[surname]<String>,
      #[email]<String>,
      #[username]<String>,
      md5([password]<String>))
}
```

The above code retains the same functionality and without alterations in its logic, nor its basic structure.

5.2.2 Compile-time Error Detection

Consider the RegisterItem class in the RUBiS project.² It contains an SQL statement, which is created by string concatenations.

```
conn.prepareStatement(
"INSERT INTO items VALUES (NULL, \"
      + name + "\", \"
      + description + "\", \"
      + initialPrice + "\", \"
      + quantity + "\", \"
      + reservePrice + "\", \"
      + buyNow + "\", 0, 0, \"
```

²<https://github.com/bkarak/jmod-ports/blob/master/RUBiS/java/Servlets/edu/rice/rubis/servlets/RegisterItem.java>

```

+ startDate + "\", \""
+ endDate + "\", \""
+ userId + "\", "
+ categoryId+ ")");

```

We created the `InsertItemQuery` type, similar to our porting strategy in the previous section:

```

public external InsertItemQuery extends SQLQuery<SQLConfiguration> {
INSERT INTO items
VALUES (NULL,
      #[name]<java.lang.String>,
      #[description]<java.lang.String>,
      #[initial_price]<float>,
      #[quantity]<int>,
      #[reserve_price]<float>,
      #[buy_now]<float>,
      #[start_date]<java.util.Date>,
      #[end_date]<java.util.Date>,
      #[user_id]<int>,
      #[category_id]<int>)
}

```

To compile it, we executed the following command:

```

$ jmodc --compile-with-javac -i RUBiS/jmod/
[...]
[INFO] .../rubis/servlets/RegisterItem.java parsed (Java)
[...]
[INFO] .../rubis/servlets/InsertItemQuery.jmod parsed (External)
[...]
[INFO] SQLModule module called for .../servlets/InsertItemQuery.jmod

```

The above is an excerpt from the compiler output. In the first phase, the compiler identifies the *external type* and then it calls the SQL module to generate the code.

If we accidentally wrote a mistyped input like `insrt` instead of the SQL keyword `insert`, at compile-time the error would be reported:

```

$ jmodc --compile-with-javac -i RUBiS/jmod/
[...]
[INFO] SQLModule module called for .../InsertItemQuery.jmod
[ERROR] Could not parse SQL Statement - INSRT INTO items [...]
[ERROR] .../InsertItemQuery.jmod, error in SQL syntax : [...]
[...]

```

The compiler discovered the error in the SQL query, then it reported it as an error message. Similarly, errors regarding the database schema are reported, like table and column identifier errors or type incompatibilities.

5.2.3 Measuring the Impact of J% on Size and Product Metrics

The correlation of the external and internal quality of a software system is considered self-evident within the software engineering community. Empirical studies have shown that there is a direct connection between the internal and external quality attributes of software (as measured by various metrics), which can be traced back to the process under which it was developed and forward to the satisfaction its users enjoy during use. This prevalent model of software quality is presented in the *ISO-9126:1/1998* standard [94] and its more recent successor, SQuaRE [136].

A number of surveys in the literature attempt to categorize metrics and evaluate their usefulness [77, 180, 209, 283]. In order to achieve this, these studies examine whether the proposed metrics are interdependent, in an attempt to uncover relations between them and identify the best quality indicators among them. Most studies in this field are targeting two main facets of the relationship between metrics and quality, the first of which is the number of defects that are contained in it and its overall fault proneness, and the second is the effort required for its maintenance.

Regarding the defect density, some studies such as [26, 142, 174, 192] try to correlate it directly to metrics, while others attempt to validate the significance of the various metrics proposed in the literature for predicting the fault proneness of software components [42, 138]. Empirical methods have only recently been used to predict the fault proneness of classes based on complexity metrics [154, 201], although there are some older empirical studies as well [41].

Regarding software maintainability, recent studies attempt to use metrics in order to predict the maintenance effort that will be required for a system in order to keep it up to date with changing requirements [20, 168]. As software maintenance accounts for a large part of its total cost, a number of studies attempts to detect factors that affect maintainability [188] and correlate metrics to it [20, 58, 67, 104]. Other research studies try to link the design of a software system to maintainability [9, 122], while recently there has been some empirical research on this aspect [71, 143].

There is also a number of studies focusing on different aspects of software quality, such as re-usability [117] and testability [44]. Internal cohesion has also been examined as a factor of quality [23]. Software complexity and the metrics used to evaluate it have also been the target of study [51, 119, 278].

Probably the most influential factor for software quality is its design. A good design will allow a software system to evolve with little effort and make alterations easier and cheaper along the way. Studies like [36, 78, 151] attempt to correlate attributes of the design of a system to its quality, measured by defect density and maintenance effort, and provide predictive models based on the values of the metrics for these attributes. Others aim to validate existing studies by replicating the findings of others [43].

Metrics are also used to evaluate the stability of the design of a system, thus assessing its quality based on the modifications that are performed on its internal structure [255].

5.2.3.1 Selected Metrics and Evaluation Scheme

Software metrics provide a means to extract useful and measurable information about the structure of a software system. Today the software engineer or developer has a very large list of metrics at his disposition in order to gain the required insight for understanding and evaluating the size, complexity and overall structure of a system [112, 142, 159]. This has led researchers to try to discover which set of metrics is most appropriate for each environment [77, 180, 209, 283]. There also exist metrics that can be used to evaluate other aspects of a system, such as the process that was used to develop it. Software metrics can be classified into three categories (Figure 5.2) [90, 142]:

Product Metrics that describe internal and external characteristics of software, such as its size,

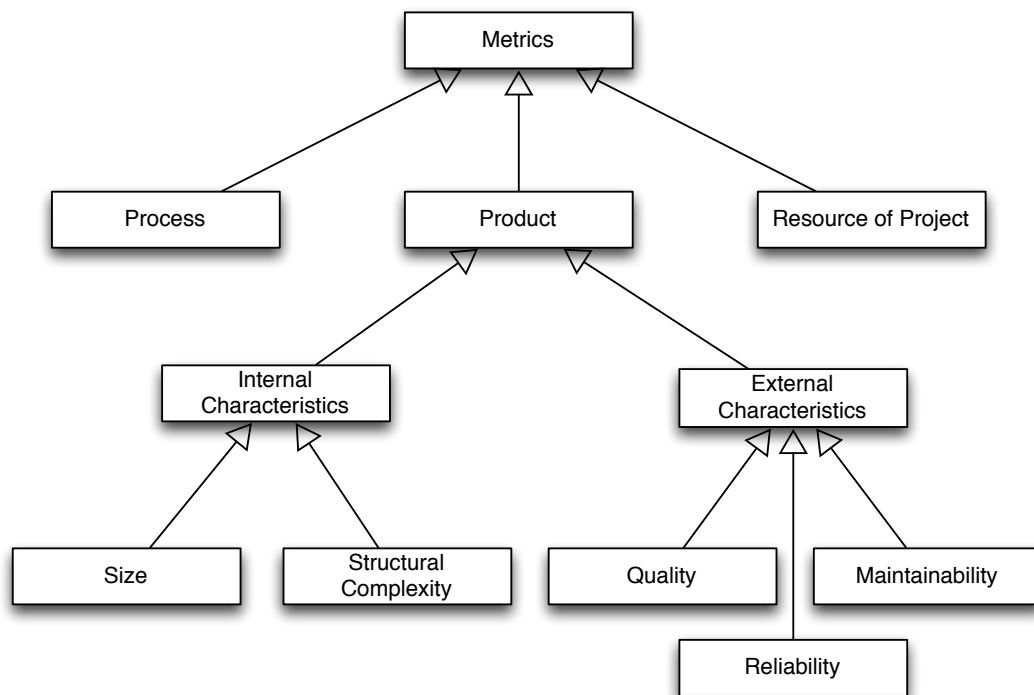


Figure 5.2: Metrics Categorisation

complexity, performance and usability,

Process Metrics that are relevant to the software development and maintenance process. Such metrics are for example the time required to repair a bug, or the effectiveness of bug removal process, and

Project Metrics which are related to other characteristics of a software project, such as the number and experience of the development team members, the conformance to quality assurance processes or the total cost. These are often referred to as Resources.

The metrics that belong to all these three categories can also be distinguished into *internal* and *external* metrics [90]. *Internal metrics* are those that can be measured purely by examining the product, process or the resource on its own, whereas *External metrics* are those that can be measured only with respect to how the product, process or resource relates to its environment.

The evaluation scheme uses product metrics metrics to assess the level of changes that J% introduces into plain Java code. A typical selection of size of metrics should include those that measure the size of the code. The following metrics are proposed and redefined to include support for *external* code.

Lines of Code (LOC) simply counts the lines of source code of a software project. Typically, LOC is calculated by simply counting the lines in the code unit (separated by the line-feed character). This metric includes executable code and comments.

```

// ... single line comment ... - Valid
System.out.println(1);         - Valid
Object.toString(); // comment - Valid
<empty line>                   - Invalid
  
```


Table 5.1: The size metrics that were used in the evaluation process.

Metric	Description
Files	Number of files
Lines of Code (LOC)	Total lines of source code
Ext. LOC	Total lines of the external type declarations
SQL Queries	Total number of queries
Classes	Total number of classes
Interfaces	Total number of interfaces
Packages	Total number of packages
External Types	Total number of external type declarations

Lines of External Code Total lines of *external* code. This include *external* code and comments, since each DSL has its own notation for comments. This metric applies only to J% projects. Java project consider all DSL code as `String` objects, consequently this metric is set to value 0.

Number of Classes Total number of classes (`class`) per project. Anonymous classes are not counted.

Number of Enumerations Total number of enumerations (`enum`) per project.

Number of Interfaces Total number of interfaces (`interface`) per project.

Number of Packages Total number of packages (`package`) per project.

5.2.3.2 Results

The evaluation process aims to analyse the impact of J% in real-world projects. J% demands that each SQL code block should be declared as a new external type. This can be easily related with significant growth in terms of user-defined types and code. For that reason, the third evaluation experiment will focus on Size & Complexity (Table 5.1). To isolate efficiently the level of change that our approach imposes, we evaluate each project by comparing its original form with a J% version.

The selected projects include; *JCrontab* an open source crontab replacement, *AddressBook*, the sample program that is distributed with the Java DB, *ExamJ* an open source Java editor, which aims to provide a framework to tutor and grade programming language students and the popular benchmarking web platform RUBiS [49]. All these applications use SQL and a relational database as a backend to store and access their data. All ported applications are published in Github.³

Table 5.2 summarises all size metric measurements for all projects. Table 5.3 contains the list of all the Java files per project that were modified during the porting process. Note that, in general, the files of the projects remained unaffected and only the files that contained SQL statements were altered. The results show that by using J%, we should expect slightly increased LOC and number of files. This is the main problem of J% in its current specification. For each SQL query a new external type should be declared, which practically means the creation of a new source file. Consequently, the increased LOC can be explained by the declaration of boiler-plate code that each external type requires. External LOC (Ext. LOC) counts the lines of code for the external types' source files. Since Java does not support external types, the count is always zero for the

³<https://github.com/bkarak/jmod-ports>

Table 5.2: Size metrics for JCrontab, Address Book, ExamJ, and RuBiS.

	JCrontab		AddressBook		ExamJ		RUBiS	
	Java	J%	Java	J%	Java	J%	Java	J%
Files	43	48	8	13	15	27	37	65
LOC	3,362	3,386	1,242	1,259	3,852	3,893	8,831	8,952
Ext. LOC	-	18	-	28	-	40	-	72
SQL Queries	5	5	6	5	13	12	44	28
External Types	-	5	-	5	-	12	-	28
Classes	40	40	8	8	15	15	37	37
Interfaces	3	3	0	0	0	0	0	0
Packages	6	6	2	2	1	1	3	4

Java version of the projects. Note that the number of SQL Queries is reduced in *AddressBook*, *ExamJ* and *RUBiS*. J% encourages query reusability and for the same queries, the same external type can be reused. For example, the *RUBiS* project has originally 44 SQL queries and 16 of them could be reused.

The SQL code is used almost unmodified, except the external reference declarations. These are the source of the second problematic situation. For each reference that is defined in an external type, a field parameter in the type's constructor is generated. This is good and bad simultaneously; the strict constructor enforces compile-time validation of each parameter that is passed to the SQL code and in addition is backed-up by a prepared statement when it is passed onto the query. Its disadvantage is that the generation scheme is not very practical when one has many parameters, and it is complex to remember their order, when the external type is actually used with the Java code. The example code in Section 5.2.1 points out to this problem. The resulting generated code of *InsertUserQuery* type has five constructor parameters.

5.2.4 Compilation Overhead

The next step in our evaluation included the measurement of the compiler's overhead, in terms of compilation time. The compilation process includes two phases; the code generation where the SQL checking is performed, and Java code is generated, and the compilation of the generated code to Java bytecode. Consider the following program:

```
package org.jmod;

import java.sql.*;
import java.util.Properties;

public class SimpleSQLProgram {
    public static void main(String[] args) {
        final String jdbcUrl = "jdbc:mysql://localhost/test";
        final String jdbcDriver = "com.mysql.jdbc.Driver";

        try {
            Properties connectionProps = new Properties();
            connectionProps.put("user", "root");
            connectionProps.put("password", "");
            Class.forName(jdbcDriver);
```

Table 5.3: LoC differentiation per file.

	Java	J%	Diff
<i>JCrontab</i>			
org/jcrontab/data/GenericSQLSource.java	253	236	-17
<i>AddressBook</i>			
com/sun/demo/addressbook/db/AddressDao.java	297	261	-36
<i>ExamJ</i>			
examj/add_user.java	171	159	-12
examj/code_send.java	178	160	-18
examj/load_code_db.java	407	382	-25
examj/remove_user.java	123	122	-1
<i>RUBiS</i>			
edu/rice/rubis/servlets/Auth.java	54	51	-3
edu/rice/rubis/servlets/BrowseCategories.java	152	153	1
edu/rice/rubis/servlets/BrowseRegions.java	85	86	1
edu/rice/rubis/servlets/BuyNow.java	138	138	0
edu/rice/rubis/servlets/PutBid.java	199	194	-5
edu/rice/rubis/servlets/PutComment.java	156	156	0
edu/rice/rubis/servlets/RegisterItem.java	237	225	-12
edu/rice/rubis/servlets/RegisterUser.java	227	215	-12
edu/rice/rubis/servlets/SearchItemsByCategory.java	205	201	-4
edu/rice/rubis/servlets/SearchItemsByRegion.java	203	198	-5
edu/rice/rubis/servlets/ServletPrinter.java	713	710	-3
edu/rice/rubis/servlets/StoreBid.java	269	254	-15
edu/rice/rubis/servlets/StoreBuyNow.java	204	189	-15
edu/rice/rubis/servlets/StoreComment.java	176	162	-14
edu/rice/rubis/servlets/ViewBidHistory.java	168	166	-2
edu/rice/rubis/servlets/ViewItem.java	147	146	-1
edu/rice/rubis/servlets/ViewUserInfo.java	201	199	-2

Table 5.4: Size metrics for the participants.

	java	simple	ns-aware	live-db
Files	1	2	3	3
LOC	31	42	51	54
Ext. LOC	0	9	9	9
SQL Queries	1	1	1	1
Classes	1	2	2	2
External Types	0	1	1	1

```

Connection conn = DriverManager.getConnection(
    jdbcUrl, connectionProps);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
    "select * from customer");
while(rs.next()) {
    System.out.println(rs.getString("customer_name"));
}
rs.close();
stmt.close();
conn.close();
} catch (Exception e) {
    System.err.println("Oops ... " + e.toString());
}
}
}

```

The code opens a connection to a MySQL database, performs a `select` query, then iterates over the results and prints all the customers' names. We ported this program and produced three variants of it; one that performed only a basic check on the SQL statement (*simple*), a second that performed the basic check, then also examined if the query was valid against a specific database schema (*ns-aware*), and a third that executed the query to the actual database to validate it (*live-db*). Table 5.4 presents a set of size metrics for each variation of the program.

Observe that each version of the program differs in terms of LOC. The original program had 31 lines of code, while the *simple*, *ns-aware*, and *live-db* versions, were 26%, 39% and 42% bigger. This phenomenon was analysed in the previous sections, where we identified as the source of the problem, the declaration of the *external types* and their *configurations*.

The test included an iteration of 2000 compilations for each version of the program. Table 5.5 lists the hardware and software characteristics of the benchmark environment. The results were analysed, and a list of basic statistical measures are presented in Table 5.6. As expected, the Java version compiles faster with an average value of 705.74 msecs, while the imposed overhead for the *simple*, *ns-aware*, and *live-db* versions is 24.68% (936.94 msecs), 24.64% (936.45 msecs), and 38.80% (1153.10 msecs) respectively. The results show that is cheap to enable query check against a database schema. Of course, results may vary. In the case of a more complex schema, it is possible that the *ns-aware* is slower. The *live-db* variation was the slowest, which was also expected, since it requires to connect to a live database server and execute each query.

Table 5.7 contains the results of a much simpler experiment. A simple Java program was compiled by *jmodc* (J%’s compiler) and *javac* to measure compilation time. Since *jmodc* in pure Java code, it copies the file, then it compiles it with *javac*, the results proven that is slower than his contestant. The experiment included 100 iterations and *jmodc* was 13.3% slower than *javac*.

Table 5.5: Hardware and software configuration.

Operating System	<i>MacOS X 10.9.1</i>
Java Runtime Env.	<i>1.7.0_40 (64-bit)</i>
System CPU	<i>Intel Core 2 Duo, 3.06Ghz</i>
System Memory	<i>12GB</i>

Table 5.6: Basic descriptive statistic measures for compilation time (*ms*).

measure	java	simple	ns-aware	live-db
min	676.00	906.00	909.00	1,116.00
max	1,320.00	1,493.00	1,103.00	1,795.00
range	644.00	587.00	194.00	679.00
1 st Qrtl	789.00	935.00	932.00	1,147.00
3 rd Qrtl	696.00	914.00	923.00	1,137.00
median	683.00	921.00	925.00	1,123.00
mean	705.74	936.94	936.45	1,153.10
stddev	29.31	29.12	17.06	37.69
overhead (%)	<i>base</i>	24.68	24.64	38.80

Table 5.7: Basic descriptive statistic measures for compiler overhead (*ms*).

	jmodc	javac
min	1241	1086
max	2683	1770
range	1442	684
1 st Qrtl	1367	1185
3 rd Qrtl	1241	1240
median	1333	1180
mean	1463.2	1283.5
stddev	190.3	135.2
overhead (%)	13.3	0.0

5.3 Conclusions

The main contribution of J% is the support of embedded DSL code in Java programs in a type-safe and in a syntax-checked way. Each DSL statement is checked, and all errors are reported at compile-time. In addition, Java types are mapped into DSL types, and all interactions between the languages are also checked at compile-time. The integration of the DSLs provides developers some notable features. The first involves an extensive compile-time configuration. In essence, each DSL can be configured with different code generation and testing features. This is achieved with the utilisation of the external configuration concept introduced by our approach. Query validation is another key feature of J%. If this feature is enabled, queries can be checked against a specified database schema, which is provided as part of the query configuration.

The SQL module utilises the existing JDBC specification, without alienating the developer with new APIs or frameworks. In addition, our approach supports the transparent usage of prepared statements. Furthermore, J% employs the performance optimisations offered by the prepare statements compilation in a transparent manner. Finally, by using prepared statements, J% provides shielding against input validation attacks. This feature could be very useful, especially in the case of web applications.

As already discussed in Chapter 2, none of the related approaches support the aforementioned characteristics as a whole. Moreover, as opposed to our module, these methods usually define complex semantics to support their functionality, even by defining new DSLs, like those of Hibernate and the Scala (Section 2.4). One of the disadvantages of our approach is the fact that for every single query the developer must define a new external type, which may lead to significant namespace pollution for large projects.

Features like type mapping and compile-time configuration, greatly support the development process and permit the implementation of advanced features, such as live database testing in the SQL module. To extensively assess J%'s impact to the software development process, many more software projects must be developed and more experiments must be conducted; thus measurements presented here were indicative.

Chapter 6

Conclusions, Open Issues and Future Research Directions

Simplicity does not precede complexity, but follows it.
Alan J. Perlis

6.1 Summary

DSLs are programming languages that focus on a specific problem domain. For that purpose, they sacrifice syntactic flexibility. They are often called *micro-languages* or *little languages*. On the other hand, GPLs have a wider scope and provide a set of processing capabilities applicable to various problem domains [29].

The GPL-DSL integration brings forth many practical and research issues. An example is the SQL integration with the Java programming language, which is implemented by the JDBC application library [91]. This approach compels the programmer to pass the SQL query as a `String` to the application library at runtime. The Java compiler is completely unaware of the SQL query, and the programmer finds out possible errors usually by a runtime exception raised by the JDBC driver. Such errors remain undetected, if during software's testing phase the SQL query never been invoked.

This thesis introduces J%, a DSL-aware extension of the Java programming language. Its purpose is to enrich the current Java syntax in order effectively to support DSLs. The prototype implementation consists of a compiler that translates the J% source code to Java compatible code. Thus, it provides an extensible way to embed DSLs into Java.

In Chapter 2 several methods that are used for GPL-DSL integration were discussed. These methods included,

- the usage of an application library,
- integration through language extension, by introducing operators in the host language, thus extending it,
- extending the compiler by introducing facilities that allow the developers to introduce their new syntactic idioms, in order to support other languages.

These are the most common ways to integrate a DSL within a GPL. The first two methods are used mainly by the DSL-oriented research community. Their main focus is to design and develop DSLs, efficiently integrate them with other languages, limit their approaches in terms of efficiency. An example of this approach is the case of regular expressions with Perl. Perl supports efficiently

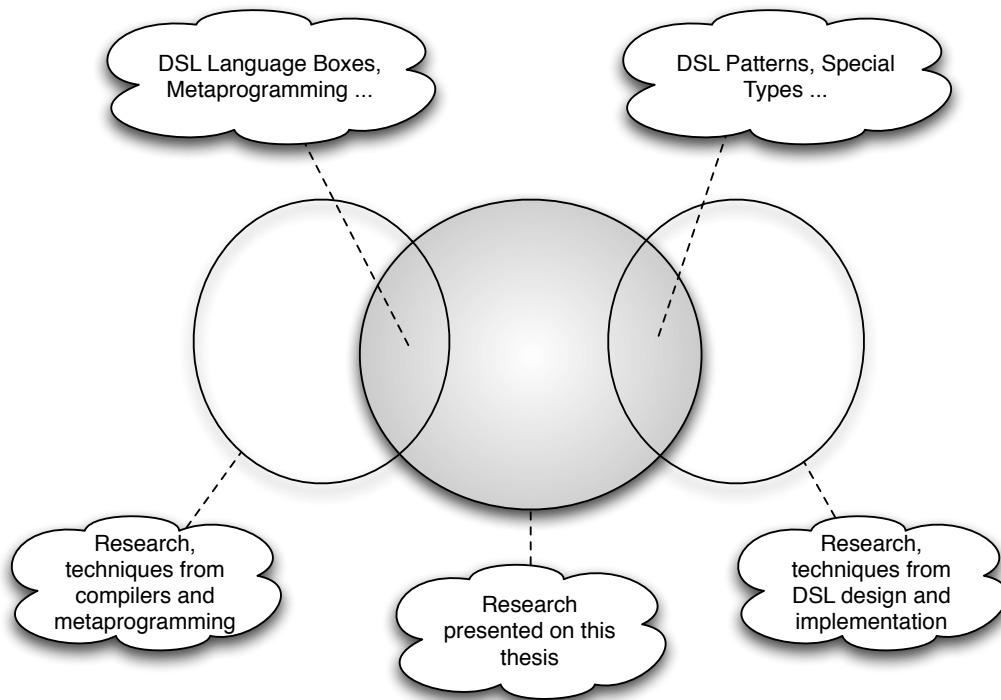


Figure 6.1: Research on DSL-GPL integration

regular expressions and for that it offers a rich set of operators at language level. This extension cannot be easily adapted for use by other GPLs, without complicating their syntax.

The third method is used by the language designers and metaprogramming experts. The main concept behind their work is the ad-hoc extension of the programming language at compile-time, usually by offering complex and exotic mechanisms at syntax level. A variant of this approach is using metaprogramming that produces source-to-source transformation and rule rewriting techniques that are utilising existing APIs.

The main problem of these aforementioned research aspects is that they do not address the DSL-GPL integration problem. In fact, this is treated as a side-product of their main research effort. The DSL-oriented researchers try to implement and embed their DSL into a specific GPL. The compiler experts try to find a theoretically correct way to make the syntax of a language extensible without taking under consideration practical aspects of the software development process, like the fact that it is not possible to rewrite all DSLs from scratch ignoring current implementations. The aforementioned situation was also pointed out by Mernik et al. in reference [187].

This thesis deals with this problem, introduces the fusion of those approaches, thus providing a method to address this problem (Figure 6.1).

6.2 Contribution

In [32], Bjarne Stroustrup is proposing a series of questions that asks himself every time he is tempted to create a new major language. Those questions are derived from his experience as the lead C++ language designer and are not widely accepted by the scientific community, but they are logical and valid. In this section we will try to list and answer these questions in order to better understand J%'s contribution.

▷ *What problem would the new language solve?*

The main contribution of J% is the development of a framework by which arbitrary DSLs can be embedded in Java programs in a type-safe and syntax-checked way. Other efforts either do not support arbitrary DSLs or do not provide type safety and syntax checking. This problem is not efficiently addressed by current research and implementations (Chapter 2).

▷ *Who would it solve problems for?*

J% simplifies the usage of DSLs with Java for the developer without ignoring the theoretical concepts of programming languages, which are used to fortify this design (Chapter 3).

▷ *What dramatically new could be provided (compared to every existing language)?*

J% is extensible without defining complex APIs or complex syntactic idioms (Section 4.3).

▷ *Could the new language be effectively deployed (in a world with many well-supported languages)?*

J% is a Java extension and can easily be integrated with all the Java-related technologies.

This research work contains a series of scientific contributions in various aspects of DSLs and GPLs. The design and the development of the J% system provided the following generic contributions:

Static Typing The compilation process of the DSL is type-safe. Values that are exchanged between the host language and each DSL are safe with the usage of a generalised type mapping mechanism (Section 3.3.4).

DSL Syntax Validation The DSL code is syntactically checked at compile time and all errors are reported by the compiler.

Language DSL Blocks J% provides one common way to embed DSLs into Java. The host language grammar is not burdened with custom extensions, each time a new DSL module is included.

Domain-Specific Knowledge Each DSL retains its syntax. There is only a minimal addition of external references to each DSL's grammar of each language to support type mapping with J%.

Extensible Developers can create their own external module and support various DSLs by implementing a well defined API. J% can include as many DSLs as needed. All related metaprogramming facilities are treated as DSL. Since the modules are implemented in plain Java, it is easy to create an external module that supports the desired templating language and then use it for metaprogramming. The Velocity [95] templating engine is used by all implemented modules as an internal templating facility.

The contribution of J% SQL module is the development of a framework, which can embed SQL code in Java programs in a type-safe and syntax checked way. Each SQL statement is syntactically checked at compile-time. In addition, Java types are mapped into SQL types and all interactions between the two languages are also checked at compile-time. The integration of the SQL query language to the J% compiler system offered the following unique characteristics:

Extensive Query Configuration Each query can be configured with different code generation and testing features. This is implemented with the utilisation of the external configuration concept, which is introduced in J%.

Query Validation The query can be checked, if enabled, against a specified database schema. The schema is provided externally.

JDBC Compatibility The SQL module utilises the existing JDBC framework, without alienating the developer with new API or frameworks. Mapping between the types also uses the proposed JDBC convention.

Domain Specific Optimisation The code generation template used in the SQL module utilises prepared statements, thus all queries in addition to the above features are parametrised correctly with proper values and harnessing performance optimisations offered by the prepare statements compilation in a transparent manner.

J% also supports regular expressions by utilising two existing APIs, the standard regular expression library⁽¹⁾ included in the Java Runtime Environment (JRE) and the FIRE/J library. The selection of each engine is realised as a simple field editing in the external configuration type. Of course, this can be selected for each regular expression pattern separately.

In addition, FIRE/J is not a typical regular expression engine. When used, it offers great performance benefits in terms of execution. The speed-up in performance is achieved through a Just-In Time (JIT) compilation mechanism that translates each pattern into tailor-made Java code. This work is thoroughly analysed in Appendix B.

Along with all the technical contributions, in Section 3.1.2, the key factors for the successful design of a GPL-DSL integration were introduced. These factors are a collection of existing characteristics appeared in various approaches that were presented in Chapter 2.

6.3 Future Work

Because of J%'s nature, other optimisations could be introduced, in terms of DSL code reusability and overall language design. Within the boundaries of our research we intend to examine in the future the following concepts:

Host Language Support Current J% design and implementation focus only at the extension of the Java programming language. It would be interesting to explore, how the same set of techniques and methods could be applied on other languages, like C++ or Python.

Dynamic DSL Generation J% deals only with static DSL statements. The next step could include the utilisation of existing research [111, 152] and ultimately the provision of a mechanism that could check dynamically generated statements.

Unified Debugging Process Typical debugging solutions do not work with the current J% implementation. The J% debugger should be implemented and support all the modules for the integrated DSLs. Debugging facilities, such as breakpoints, must work in a unique way to match the current mainstream implementations for the GPLs.

DSL-aware Refactorings So far refactoring [96, 246] affects the Java language. It would be challenging to experiment on performing DSL refactorings and on changing blocks of code that could modify and recokn with both languages.

¹java.util.regex

Some language-specific issues were also identified, problems that could possibly extend more the J% semantics and provide better support for DSL integration. These are:

Support More Database Backends Support for SQL module focuses on the mysql database dialect. We plan to provide support for other major database systems. The support will include better analysis of custom sql statements and support for type mapping for unique types that each database backend offers. The work to support each rdbms, will further expand the module's capabilities and provides the starting point to address next-level set of problems, like enhancing database security model at compile-time etc.

Parameter Object Class Constructor With the current code generation approach, complex sql statements with many *external reference* definitions, can lead to complex constructors, with many parameters. The code generation mechanism can be modified to generate parameter objects to group all the constructor parameters in one coherent class definition.

Anonymous External Types External type definition is implemented in J% with the declaration of a new external type. This could lead into problematic situations where numerous types must be defined, if the software project uses excessively DSL code statements. This situation is very common in database applications, where the usage of DSL queries is very often. *Anonymous types* is a standard feature of the Java programming language and by introducing it to J% will mitigate the effect of one-type-per-DSL. The following code exhibits a possible version of J% code, which supports this feature. The syntax is similar to the Java version of anonymous types [110] declaration, which is logical, since J% is an extension of the Java programming language and it is required to keep the same syntax where possible.

```
public void anonymousExternal() {
    // Anonymous external type
    Regex<Config> r = new Regex<Config> {
        [0-9]*
    };
}
```

Mixing of Java with DSL Code There is a limitation with the current J% specification, which so far introduces a sophisticated yet monolithic approach to include DSLs in Java code. Consider a Java program where the standard regular expression module is used (`org.jmod.dsl.regex`) in order to match integers. This situation involves a typical external reference declaration, e.g. `NumberRegex` that will contain the DSL code.

```
public external NumberRegex extends Regex<Configuration> {
    [0-9]*
}
```

Let's assume that it is required by the `NumberRegex` external type to define a method named `foo` to implement a specific functionality. To support it, the declaration of a wrapper class is required that will encapsulate the `NumberRegex` functionality and also implement the required functionality by declaring the `foo` method (Figure 6.2).

The proposed syntax alteration in order to fully implement this feature is exhibited in the following code fragment. Since additional code could be written along with the DSL code in the proposed external type, language boxes [213] should be used to isolate the DSL code from the plain Java code.

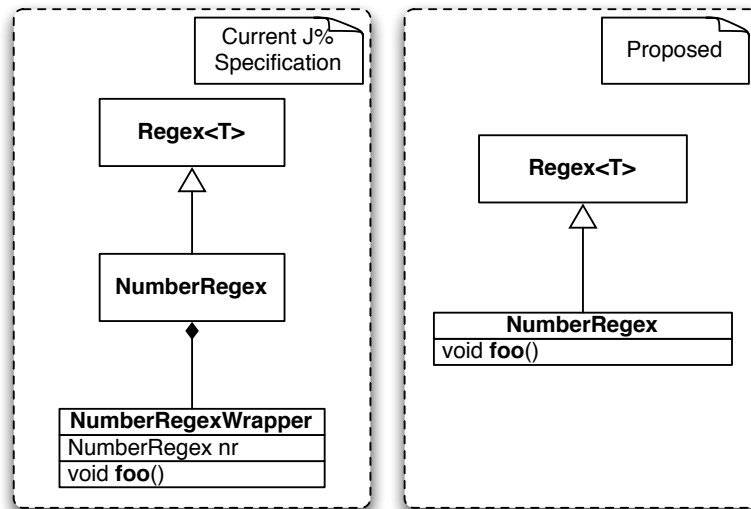


Figure 6.2: Optimising the example's code structure with the proposed feature

```

public external NumberRegex extends Regex<Configuration> {
  [[ [0-9]* ]]

  public int foo() { ... }
}

```

External Configuration via Annotations The introduction of annotations in Java since the version 1.5, provided the solid ground for the developers to introduce a series of compile-time custom checks and to generate boilerplate code. Annotations could also be used in J% to provide compile-time information for the DSL, a feature that currently is implemented with the usage of configuration types. Consider the following code.

```

@sql {
  SQLMOD_NS_AWARE = false
}
public external CustomerQuery extends SQLQuery {
  select * from customer where id = #[id]<int>
}

```

The annotation (`@sql`) instructs the compiler to skip compile-time namespace checks for the following SQL query. Traditionally this could be done by type parametrisation of the `SQLQuery` *external base type*. The addition of this feature requires redesign of the original J% configurations concept. Alternatively, the same *configuration type* could be maintained and use the annotations to generate the boilerplate code that comes with each configuration.

Appendix A

Publication List

A.1 Book Chapters

KARA15b Vassilios Karakoidas. *Domain-Specific Languages*. Encyclopedia of Computer Science and Technology (ECST), 2014, (accepted expected late 2015)

A.2 Journal Articles

KASP08 Vassilios Karakoidas and Diomidis Spinellis. FIRE/J: Optimizing regular expression searches with generative programming. *Software: Practice and Experience*, 38(6):557-573, May 2008, doi:10.1002/spe.841 (citations: 3)

SGK+09 Diomidis Spinellis, Georgios Gousios, Vassilios Karakoidas, Panagiotis Louridas, Paul J. Adams, Ioannis Samoladas and Ioannis Stamelos, Evaluating the Quality of Open Source Software, *Electronic Notes in Theoretical Computer Science (ENTCS)*, 233:5-28, March 2009, doi:10.1016/j.entcs.2009.02.058 (citations: 52)

MKL+11 Dimitrios Mitropoulos, Vassilios Karakoidas, Panagiotis Louridas and Diomidis Spinellis, Countering Code Injection Attacks: A Unified Approach, *Information Management and Computer Security*, 19(3), 2011 (citations: 5)

KARA15c Vassilios Karakoidas, Dimitris Mitropoulos, Panagiotis Louridas and Diomidis Spinellis. A Pragmatic Approach to Integrate Java with SQL using J%, *Computer Languages, Systems and Structures*, 2015, doi:10.1016/j.cl.2015.01.001 (citations: 1)

A.3 Conference Articles

GKSL07 Georgios Gousios, Vassilios Karakoidas, Konstantinos Stroggylos, Panagiotis Louridas, Vasileios Vlachos, and Diomidis Spinellis, *Software Quality Assessment of Open Source Software*, In proceedings of the 11th Panhellenic Conference on Informatics (PCI 2007), Patras, May 2007 (citations: 17)

KASP09 Vassilios Karakoidas and Diomidis Spinellis. *J%: Integrating Domain-Specific Languages with Java*. In Vassilios Chrissikopoulos, Nikolaos Alexandris, Christos Douligeris, and Spyros Sioutas, editors, Proceedings of 13th Panhellenic Conference on Informatics (PCI 2009), pages 109-113. IEEE Computer Society, September 2009, doi:10.1109/PCI.2009.26 (citations: 3)

MKSP09 Dimitris Mitropoulos, Vassilios Karakoidas, and Diomidis Spinellis. Fortifying applications against XPath injection attacks. In A. Poulymenakou, N. Pouloudi, and K. Pramataris, editors, 4th Mediterranean Conference on Information Systems, pages 1169-1179, September 2009 (citations: 7)

KMGSL15 Vassilios Karakoidas, Dimitris Mitropoulos, Georgios Gousios, Diomidis Spinellis, and Panagiotis Louridas. *Generating the Blueprints of the Java Ecosystem*. In MSR '15: Proceedings of the 2015 International Working Conference on Mining Software Repositories, (submitted)

A.4 Magazines

KARA14a Vassilios Karakoidas, *On Domain-Specific Languages Usage (why DSLs really matter)*, XRDS: Crossroads, The ACM Magazine for Students, ACM, vol. 20, no. 3, New York, NY, USA, pp. 16-17, 2014. (citations: 1)

Appendix B

FIRE/J Benchmarking

The benchmark process consists of two separate experiments. For overall performance evaluation, we benchmark expressions that are used in everyday programs. In addition, we also evaluate two specialized regular expressions against small buffers: an IPv4 address matcher (medium size and complexity) and an Apache log entry (big size and complexity).

Each test is executed separately. Thus, we guarantee that each test will run isolated and its performance will not be affected by any previous executions. In addition, we perform a warm-up period for the virtual machine in order to eliminate overhead, such as class bytecode verification and also give the JIT the opportunity to compile frequently used methods in to native code. Each Java benchmark is also executed in a separate virtual machine process.

In order to obtain accurate time values we used the method `System.currentTimeMillis()`, which according to literature is the most efficient and correct method to use in similar cases [231]. The caching engine of FIRE/J was deactivated during the experiment. The benchmark was executed on a Pentium 4 CPU at 2.8 Ghz with 512 Megabytes of memory. The underlying operating system was a Linux distribution with the 2.6.16 kernel version.

B.1 Evaluation Criteria

To theoretically support our results we adopted two complexity metrics that were introduced by Andrej Ehrenfeucht et al. [80]. Using these we can categorize and compare the regular expressions we use in throughout our benchmarking process.

Size (S) Number of alphabetical symbols in the expression

Length (L) Length of longest non-repeating path through the expression

The *Size* complexity metric represents the number of states and transitions of the generated automaton, while *Length* indicates the number of minimum steps an engine has to perform in order to achieve a match.

Compile Time (t_{comp}) overhead indicates the time required by an engine to parse a regular expression and generate the automaton. For the FIRE/J engine this includes the time of the JIT compilation.

Execution Time (t_{match}) indicates regular expression engine match efficiency. It represents the amortized execution time of a single compiled regular expression against specified text.

Break-even point indicates the number of executions that must be performed in order to cover the compile time difference between two regular expression engines. To calculate it, we used the following function:

$$f(r) = t_{match}r + t_{comp}$$

where t_{match} represents the execution time of the regular expression engine, t_{comp} the compile time and r the number of invocations. The break-even point is calculated as the number of invocations r that satisfies the following equation:

$$f_a(r) = f_b(r)$$

The results of our experiments are illustrated as graphs of the *execution time* against the *length* and *size* for each regular expression, thus providing the means to relate execution time with these, more abstract characteristics. As a result, we investigate how the size of the generated automaton, and the number of matching steps involved affects overall performance.

B.2 Benchmark Participants

Our benchmarks include the Sun regular expressions engine [3] (J2SDK) as the most fully featured engine indicated by the bibliography [100] and practitioners, and the Automaton engine (Automaton) as the only DFA engine [190]. Other Java implemented engines such as *GNU regexp* [1], *Jakarta regexp* [172] and *Jakarta ORO* [225] were tested and finally not included in the benchmark, because their performance was not exceptional.

We also found interesting to include non-Java platforms. We chose the mono implementation [59] as another virtual machine based platform, Perl [123, 272], the GNU is Not Unix (GNU) regular expression library in C [2] and Henry Spencer's library [235]. All the above regular expression libraries are NFA-based implementations and follow the POSIX standard, with the exception of Perl, which supports PCRE. The benchmarking environment included the 1.5.0 (build 9) version of the Java Runtime environment, 5.8.8 version of Perl, the 1.1.13.8 version of MONO implementation, the 1.8-8 version of the automaton library and the alpha 3.8 of the Henry Spencer's library.

We targeted our benchmark mainly on regular expression engines and not tools like *GNU grep*, *AGrep* and *NRGrep*. Although those tools provide fast implementations and interesting variations of regular expression execution engines, it is too difficult to isolate the overhead of their execution time. These libraries have their engines integrated with the whole tool, making it almost impossible to get precise results for the engine part without the accompanying overhead.

B.3 Real-World Regular Expressions

In our experiments we used 849 of real-world regular expressions that were obtained from the regexlib¹ web site. The average occurrence of each regular expression operator for our data set is listed on Table B.1. Descriptive statistical measures for our data set are presented in Table B.3.

None of the engines had 100% compatibility with our data set. The success rates of our participants are illustrated on Table B.2. Noticeably; FIRE/J has the lowest success rate than all the engines. FIRE/J uses the Automaton DFA compiler, which bounds its success rate to 685 (as shown in Table B.2). As we will see later on, this decision was based only on our need to isolate the performance gain of the code generation optimization.

¹<http://www.regexlib.com/>

Table B.1: Average Operator Occurrence

Operator	Occurrence
.	0.15
+	0.58
*	0.7
?	1.39
	3.85
()	5.08
[]	7.32

Table B.2: Participants Success Rate (Total 849)

	Succeeded	Ratio (%)
J2SDK (Java)	805	95
Automaton (Java)	685	81
FIRE/J - Java (Java)	665	78
FIRE/J - bytecode (Java)	665	78
GNU (C)	760	90
Spencer (C)	721	85
Mono (C#)	788	92
Perl	792	93

Table B.3: Statistical Measures

	1 st Quartile	Avg.	Median	3 rd Quartile	Max
Size	68	301	153	372	3394
Length	10	20	17	26	136

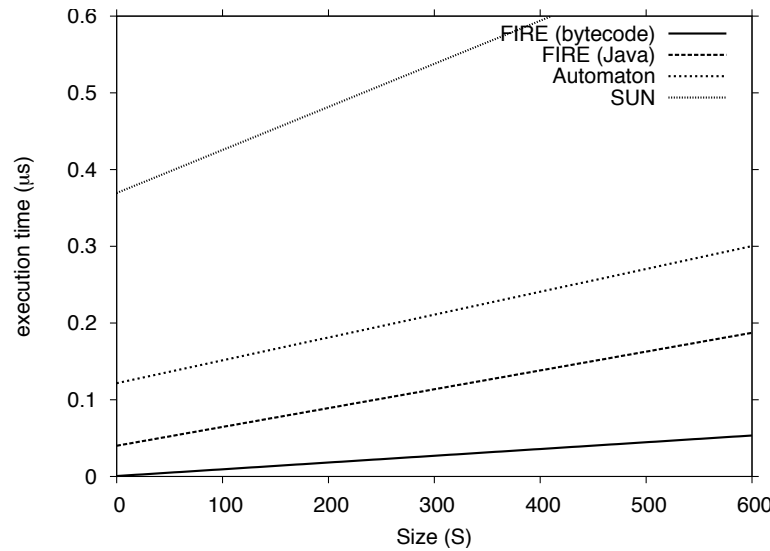


Figure B.1: Matching Performance for Java engines - Execution time / Size (S)

Table B.4: Matching Performance Statistical Measures (*ms*)

Engine	1 st Quartile	avg	median	3 rd Quartile	max
J2SDK (Java)	132	357.92	227	371	69310
Automaton (Java)	115	200.78	177	253	1191
FIRE/J - Java (Java)	40	124.28	67	134	3039
FIRE/J - bytecode (Java)	33	75.45	50	72	4351
GNU (C)	120	1135.69	980	1630	18440
Spencer (C)	230	2497.87	680	2680	108700
Mono (C#)	434	928.59	667	1072	8004
Perl	90	128.08	107	144	672

We narrowed down our results only to the common success cases and then performed linear regression to be able to compare them directly. The common successful cases were 629. We provide four performance charts (Figure B.1, B.2, B.3 and B.4) to compare the FIRE/J bytecode implementation with the other participants. The charts show the *execution time* (Y axis) related to the *size* and *length*.

As one can see from the graphs that FIRE/J is the fastest engine, having in general faster execution times than all its competitors. The Automaton engine is the second fastest participant. This was expected, because it is the only engine except FIRE/J that is based on a DFA. Surprisingly, the native libraries we tested were significantly slower. Those results can be explained having in mind the following two factors: (a) the engines are based on DFAs and (b) they were implemented many years before, and modern programming abstractions and techniques were not used in their implementations. The Spencer engine proved to have poor performance and was excluded from the graphs. Table B.4 lists some common statistical measures calculated from our results.

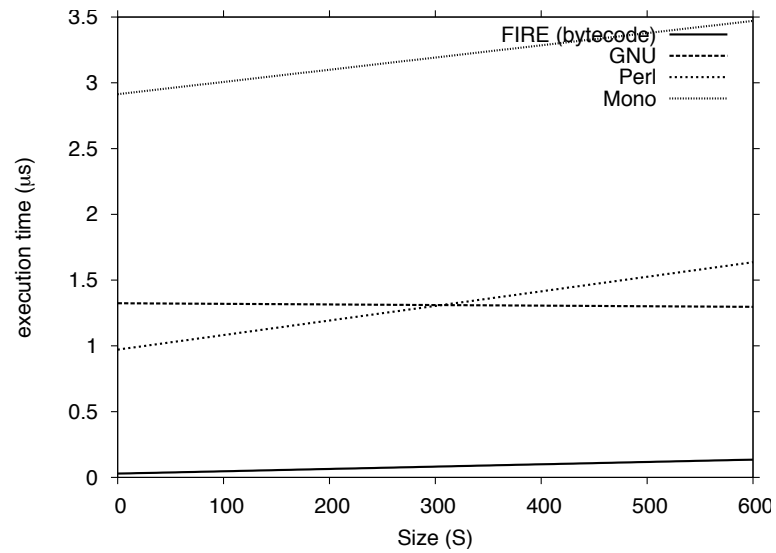


Figure B.2: Matching Performance for non-Java engines - Execution Time / Size (S)

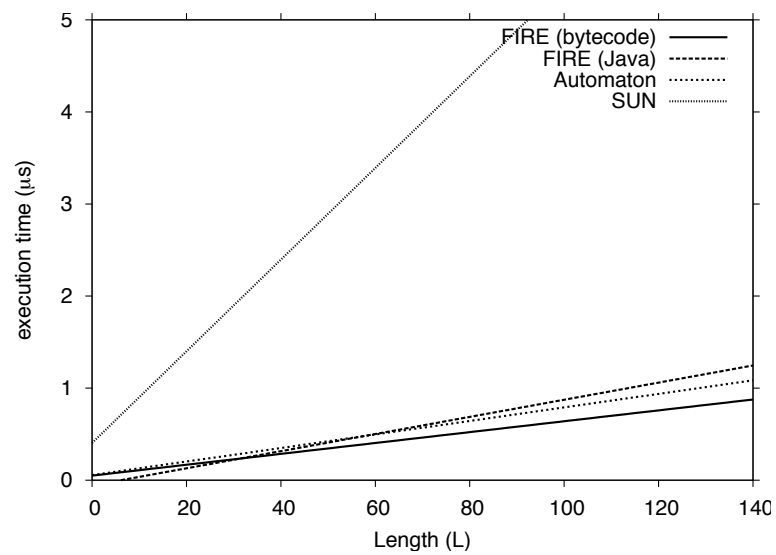


Figure B.3: Matching Performance for Java engines - Execution Time / Length (L)

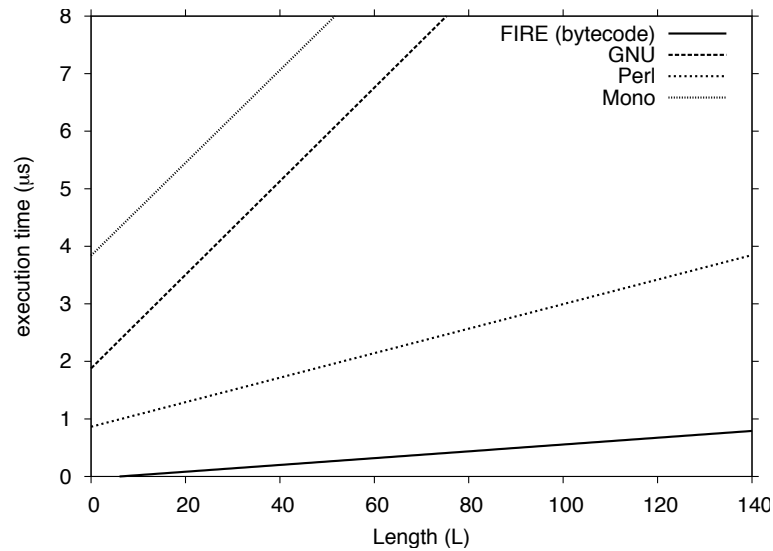


Figure B.4: Matching Performance for non-Java engines - Execution Time / Length (L)

B.4 Hand Crafted Regular Expressions

We also performed two tailored tests with hand-crafted expressions, a typical matching example and a more complex regular expression. As our first case we used a regular expression that matches an IPv4 address [100].

```
(([01]?[0-9][0-9]?|2[0-4][0-9]|25[0-5])\\.){3}([01]?[0-9][0-9]?|2[0-4][0-9]|25[0-5])
```

The second regular expression involves a verifier for an Apache web server log entry:

```
([0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}/[a-zA-Z]{3}/[0-9]{4}: [0-9]{2}: [0-9]{2}: [0-9]{2} [ ]\\+[0-9]{4}. [ ] . (GET|POST) [ ] [a-zA-Z/~0-9.%]+ [ ] HTTP/[1-9]\\.[1-9]{1,2}. [ ] [0-9]+ [ ] [0-9]+ [ ] .- [ ] . [a-zA-Z0-9./]+ [ ] \\(.+\\) [ ] [a-zA-Z/0-9]+ [ ] [a-zA-Z/.0-9]+.
```

The data we used included were a 15-character Internet Protocol Version 4 (IPv4) address (255.255.255.255) and an Apache log entry of 193 characters length. Table B.5 shows the results of the benchmark process. As we can see the FIRE/J engine is faster than all its competitors.

It is also interesting to note, that the FIRE/J-Java generator loses slightly from the automaton engine in the more complex regular expression. Our experiments showed that the FIRE/J-Java generator loses matching speed when the DFA has many states. In this case the generated class file is very big in terms of size, which causes cache misses that compromise the overall performance of the engine. The FIRE/J-bytecode generator will have eventually the same performance problem for more complex regular expressions.

B.4.1 Calculating Compilation Overhead

The compilation benchmark process was simple; we calculated the average compilation time for the Apache log expression (see Section B.4). The caching modules of FIRE/J were disabled during the experiment. The break-even point is defined as the number of matches that each engine has to execute in order to cover the compile time overhead.

As shown in Table B.6, Perl has the fastest compile time. JDK, GNU and Mono will never break even against Perl, since they had slower matching performance (see Section B.4). Automaton

Table B.5: Matching Performance (μs)

	IPv4	Apache Log
J2SDK (Java)	3.5	33.8
Automaton (Java)	0.3	4.6
FIRE/J - Java (Java)	0.2	4.9
FIRE/J - bytecode (Java)	0.1	1.2
GNU (C)	9.2	67.4
Spencer (C)	118.4	7094.8
Mono (C#)	14.5	39.0
Perl	3.8	6.0

Table B.6: Compilation Performance (μs)

Engine	Compile Time	Break-even point
J2SDK (Java)	62.0	n/a
Automaton (Java)	43,992.0	32,280
FIRE/J - Java (Java)	260,904.0	247,255
FIRE/J - bytecode (Java)	115,690.0	24,313
GNU (C)	160.4	n/a
Spencer (C)	62.0	n/a
Mono (C#)	5.3	n/a
Perl	0.9	0

will cover the compilation overhead after 32,280 matching invocations and FIRE/J-bytecode after 24,313.

Our implementation uses the Automaton engine for the translation of each regular expression to the DFA. To isolate the overhead of our optimization technique we compiled the regular expression with Automaton and FIRE/J, which includes the code generation, and subtracted the compilation times. According to our benchmarks, the overhead time of the DFA to native code was approximately 217 *ms* for Java source generation and 72 *ms* for bytecode generation; about 83% and 62% of the total compilation time.

Appendix C

Cross Language Metric Tool

C.1 Cross-Language Metric Tool

CLMT calculates software complexity metrics on a variety of programming languages. Originally, CLMT was developed as part of the European Union (EU) research project Software Quality Observatory of Open Source Software (SQO-OSS)¹ and was integrated as the main plug-in for metric calculation [243], with support for Java, C and C++.

Table C.1 enumerates the metrics that are supported by CLMT. Although most metrics work with the Java programming language and J%. Many metrics are also supported for C and C++.

The metrics are categorised into higher-level *Metric Groups*. This grouping ensures that all the correlated metrics will be calculated together, in favour to performance. Table C.2 summarises the main groups that CLMT supports.

CLMT accepts as input an XML file that describes specific metric calculation tasks. These calculation tasks are analysed along with the associated source files and metrics are calculated. The results are presented as textual output or as XML structured documents. Appendix C contains the *task* and *result* specification as Document Type Definition (DTD) grammars.

C.2 Result DTD

The following DTD describes the schema that is used by the result XML files.

```
<!--  
  Document Type Definition (DTD) for Result XML  
-->  
  
<!ELEMENT results (result+) >  
  
<!ELEMENT result (id, result-category, filename, measurement+) >  
  
<!ELEMENT measurement (name, value, result-type) >  
  
<!ELEMENT id (#PCDATA) >  
<!ELEMENT name (#PCDATA) >  
<!ELEMENT value (#PCDATA) >  
<!ELEMENT filename (#PCDATA) >
```

¹<http://www.sqo-oss.org/>

Table C.1: Built-in Supported Metrics

Description	Java	J%
High-level Structuring Proportion Metric (project)	•	
Class Structuring (project)	•	
Operation Structuring (project)	•	
Intrinsic Operation Complexity (project)	•	
Lines of Code (Java, Enumeration)	•	
Number Of Methods (Java, Enumeration)	•	
Afferent Couplings (module)	•	
Efferent Couplings (module)	•	
Instability Metric (module)	•	
Number of Native Methods (Java, Project)	•	
Number of Native Methods (Java, CodeUnit)	•	
Function-Oriented Code (Java)	•	
Number of protected methods (class)	•	•
Number of public methods (class)	•	•
Number of Private Methods (class)	•	•
Total number of methods (class)	•	•
Number of public attributes (class)	•	•
Number of private attributes (class)	•	•
Number of protected attributes (class)	•	•
Number of attributes (class)	•	•
Lines Of Code (class)	•	•
Data-Access Metric	•	•
McCabe Cyclomatic Complexity (method)	•	
McCabe Cyclomatic Complexity (project)	•	
Response for a Class Metric	•	
Number of classes (project)	•	
Number of Enumeration (project)	•	
Number of Interfaces (project)	•	
Number of External Types (project)	•	
Source Lines of Code (project)	•	
Comments Lines of Code (project)	•	
Number of files (project)	•	
Number of modules (project)	•	
Number of files that could not be processed (project)	•	
Total line count (project)	•	
Total External line count (project)	•	
Attribute Hiding Factor (project)	•	
Method Hiding Factor (project)	•	
Coupling between object classes Metric	•	
Weighted methods per class	•	
Lines of Code (codeunit)	•	
Lines of Comments (codeunit)	•	
Total Number of Lines (codeunit)	•	
Lines of DSL Code (codeunit)	•	
Lack of cohesion in methods Metric	•	
Depth of Inheritance Tree Metric	•	
Number of Concrete Classes (module)	•	
Number of Abstract Classes (module)	•	
Abstractness (module)	•	
Lines Of Code (Java, Interface)	•	
Number Of Methods (Java, Interface)	•	
Number of Method Parameters (method)	•	
Lines Of Code (method)	•	
Number of Children Metric	•	
Number of files (module)	•	
Number of lines (module)	•	
Number of classes (module)	•	
Number of interfaces (module)	•	
Number of enumerations (module)	•	

Table C.2: Built-in Metric Groups

Name	Description	# Metrics
Overview	Overview Pyramid Metrics	4
EnumerationMetrics	Basic Metrics for Enumerations	2
Instability	Instability Metric	3
JavaMetrics	Number of Java metrics	3
ClassStatistics	Simple Class Statistics	10
McCabe	McCabe Cyclomatic Complexity	2
ObjectOrientedProjectStatistics	Object Oriented Project Statistics	4
ProjectStatistics	Project Statistics Calculation	9
WeigthedMethodsPerClass	Weighted methods per class	1
CodeStatistics	Code Statistics	4
Abstractness	Abstractness Metric	3
InterfaceMetrics	Basic Metrics for Interfaces	2
MethodStatistics	Method Statistics	2
NumberOfChildren	Number of Children Metric	1
ModuleStatistics	Calculate Module Statistics per project	5

```
<!ELEMENT result-type (#PCDATA) >
```

```
<!ELEMENT result-category (#PCDATA) >
```


Appendix D

Domain-Specific Languages Bibliography

The first DSL, apt, appeared at 1957–1958 [220]. Although, the scientific community didn't give the proper attention to the design and the development of DSLs until the 1990s. There are two main publications that can act as bibliographical surveys:

- A. van Deursen, P. Klint, J. Visser. *Domain-Specific Languages: An Annotated Bibliography*. ACM SIGPLAN Notices, 35(6):26-36, 2000.
- Marjan Mernik, Jan Heering, and Anthony M. Sloane. *When and how to develop domain-specific languages*. ACM Computing Surveys, 37(4):316-344, 2005.

The major conferences and workshops that were devoted partly or in the whole to DSLs are included in the following (not exhaustive) list:

- *USENIX Symposium on Very High Level Languages (VHLL 94)*, 1994.
- *Workshop on Compiler Techniques for Application Domain Languages and Extensible Language Models (ALEL 96)*, 1996.
- *Design of Application Languages and Extensible Language Models (DALEL 96)*, 1996.
- *DSL 97 – First ACM SIGPLAN Workshop on Domain-Specific Languages, in Association with POPL97*, 1997.
- *USENIX Conference on Domain-Specific Languages*, 1997.
- *Second USENIX conference on Domain-Specific Languages (DSL 99)*, 1999.
- *Third International Workshop on Generating Tools from Language Definitions (GTLD'99)*, 1999.
- *International Conference on Domain-Specific Languages (DSL 99)*, 1999.
- *34th Hawaii International Conference on System Sciences (HICSS-34)*, 2001.
- *35th Hawaii International Conference on System Sciences (HICSS-35)*, 2002.
- *36th Hawaii International Conference on System Sciences (HICSS-36)*, 2003.
- *XP2005 Workshop: Agile Development with Domain Specific Languages*

- *DSL-WC 2009 – IFIP Domain Specific Language Working Conference, 2009.*
- *DSL 2011 – IFIP Working Conference on Domain-Specific Languages, 2011.*
- *International Workshop on DSL Architecting and DSL-based Architectures (DADA'14), 2014.*

Various peer-reviewed journals published special issues, which focused to the area of DSLs:

- Special Issue on Domain-Specific Languages, *IEEE Transactions on Software Engineering*, 25(3), May/June 1999.
- Special Issue on Domain-Specific Languages, Part I, *Journal of Information Technology*, 9(4), 2001.
- Special Issue on Domain-Specific Languages, Part II, *Journal of Information Technology*, 10(1), 2002.

The following PhD Theses and books are also focused on the design and implementation of DSLs:

- S. Thibault. *Domain-Special Languages. Conception, Implementation and Application*. PhD thesis, University of Rennes, France, 1998.
- Philipp W. Kutter. *MONTAGES: Engineering of Computer Languages*. PhD thesis, Swiss Federal Institute of Technology, Zurich, January 2004.
- Debasish Ghosh. *DSLs in Action*. Manning Publications, 2010.
- Debasish Ghosh. *DSL for the uninitiated*. ACM Queue, June 2011.
- G. Dubochet. *Embedded Domain-Specific Languages using Libraries and Dynamic Metaprogramming*. PhD Thesis, EPFL, Lausanne, 2011.
- Martin Fowler. *Domain-specific languages*. Addison-Wesley, Upper Saddle River, NJ, 2011.
- J. C. Vogt. *Type Safe Integration of Query Languages into Scala*, Ph.D. thesis, RWTH Aachen University, 2011.
- Markus Voelter, *DSL Engineering: Designing, Implementing and Using Domain-specific Languages*, CreateSpace Independent Publishing Platform, 2013.

The above list does not exhaust the literature in the field, but it still provides a basic guide for research in the field.

Appendix E

Summary of the J% Grammar

The J% complete grammar is presented here. It uses the following BNF conventions:

- $[x]$ denotes zero or one occurrences of x .
- $\{x\}$ denotes zero or more occurrences of x .
- $x \mid y$ means one of either x or y .
- All characters literals, are enclosed in double quotations e.g. "<" TypeArguments ">"

Identifier:

IDENTIFIER

AnyCharacter:

ANY_CHARACTER_MATCH

QualifiedIdentifier:

Identifier { "." Identifier }

Literal:

IntegerLiteral
FloatingPointLiteral
CharacterLiteral
StringLiteral
BooleanLiteral
NullLiteral

Expression:

Expression1 [AssignmentOperator Expression1 "]"]

AssignmentOperator:

"="

"+="

"_="

"*="

"/="

"&="

"|="

```

"^="
"%="
"<<="
">>="
">>>="

```

Type:

```

Identifier [ TypeArguments ] { "." Identifier [ TypeArguments ] } { "[" "]" }
BasicType

```

TypeArguments:

```

< TypeArgument { "," TypeArgument } >

```

TypeArgument:

```

Type "?" [ ( "extends" | "super" ) Type ]

```

StatementExpression:

```

Expression

```

ConstantExpression:

```

Expression

```

Expression1:

```

Expression2 [ Expression1Rest ]

```

Expression1Rest:

```

"?" Expression ":" Expression1

```

Expression2:

```

Expression3 [ Expression2Rest ]

```

Expression2Rest:

```

{ InfixOp Expression3 }
Expression3 "instanceof" Type

```

InfixOp:

```

"|"
"&&"
"|"
"^"
"&"
"=="
"!="
"<"
">"
"<="
">="
"<<"
">>"

```

```

">>>"
"+"
"_"
"*"
"/"
"%"

```

Expression3:

```

PrefixOp Expression3
( Expression | Type ) Expression3
Primary { Selector } { PostfixOp }

```

Primary:

```

ParExpression
NonWildcardTypeArguments ( ExplicitGenericInvocationSuffix | "this" Arguments )
"this" [ Arguments ]
"super" SuperSuffix
Literal
"new" Creator
Identifier { "." Identifier }[ IdentifierSuffix ]
BasicType { "[" "]" } ".class"
"void.class"

```

IdentifierSuffix:

```

[ "(" ] { "[" "]" } "." "class" | Expression ] ")"
Arguments
"." ( "class" | ExplicitGenericInvocation | "this" | "super" Arguments | "new"
[ NonWildcardTypeArguments ] InnerCreator )

```

ExplicitGenericInvocation:

```

NonWildcardTypeArguments ExplicitGenericInvocationSuffix

```

NonWildcardTypeArguments:

```

"<" TypeList ">"

```

ExplicitGenericInvocationSuffix:

```

"super" SuperSuffix
Identifier Arguments

```

PrefixOp:

```

"++"
"--"
"! "
"~"
"+"
"_"

```

PostfixOp:

```

"++"

```


"--"

Selector: Selector:

"." Identifier [Arguments]
 "." ExplicitGenericInvocation
 "." "this"
 "." "super" SuperSuffix
 "." "new" [NonWildcardTypeArguments] InnerCreator
 [Expression]

SuperSuffix:

Arguments
 "." Identifier [Arguments]

BasicType:

"byte"
 "short"
 "char"
 "int"
 "long"
 "float"
 "double"
 "boolean"

Arguments:

"(" [Expression { "," Expression }] ")"

Creator:

[NonWildcardTypeArguments] CreatedName "(" ArrayCreatorRest | ClassCreatorRest ")"

CreatedName:

Identifier [NonWildcardTypeArguments] { "." Identifier [NonWildcardTypeArguments] }

InnerCreator:

Identifier ClassCreatorRest

ArrayCreatorRest:

["("] { "[" "]" } ArrayInitializer | Expression [[Expression]] { "[" "]" })"

ClassCreatorRest:

Arguments [ClassBody]

ArrayInitializer:

{ [VariableInitializer { "," VariableInitializer } [","]] }

VariableInitializer:

ArrayInitializer
 Expression

```

ParExpression:
    "(" Expression ")"

Block:
    { BlockStatements }

BlockStatements:
    { BlockStatement }

BlockStatement :
    LocalVariableDeclarationStatement
    ClassOrInterfaceDeclaration
    [ Identifier ":" ] Statement

LocalVariableDeclarationStatement:
    [ "final" ] Type VariableDeclarators ";"

Statement:
    Block
    assert Expression [ ":" Expression ] ";"
    "if" ParExpression Statement [ "else" Statement ]
    "for" "(" ForControl ")" Statement
    "while" ParExpression Statement
    "do" Statement "while" ParExpression ";"
    "try" Block ( Catches | [ Catches ] "finally" Block )
    "switch" ParExpression { SwitchBlockStatementGroups }
    "synchronized" ParExpression Block
    "return" [ Expression ] ";"
    "throw" Expression ";"
    "break" [ Identifier ]
    "continue" [ Identifier ]
        ";"
        StatementExpression ";"
        Identifier : Statement

Catches:
    CatchClause { CatchClause }

CatchClause:
    "catch" "(" FormalParameter ")" Block

SwitchBlockStatementGroups:
    { SwitchBlockStatementGroup }

SwitchBlockStatementGroup:
    SwitchLabel BlockStatements

SwitchLabel:
    "case" ConstantExpression ":"

```

```

"case" EnumConstantName ":"
"default" ":"

```

```

MoreStatementExpressions:
{ ",", StatementExpression }

```

```

ForControl:
  ForVarControl
  ForInit ";" [ Expression ] ";" [ ForUpdate ]

```

```

ForVarControl
  [ "final" ] [ Annotations ] Type Identifier ForVarControlRest

```

```

Annotations:
  Annotation [ Annotations ]

```

```

Annotation:
  "@" TypeName [ "(" [ Identifier "=" ] ElementValue ")" ]

```

```

ElementValue:
  ConditionalExpression
  Annotation
  ElementValueArrayInitializer

```

```

ConditionalExpression:
  Expression2 Expression1Rest

```

```

ElementValueArrayInitializer:
  { [ ElementValues ] [ "," ] }

```

```

ElementValues:
  ElementValue [ ElementValues ]

```

```

ForVarControlRest:
  VariableDeclaratorsRest ";" [ Expression ] ";" [ ForUpdate ]
  ":" Expression

```

```

ForInit:
  StatementExpression Expressions

```

```

Modifier:
  Annotation
  "public"
  "protected"
  "private"
  "static"
  "abstract"
  "final"
  "native"

```

```

"synchronized"
"transient"
"volatile"
"strictfp"

VariableDeclarators:
    VariableDeclarator { "," VariableDeclarator }

VariableDeclaratorsRest:
    VariableDeclaratorRest { "," VariableDeclarator }

ConstantDeclaratorsRest:
    ConstantDeclaratorRest { , ConstantDeclarator }

VariableDeclarator:
    Identifier VariableDeclaratorRest

ConstantDeclarator:
    Identifier ConstantDeclaratorRest

VariableDeclaratorRest:
    { "[" "]" } [ "=" VariableInitializer]

ConstantDeclaratorRest:
    { "[" "]" } "=" VariableInitializer

VariableDeclaratorId:
    Identifier { "[" "]" }

CompilationUnit:
    [ [ Annotations ] "package" QualifiedIdentifier ";" ] { ImportDeclaration }
    { TypeDeclaration }

ImportDeclaration:
    "import" [ "static" ] Identifier { "." Identifier } [ ".*" ] ";"

TypeDeclaration:
    ClassOrInterfaceDeclaration
    ExternalDeclaration
";"

ExternalDeclaration:
    { Modifier } "external" Identifier "extends" Type ExternalBody

ExternalBody:
    "{" { AnyCharacter } "}"

ClassOrInterfaceDeclaration:
    { Modifier } ( ClassDeclaration | InterfaceDeclaration )

```

ClassDeclaration:

NormalClassDeclaration
EnumDeclaration

NormalClassDeclaration:

"class" Identifier [TypeParameters] ["extends" Type] ["implements" TypeList] ClassBody

TypeParameters:

"<" TypeParameter { "," TypeParameter } ">"

TypeParameter:

Identifier ["extends" Bound]

Bound:

Type { "&" Type }

EnumDeclaration:

"enum" Identifier ["implements" TypeList] EnumBody

EnumBody:

{ [EnumConstants] [","] [EnumBodyDeclarations] }

EnumConstants:

EnumConstant
EnumConstants "," EnumConstant

EnumConstant:

Annotations Identifier [Arguments] [ClassBody]

EnumBodyDeclarations:

";" { ClassBodyDeclaration }

InterfaceDeclaration:

NormalInterfaceDeclaration
AnnotationTypeDeclaration

NormalInterfaceDeclaration:

"interface" Identifier [TypeParameters] ["extends" TypeList] InterfaceBody

TypeList:

Type { "," Type }

AnnotationTypeDeclaration:

"@" "interface" Identifier AnnotationTypeBody

AnnotationTypeBody:

{ [AnnotationTypeElementDeclarations] }

```

AnnotationTypeElementDeclarations:
    AnnotationTypeElementDeclaration
    AnnotationTypeElementDeclarations AnnotationTypeElementDeclaration

AnnotationTypeElementDeclaration:
    { Modifier } AnnotationTypeElementRest

AnnotationTypeElementRest:
    Type Identifier AnnotationMethodOrConstantRest ";"
    ClassDeclaration
    InterfaceDeclaration
    EnumDeclaration
    AnnotationTypeDeclaration

AnnotationMethodOrConstantRest:
    AnnotationMethodRest
    AnnotationConstantRest

AnnotationMethodRest:
    "(" ")" [ DefaultValue ]

AnnotationConstantRest:
    VariableDeclarators

DefaultValue:
    "default" ElementValue

ClassBody:
    { "{" ClassBodyDeclaration "}" }

InterfaceBody:
    { "{" InterfaceBodyDeclaration "}" }

ClassBodyDeclaration:
    ";"
    [ "static" ] Block
    { Modifier } MemberDecl

MemberDecl:
    GenericMethodOrConstructorDecl
    MethodOrFieldDecl
    "void" Identifier VoidMethodDeclaratorRest
    Identifier ConstructorDeclaratorRest
    InterfaceDeclaration
    ClassDeclaration

GenericMethodOrConstructorDecl:
    TypeParameters GenericMethodOrConstructorRest

```

```

GenericMethodOrConstructorRest:
    ( Type | "void" ) Identifier MethodDeclaratorRest
    Identifier ConstructorDeclaratorRest

MethodOrFieldDecl:
    Type Identifier MethodOrFieldRest

MethodOrFieldRest:
    VariableDeclaratorRest
    MethodDeclaratorRest

InterfaceBodyDeclaration:
    ";"
    { Modifier } InterfaceMemberDecl

InterfaceMemberDecl:
    InterfaceMethodOrFieldDecl
    InterfaceGenericMethodDecl
    "void" Identifier VoidInterfaceMethodDeclaratorRest
    InterfaceDeclaration
    ClassDeclaration

InterfaceMethodOrFieldDecl:
    Type Identifier InterfaceMethodOrFieldRest

InterfaceMethodOrFieldRest:
    ConstantDeclaratorsRest ";"
    InterfaceMethodDeclaratorRest

MethodDeclaratorRest:
    FormalParameters { "[" "]" } [ "throws" QualifiedIdentifierList ] ( MethodBody | ";" )

VoidMethodDeclaratorRest:
    FormalParameters [ "throws" QualifiedIdentifierList ] ( MethodBody | ";" )

InterfaceMethodDeclaratorRest:
    FormalParameters { "[" "]" } [ "throws" QualifiedIdentifierList ] ";"

InterfaceGenericMethodDecl:
    TypeParameters ( Type | "void" ) Identifier InterfaceMethodDeclaratorRest

VoidInterfaceMethodDeclaratorRest:
    FormalParameters [ "throws" QualifiedIdentifierList ] ";"

ConstructorDeclaratorRest:
    FormalParameters [ "throws" QualifiedIdentifierList ] MethodBody

QualifiedIdentifierList:
    QualifiedIdentifier { "," QualifiedIdentifier }

```

FormalParameters:

"(" [FormalParameterDecls] ")"

FormalParameterDecls:

["final"] [Annotations] Type FormalParameterDeclsRest "]"

FormalParameterDeclsRest:

VariableDeclaratorId ["," FormalParameterDecls]
... VariableDeclaratorId

MethodBody:

Block

EnumConstantName:

Identifier

Appendix F

Code Generation Templates

F.1 Regular Expression Module

```
package $PACKAGE;

import org.jmod.dsl.regex.Regex;
import $CLASS_CONFIGURATION;

public class $CLASSNAME extends Regex<$CLASS_CONFIGURATION> {
    private final String _regex = "$REGEX";

    public $CLASSNAME() {
        super(_regex, new $CLASS_CONFIGURATION());
    }
}
```

F.2 SQLModule

```
package $PACKAGE;

import org.jmod.dsl.sql.SQLQuery;
import $CLASS_CONFIGURATION;

import java.sql.*;

public class $CLASSNAME extends SQLQuery<$CLASS_CONFIGURATION> {
#foreach( $VDECL in $SQL_CODE_VAR_DECL )
    private $VDECL;
#end

    public $CLASSNAME($SQL_CODE_CONSTRUCTOR) {
        super(new $CLASS_CONFIGURATION());
        this.sqlStatement = "$SQL_CODE";
#foreach( $MAPPED_TYPES in $SQL_CODE_MAPPED_TYPES )
        this.$MAPPED_TYPES.getVarName() = $MAPPED_TYPES.getVarName();
#end
    }

    public PreparedStatement getStatement(Connection c) throws SQLException {
        PreparedStatement pstmt = c.prepareStatement(this.sqlStatement);
#foreach( $STMNT in $SQL_CODE_PS_STATEMENTS )
        pstmt.$STMNT;
#end

        return pstmt;
    }
}
```

Acronyms

ACM	Association for Computing Machinery
AOP	Aspect-Oriented Programming
ANSI	American National Standards Institute
API	Application Programming Interface
APT	Java Annotation Processing Tool
AST	Abstract Syntax Tree
BNF	Backus-Naur Form
CCS	Computing Classification System
CK	Chidamber and Kemerer
CLMT	Cross-Language Metric Tool
CLOC	Comments Lines of Code
CPU	Central Processing Unit
CQL	Cassandra Query Language
CSS	Cascading Style Sheets
DFA	Deterministic Finite Automata
DSC	DSL Syntax Checking
DSL	Domain-Specific Language
DSO	Domain-Specific Optimisation
DTD	Document Type Definition
DTL	Database Template Library
DOM	Document Object Model
ECOOP	European Conference of Object-Oriented Programming
EU	European Union
EXT	Extensible

FIRE/J Fast Implementation of Regular Expressions

GNU GNU is Not Unix

GPL General-Purpose Language

HTML Hyper-Text Markup Language

IDE Integrated Development Environment

IEEE Institute of Electrical & Electronics Engineers

IP Internet Protocol

ISO International Organization for Standardization

IPv4 Internet Protocol Version 4

IXR Intermediate XML Representation

JAR Java Archive

JDBC Java Database Connection

JDK Java Development Kit

JIT Just-In Time

JOOQ Java Object Oriented Query

JPA Java Persistence API

JRE Java Runtime Environment

JSON JavaScript Object Notation

JSP Java Server Pages

JSR Java Specification Request

JTS Jakarta Tool Suite

JVM Java Virtual Machine

LC Line Count

LED Light Emitted Diode

LINQ Language INtegrated Query

LMS Lightweight Modular Staging

LOC Lines of Code

NFA Non-Deterministic Finite Automata

ODB Object Database

ODBC Open Database Connectivity

OO Object-Oriented

OPT Optimized

ORM Object-relational Mapping

OSS Open Source Software

POLA Principle of Least Astonishment

PCRE Perl Compatible Regular Expressions

PDF Portable Document Format

PhD Doctor of Philosophy

POSIX Portable Operating System Interface for Unix

PoPL Principles of Programming Languages

RAD Rapid Application Development

RDBMS Relational DataBase Management System

RFC Request For Comments

SAX Simple API for XML

SDK Software Development Kit

SEC Security

SENSE Software Engineering and Security

SOCI Simple Oracle Call Interface

SLICK Scala Language-Integrated Connection Kit

SLOC Source Lines of Code

STC Static Type Checking

STL Standard Template Library

SQL Structured Query Language

SQL DOM SQL Document Object Model

SQO-OSS Software Quality Observatory of Open Source Software

TXL Turing eXtender Language

UML Unified Modeling Language

UniPi University of Piraeus

URL Uniform Resource Location

VAL Value-oriented Algorithmic Language

VHDL VHSIC Hardware Description Language

XJ XML Enhancements for Java

XHTML Extensible Hypertext Markup Language

XML eXtensible Markup Language

XOC eXtension-Oriented Compiler

XPATH XML Path

XSD XML Schema Definition

XSLT Extensible Stylesheet Language and Transformations

XTC eXTensible Compiler

Bibliography

- [1] Regular Expressions for Java, October 2001. URL <http://savannah.gnu.org/projects/gnu-regexp/>.
- [2] Regex - GNU Regular Expression Library, 2001. URL http://www.gnu.org/software/libc/manual/html_node/Regular-Expressions.html.
- [3] J2SDK Regular Expressions Engine 1.4.x, 2003. URL <http://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html>.
- [4] Sensory Netowkrs (Intel), Nodalcore C Series - Content Classification Accelerator, September 2004. URL <http://www.intel.com/content/www/us/en/communications/sw-pattern-matching-solution-for-security-applications.html>.
- [5] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs 2nd edition*. MIT Press, 1998.
- [6] David Abrahams and Aleksey Gurtovoy. *C++ Template Meta-programming: Concepts, Tools, and Techniques from Boost and Beyond*. Addison-Wesley Professional, 2004. ISBN 0321227255.
- [7] W.R. Adrion. Research Methodology in Software Engineering. *ACM SIGSOFT Software Engineering Notes*, 1993.
- [8] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques and Tools*. Addison-Wesley, Reading, MA, 1985. ISBN 0-201-10088-6.
- [9] V. S. Alagar, Qiaoyun Li, and Olga S. Ormandjieva. Assessment of Maintainability in Object-oriented Software. In *TOOLS '01: Proceedings of the 39th International Conference and Exhibition on Technology of Object-Oriented Languages and Systems*, page 194, Washington, DC, USA, 2001. IEEE Computer Society.
- [10] Andrei Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2001. ISBN 0-201-70431-5.
- [11] Jim Alves-Foss, editor. *Formal Syntax and Semantics of Java*, volume 1523 of *Lecture Notes in Computer Science*, 1999. Springer. ISBN 3-540-66158-1.
- [12] Stuart Anderson and Konstantinos Tourlas. Design for Proof: An Approach to the Design of Domain-specific Languages. *Formal Aspects of Computing*, 10(5–6):452–468, May 1998. ISSN 0934-5043.

- [13] Matthias Anlauff, Philipp W. Kutter, Alfonso Pierantonio, and Asuman Sunbul. Using Domain-specific Languages for the Realization of Component Composition. In *FASE '00: Proceedings of the Third International Conference on Fundamental Approaches to Software Engineering*, pages 112–126. Springer-Verlag, 2000. ISBN 3-540-67261-3.
- [14] Aivar Annamaa, Andrey Breslav, Jevgeni Kabanov, and Varmo Vene. An Interactive Tool for Analyzing Embedded SQL Queries. In *Proceedings of the 8th Asian Conference on Programming Languages and Systems, APLAS'10*, pages 131–138, Berlin, Heidelberg, 2010. Springer-Verlag. ISBN 3-642-17163-X, 978-3-642-17163-5.
- [15] A. Arvind and K. P. Gostelow. The U-Interpreter. *Computer*, 15(2):42–49, 1982. ISSN 0018-9162.
- [16] Maurice J. Bach. *The Design of the UNIX Operating System*. Prentice Hall, Englewood Cliffs, NJ, 1986. ISBN 0132017571.
- [17] Philipp Bachmann. Static and Metaprogramming Patterns and Static Frameworks: A Catalog. An Application. In *PLoP '06: Proceedings of the 2006 conference on Pattern languages of programs*, pages 1–33, New York, NY, USA, 2006. ACM. ISBN 978-1-60558-372-3.
- [18] John W. Backus. The syntax and semantics of the proposed international algebraic language of the zurich acm-gamm conference. In *Proceeding of the International Conference on Information Processing*, 1959.
- [19] Ricardo Baeza-Yates and Gaston H. Gonnet. A New Approach to Text Searching. *Communications of the ACM*, 35(10):74–82, 1992. ISSN 0001-0782.
- [20] Rajendra K. Bandi, Vijay K. Vaishnavi, and Daniel E. Turk. Predicting Maintenance Performance using Object-oriented Design Complexity Metrics. *IEEE Transactions on Software Engineering*, 29(1):77–87, 2003. ISSN 0098-5589.
- [21] Henk Barendregt, S. Abramsky, D. M. Gabbay, T. S. E. Maibaum, and H. P. Barendregt. Lambda Calculi with Types. In *Handbook of Logic in Computer Science*, pages 117–309. Oxford University Press, 1992.
- [22] Ankica Barivsic, Vasco Amaral, Miguel Goulao, and Bruno Barroca. Quality in Use of Domain-specific Languages: A Case Study. In *Proceedings of the 3rd ACM SIGPLAN workshop on Evaluation and usability of programming languages and tools, PLATEAU '11*, pages 65–72, New York, NY, USA, 2011. ISBN 978-1-4503-1024-6.
- [23] Richard Barker and Ewan Tempero. A Large-scale Empirical Comparison of Object-oriented Cohesion Metrics. In *APSEC '07: Proceedings of the 14th Asia-Pacific Software Engineering Conference*, pages 414–421, Washington, DC, USA, 2007. IEEE Computer Society. ISBN 0-7695-3057-5.
- [24] David R. Barstow. Domain-specific Automatic Programming. *IEEE Transactions on Software Engineering*, 11(11):1321–1336, 1985. ISSN 0098-5589.
- [25] V. R. Basili, R. W. Selby, and D. H. Hutchens. Experimentation in Software Engineering. *IEEE Transactions of Software Engineering*, 12:733–743, July 1986. ISSN 0098-5589.
- [26] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. A Validation of Object-oriented Design Metrics as Quality Indicators. *IEEE Transaction in Software Engineering*, 22(10):751–761, 1996. ISSN 0098-5589.

- [27] D. Batory, B. Lofaso, and Yannis Smaragdakis. JTS: Tools for Implementing Domain-specific Languages. In *ICSR '98: Proceedings of the 5th International Conference on Software Reuse*, page 143. IEEE Computer Society, 1998. ISBN 0-8186-8377-5.
- [28] Christian Bauer and Gavin King. *Java Persistence with Hibernate*. Manning Publications, 2006. ISBN 1932394885.
- [29] Jon Bentley. Programming Pearls: Little Languages. *Communications of the ACM*, 29(8): 711–721, 1986. ISSN 0001-0782.
- [30] Elliot Berk and C. Scott Ananian. JLex: A Lexical Analyzer Generator for Java, 2000. URL <https://www.cs.princeton.edu/~appel/modern/java/JLex/>.
- [31] T. Berners-Lee, R. Fielding, and L. Masinter. Uniform Resource Identifiers (URI): Generic Syntax. RFC 2396 (Draft Standard). Obsoleted by RFC 3986, updated by RFC 2732.
- [32] Federico Biancuzzi and Shane Warden. *Masterminds of Programming: Conversations with the Creators of Major Programming Languages*. O'Reilly Media, 2009. ISBN 0596515170.
- [33] Gavin Bierman, Erik Meijer, and Wolfram Schulte. The Essence of Data Access in Cw. In *ECOOP 2005: Proceedings of the 19th European Conference on Object-Oriented Programming*, pages 287–311, 2005.
- [34] Patrick Blackburn, Johan Bos, and Kristina Striegnitz. *Learn Prolog Now! (Texts in Computing, Vol. 7)*. College Publications, 2006. ISBN 1904987176.
- [35] Joshua Bloch. How to Design a Good API and Why It Matters. In *Companion to the 21st ACM SIGPLAN symposium on Object-oriented programming systems, languages, and applications*, OOPSLA '06, pages 506–507, New York, NY, USA, 2006. ACM. ISBN 1-59593-491-X.
- [36] James Kenneth Blundell, Mary Lou Hines, and Jerrold Stach. The Measurement of Software Design Quality. *Annals of Software Engineering*, 4:235–255, 1997. ISSN 1022-7091.
- [37] IEEE Standards Board. IEEE Standard Glossary of Computer Languages (Std 610.13-1993). Technical report, IEEE Computer Society, 1993.
- [38] Martin Bravenboer and Eelco Visser. Concrete Syntax for Objects. Domain-specific Language Embedding and Assimilation without Restrictions. In Douglas C. Schmidt, editor, *Proceedings of the 19th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2004)*, pages 365–383, Vancouver, Canada, October 2004. ACM Press.
- [39] Martin Bravenboer, René de Groot, and Eelco Visser. MetaBorg in Action: Examples of Domain-specific Language Embedding and Assimilation using Stratego/XT. In R. Lämmel and J. Saraiva, editors, *Proceedings of the Summer School on Generative and Transformational Techniques in Software Engineering (GTTSE 2005)*, volume 4143, pages 297–311, Braga, Portugal, 2005. Springer Verlag.
- [40] Martin Bravenboer, René de Groot, and Eelco Visser. MetaBorg in Action: Examples of Domain-specific Language Embedding and Assimilation using Stratego/xt. In *GTTSE*, pages 297–311, 2006.

- [41] Lionel C. Bri, Stefan Ikonovovski, and Hakim Lounis. Investigating Quality Factors in Object-oriented Designs: An Industrial Case Study. In *In proceedings of the 21st IEEE International Conference on Software Engineering*, 1999.
- [42] Lionel C. Briand, Jürgen Wüst, John W. Daly, and Victor D. Porter. Exploring the Relationship Between Design Measures and Software Quality in Object-oriented systems. *Journal of Systems and Software*, 51(3):245–273, 2000. ISSN 0164-1212.
- [43] Lionel C. Briand, Jürgen Wüst, and Hakim Lounis. Replicated Case Studies for Investigating Quality Factors in Object-oriented Designs. *Empirical Software Engineering*, 6(1):11–58, 2001. ISSN 1382-3256.
- [44] Magiel Bruntink. Predicting Class Testability using Object-oriented Metrics. In *4th IEEE International Workshop on Source Code Analysis and Manipulation (SCAM'04)*, pages 136–145. IEEE Society Press, 2004.
- [45] J. A. Brzozowski. Derivatives of Regular Expressions. *Journal of the ACM*, 11(4):481–494, 1964. ISSN 0004-5411.
- [46] Eugene Burmako, Martin Odersky, Christopher Vogt, Stefan Zeiger, and Adriaan Moors. SIP 16: Self-cleaning Macros. Technical report, ScalaMacros.org, March 2012.
- [47] Luca Cardelli and Peter Wegner. On Understanding Types, Data Abstraction, and Polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985. ISSN 0360-0300.
- [48] Robert Cartwright and Mike Fagan. Soft Typing. In *PLDI '91: Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 278–292, New York, NY, USA, 1991. ACM Press. ISBN 0-89791-428-7.
- [49] Emmanuel Cecchet, Anupam Chanda, Sameh Elnikety, Julie Marguerite, and Willy Zwaenepoel. Performance Comparison of Middleware Architectures for Generating Dynamic Web Content. In *Proceedings of the ACM/IFIP/USENIX 2003 International Conference on Middleware*, Middleware '03, pages 242–261, New York, NY, USA, 2003. Springer-Verlag New York, Inc. ISBN 3-540-40317-5.
- [50] Surajit Chaudhuri, Vivek Narasayya, and Manoj Syamala. Bridging the Application and DBMS Divide Using Static Analysis and Dynamic Profiling. In *Proceedings of the 35th SIGMOD International Conference on Management of Data*, SIGMOD '09, pages 1039–1042, New York, NY, USA, 2009. ACM. ISBN 978-1-60558-551-2.
- [51] John C. Cherniavsky and Carl H. Smith. On Weyuker's Axioms for Software Complexity Measures. *IEEE Transactions on Software Engineering*, 17(6):636–638, 1991. ISSN 0098-5589.
- [52] S. R. Chidamber and C. F. Kemerer. A Metrics Suite for Object-oriented Design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994. ISSN 0098-5589.
- [53] Hyun Cho. Using Metaprogramming to Implement a Testing Framework. In *ACM-SE 47: Proceedings of the 47th Annual Southeast Regional Conference*, pages 1–2, New York, NY, USA, 2009. ACM. ISBN 978-1-60558-421-8.
- [54] Noam Chomsky. Three Models for the Description of Language. *IEEE Transactions On Information Theory*, 2(3):113–124, 1956.

- [55] Tom Christiansen and Nathan Torkington. *Perl Cookbook*. O'Reilly, 2nd edition, 2003. ISBN 0-596-00313-7.
- [56] Alonzo Church. A Formulation of Simple Theory Types. *Journal of Symbolic Logic*, 5:56–58, 1940.
- [57] Thomas Cleenewerck. Component-based DSL Development. In *GPCE '03: Proceedings of the second international conference on Generative programming and component engineering*, pages 245–264. Springer-Verlag New York, Inc., 2003. ISBN 3-540-20102-5.
- [58] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. Using Metrics to Evaluate Software System Maintainability. *Computer*, 27(8):44–49, 1994. ISSN 0018-9162.
- [59] MONO Community. The MONO Project, November 2014. URL <http://www.mono-project.com/>.
- [60] James R. Cordy. Source Transformation, Analysis and Generation in TXL. In *PEPM '06: Proceedings of the 2006 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*, pages 1–11, New York, NY, USA, 2006. ACM Press. ISBN 1-59593-196-1.
- [61] James R. Cordy and Medha Shukla. Practical Metaprogramming. In *CASCON '92: Proceedings of the 1992 conference of the Centre for Advanced Studies on Collaborative research*, pages 215–224. IBM Press, 1992.
- [62] Russ Cox, Tom Bergan, Austin T. Clements, Frans Kaashoek, and Eddie Kohler. Xoc: An Extension-oriented Compiler for Systems Programming. In *ASPLOS XIII: Proceedings of the 13th international conference on Architectural support for programming languages and operating systems*, pages 244–254, New York, NY, USA, 2008. ACM. ISBN 978-1-59593-958-6.
- [63] Russell Stensby Cox. *An Extension-oriented Compiler*. PhD thesis, Harvard University, 2008.
- [64] John Creswell. *Research Design: Qualitative, Quantitative and Mixed Methods Approaches*. Sage Publications, Thousand Oaks, Calif, 2009. ISBN 9781412965576.
- [65] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. ACM Press/Addison-Wesley Publishing Co., New York, NY, USA, 2000. ISBN 0-201-30977-7.
- [66] Michael D. Ernst. Annotations on Java Types (JSR308). Technical report, MIT, May 2007.
- [67] Melis Dagpinar and Jens H. Jahnke. Predicting Maintainability with Object-oriented Metrics - An Empirical Comparison. In *WCRE '03: Proceedings of the 10th Working Conference on Reverse Engineering*, page 155, Washington, DC, USA, 2003. IEEE Computer Society. ISBN 0-7695-2027-8.
- [68] Mark Davis and Andy Heninger. Unicode Technical Standard: Unicode Regular Expressions, 2004. URL <http://www.unicode.org/reports/tr18/>.
- [69] Rodrigo Barreto de Oliveira. The boo manifesto, 2005.

- [70] James C. Dehnert and Alexander A. Stepanov. Fundamentals of Generic Programming. In *Selected Papers from the International Seminar on Generic Programming*, pages 1–11, London, UK, 2000. Springer-Verlag. ISBN 3-540-41090-2.
- [71] Ignatios Deligiannis, Martin Shepperd, Manos Roumeliotis, and Ioannis Stamelos. An Empirical Investigation of an Object-oriented Design Heuristic for Maintainability. *Journal of Systems Software*, 65(2):127–139, 2003. ISSN 0164-1212.
- [72] Sophia Drossopoulou and Susan Eisenbach. Java is Type Safe - Probably. In *ECOOP*, pages 389–418, 1997.
- [73] Sophia Drossopoulou, Susan Eisenbach, and Sarfraz Khurshid. Is the Java Type System Sound? *Theory and Practice of Object Systems*, 5(1):3–24, 1999.
- [74] Gilles Dubochet. *Embedded Domain-specific Languages using Libraries and Dynamic Metaprogramming*. PhD thesis, Ecole Polytechnique Federale de Lausanne, May 2011.
- [75] Gilles Dubochet and Martin Odersky. Compiling Structural Types on the JVM: A Comparison of Reflective and Generative Techniques from Scala’s Perspective. In *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*, ICPOOLPS ’09, pages 34–41, New York, NY, USA, 2009. ACM. ISBN 978-1-60558-541-3.
- [76] Jonathan Scott Duff. Introduction to Perl 6 Grammars, 2014. URL <https://github.com/perl6-docs/blob/master/intro/p6-grammar-intro.pod>.
- [77] Fernando Brito e Abreu and Rogerio Carapucca. Candidate Metrics for Object-oriented Software within a Taxonomy Framework. *Journal of Systems and Software*, 26(1):87–96, 1994. ISSN 0164-1212.
- [78] Fernando Brito e Abreu and Walcelio Melo. Evaluating the Impact of Object-oriented Design on Software Quality. In *METRICS ’96: Proceedings of the 3rd International Symposium on Software Metrics*, page 90, Washington, DC, USA, 1996. IEEE Computer Society. ISBN 0-8186-7364-8.
- [79] Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. Selecting Empirical Methods for Software Engineering Research. In Forrest Shull, Janice Singer, and Dag I. K. Sjöberg, editors, *Guide to Advanced Empirical Software Engineering*, pages 285–311. Springer London, 2008. ISBN 978-1-84800-044-5.
- [80] Andrzej Ehrenfeucht and Paul Zeiger. Complexity Measures for Regular Expressions. In *STOC ’74: Proceedings of the Sixth Annual ACM Symposium On Theory of Computing*, pages 75–79, New York, NY, USA, 1974. ACM Press.
- [81] Andrew Eisenberg and Jim Melton. SQLJ Part 1: SQL Routines Using the Java Programming Language. *SIGMOD Records*, 28(4):58–63, 1999. ISSN 0163-5808.
- [82] Andrew Eisenberg and Jim Melton. SQL:1999, Formerly Known as SQL3, 2003.
- [83] Torbjörn Ekman and Görel Hedin. The JastAdd Extensible Java Compiler. In *OOPSLA ’07: Companion to the 22nd ACM SIGPLAN conference on Object oriented programming systems and applications companion*, pages 773–774, New York, NY, USA, 2007. ACM. ISBN 978-1-59593-865-7.

- [84] Dawson R. Engler. Incorporating Application Semantics and Control into Compilation. In *Proceedings of Conference on Domain-Specific Languages*. USENIX, 1997.
- [85] Sebastian Erdweg. *Extensible Languages for Flexible and Principled Domain Abstraction*. PhD thesis, Philipps-Universität Marburg, 2013.
- [86] Sebastian Erdweg, Tillmann Rendel, Christian Kästner, and Klaus Ostermann. SugarJ: Library-based Syntactic Language Extensibility. In *Proceedings of Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 391–406. ACM, 2011.
- [87] Sebastian Erdweg, Paolo G. Giarrusso, and Tillmann Rendel. Language Composition Untangled. In *Proceedings of the Twelfth Workshop on Language Descriptions, Tools, and Applications*, LDTA '12, pages 7:1–7:8, New York, NY, USA, 2012. ACM. ISBN 978-1-4503-1536-4. doi: 10.1145/2427048.2427055. URL <http://doi.acm.org/10.1145/2427048.2427055>.
- [88] Sebastian Erdweg, Felix Rieger, Tillmann Rendel, and Klaus Ostermann. Layout-sensitive Language Extensibility with SugarHaskell. In *Proceedings of Haskell Symposium*, 2012.
- [89] Rickard E. Faith, Lars S. Nyland, and Jan F. Prins. KHEPERA: A System for Rapid Implementation of Domain-specific Languages. In *Proceedings of the Conference on Domain-Specific Languages*. USENIX, October 1997.
- [90] Norman E. Fenton and Shari Lawrence Pfleeger. *Software Metrics: A Rigorous and Practical Approach*. Course Technology, 1998.
- [91] Maydene Fisher, Jon Ellis, and Jonathan Bruce. *JDBC API Tutorial and Reference*. Addison Wesley, 3rd edition, 2003.
- [92] Robert W. Floyd. The Paradigms of Programming. *Communications of the ACM*, 22(8): 455–460, 1979. ISSN 0001-0782.
- [93] Robert W. Floyd and Jeffrey D. Ullman. The Compilation of Regular Expressions into Integrated Circuits. *Journal of the ACM*, 29(3):603–622, 1982.
- [94] International Organization for Standardization. Software Engineering - Product Quality - Part 1: Quality Model. Technical Report ISO/IEC 9126-1:2001(E), ISO, 2001.
- [95] Apache Software Foundation. The Apache Velocity Project, 2007.
- [96] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, Boston, MA, 2000. ISBN 0201485672.
- [97] Martin Fowler. *Domain-specific Languages*. Addison-Wesley, Upper Saddle River, NJ, 2011. ISBN 9780321712943.
- [98] Martin Fowler and Kendall Scott. *UML Distilled: Applying the Standard Object Modeling Language*. Addison-Wesley, Boston, MA, second edition, 2000. ISBN 0-201-65783-X.
- [99] Matthew Fowler and Brahm van Niekerk. Meta-programming for the Real World. In John M. Vlissides and Douglas C. Schmidt, editors, *Companion to the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2004, October 24-28, 2004, Vancouver, BC, Canada*, page 15. ACM, 2004. ISBN 1-58113-833-4.

- [100] Jeffrey E. F. Friedl. *Mastering Regular Expressions 2nd edition*. O'Reilly and Associates, Inc., Sebastopol, CA, 2002. ISBN 0-596-00289-0.
- [101] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-oriented Software*. Addison-Wesley, Indianapolis, IN 46920, 1994. ISBN 0-201-63361-2.
- [102] Ronald Garcia and Andrew Lumsdaine. Toward Foundations for Type-reflective Metaprogramming. In *GPCE '09: Proceedings of the eighth international conference on Generative programming and component engineering*, pages 25–34, New York, NY, USA, 2009. ACM. ISBN 978-1-60558-494-2.
- [103] Paul Gazzillo and Robert Grimm. SuperC: Parsing All of C by Taming the Preprocessor. In *Proceedings of the 33rd ACM SIGPLAN conference on Programming Language Design and Implementation, PLDI '12*, pages 323–334, New York, NY, USA, 2012. ACM. ISBN 978-1-4503-1205-9.
- [104] Marcela Genero, José Olivas, Mario Piattini, and Francisco Romero. Using Metrics to Predict Object-oriented Information Systems Maintainability. In *CAiSE '01: Proceedings of the 13th International Conference on Advanced Information Systems Engineering*, pages 388–401, London, UK, 2001. Springer-Verlag. ISBN 3-540-42215-3.
- [105] Debasish Ghosh. *DSLs in Action*. Manning Publications, 2010. ISBN 9781935182450.
- [106] Debasish Ghosh. DSL for the Uninitiated. *ACM Queue*, June 2011.
- [107] Joseph (Yossi) Gil and Keren Lenz. Simple and Safe SQL Queries with C++ Templates. *Science of Computer Programming*, 75:573–595, July 2010. ISSN 0167-6423.
- [108] Robert L. Glass, Venkataraman Ramesh, and Iris Vessey. An Analysis of Research in Computing Disciplines. *Communications of the ACM*, pages 89–94, 2004.
- [109] V-M. Glushkov. The Abstract Theory of Automata. *Dussian Mathematical Surveys*, 16:1–53, 1961.
- [110] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha. *The Java Language Specification, 3rd edition*. Addison-Wesley, 2005.
- [111] Carl Gould, Zhendong Su, and Premkumar Devanbu. Static Checking of Dynamically Generated Queries in Database Applications. In *Proceedings of the 26th International Conference on Software Engineering (ICSE'04)*, pages 645–654. IEEE, May 2004.
- [112] Georgios Gousios. *Tools and Methods for Large-scale Empirical Software Engineering Research*. PhD thesis, Athens Univeristy of Economincs and Business, 2009.
- [113] Jeff Gray, Kathleen Fisher, Charles Consel, Gabor Karsai, Marjan Mernik, and Juha-Pekka Tolvanen. DSLs: The Good, the Bad, and the Ugly. In *Companion to the 23rd ACM SIGPLAN conference on Object-oriented programming systems languages and applications, OOPSLA Companion '08*, pages 791–794, New York, NY, USA, 2008. ACM. ISBN 978-1-60558-220-7.
- [114] Robert Grimm. Better Extensibility through Modular Syntax. In *PLDI '06: Proceedings of the 2006 ACM SIGPLAN conference on Programming language design and implementation*, pages 38–51, New York, NY, USA, 2006. ACM. ISBN 1-59593-320-4.

- [115] The Open Group. The Single Unix Specification, Version 4, 2013. <http://www.unix.org/version4/>.
- [116] Torsten Grust and Manuel Mayr. A Deep Embedding of Queries into Ruby. In *Proceedings of the 2012 IEEE 28th International Conference on Data Engineering*, ICDE '12, pages 1257–1260, Washington, DC, USA, 2012. IEEE Computer Society. ISBN 978-0-7695-4747-3. doi: 10.1109/ICDE.2012.121. URL <http://dx.doi.org/10.1109/ICDE.2012.121>.
- [117] Gui Gui and Paul D. Scott. Ranking Reusability of Software Components using Coupling Metrics. *Journal of Systems and Software*, 80(9):1450–1459, 2007. ISSN 0164-1212.
- [118] Gopal Gupta and Enrico Pontelli. Specification, implementation, and verification of domain specific languages: A logic programming-based approach. In *Computational Logic: Logic Programming and Beyond, Essays in Honour of Robert A. Kowalski, Part I*, pages 211–239. Springer-Verlag, 2002. ISBN 3-540-43959-5.
- [119] Gursaran and Gurdev Roy. On the Applicability of Weyuker Property to Object-oriented Structural Inheritance Complexity Metrics. *IEEE Transactions on Software Engineering*, 27(4):381–384, 2001. ISSN 0098-5589.
- [120] Stefan Hanenberg. An Experiment about Static and Dynamic Type Systems: Doubts about the Positive Impact of Static Type Systems on Development Time. In *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*, OOPSLA '10, pages 22–35, New York, NY, USA, 2010. ACM. ISBN 978-1-4503-0203-6.
- [121] Matthew Harren, Mukund Raghavachari, Oded Shmueli, Michael G. Burke, Rajesh Bordawekar, Igor Pechtchanski, and Vivek Sarkar. XJ: Facilitating XML Processing in Java. In *WWW '05: Proceedings of the 14th international conference on World Wide Web*, pages 278–287, New York, NY, USA, 2005. ACM. ISBN 1-59593-046-9.
- [122] R. Harrison, S. Counsell, and R. Nithi. Experimental Assessment of the Effect of Inheritance on the Maintainability of Object-oriented Systems. *Journal of Systems and Software*, 52(2-3):173–179, 2000. ISSN 0164-1212.
- [123] Philip Hazel. Perl Compatible Regular Expressions, 2006. URL <http://www.pcre.org/>.
- [124] Jan Heering and Marjan Mernik. Domain-specific Languages for Software Engineering. In *Proceedings of the 35th Hawaii International Conference on System Sciences*. IEEE, 2002.
- [125] Zef Hemel, Lennart C. L. Kats, and Eelco Visser. Code Generation by Model Transformation: A Case Study in Transformation Modularity. In J. Gray, A. Pierantonio, and A. Vallecillo, editors, *Theory and Practice of Model Transformations. First International Conference on Model Transformation (ICMT 2008)*, pages 183–198, Heidelberg, July 2008. Springer. ISBN 978-3-540-69926-2.
- [126] Zef Hemel, Lennart C. L. Kats, Danny M. Groenewegen, and Eelco Visser. Code Generation by Model Transformation: A Case Study in Transformation Modularity. *Software and Systems Modeling*, 9(3):375–402, June 2010.
- [127] David Herman and Philippe Meunier. Improving the Static Analysis of Embedded Languages via Partial Evaluation. In *ICFP '04: Proceedings of the ninth ACM SIGPLAN international conference on Functional programming*, pages 16–27, New York, NY, USA, 2004. ACM. ISBN 1-58113-905-5.

- [128] R.M. Herndon and V.A. Berzins. The Realizable Benefits of a Prototyping Language. *IEEE Transactions on Software Engineering*, SE-(14):803–809, 1998.
- [129] Tomoyuki Higuchi and Atsushi Ohori. A Static Type System for JVM Access Control. *ACM Transactions on Programming Languages and Systems*, 29(1):4, 2007. ISSN 0164-0925.
- [130] Andreas Höfer and Walter F. Tichy. Status of Empirical Research in Software Engineering. In *Proceedings of the 2006 international conference on Empirical software engineering issues: critical assessment and future directions*, pages 10–19, Berlin, Heidelberg, 2007. Springer-Verlag.
- [131] P. Hudak. Modular Domain-specific Languages and Tools. In *Proceedings of Fifth International Conference on Software Reuse*, pages 134–142. IEEE, IEEE Computer Society Press, 1998.
- [132] Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2007. ISBN 0521692695.
- [133] ISO. *ISO/IEC DIS 9075-10: Information Technology — Database Languages — SQL — Part 10: Object Language Bindings (SQL/OLB)*. International Organization for Standardization, Geneva, Switzerland.
- [134] ISO. The ANSI C Standard (C99). Technical report, ISO/IEC, 1999.
- [135] ISO. *ISO/IEC 9075-1:1999: Information Technology — Database Languages — SQL — Part 1: Framework (SQL/Framework)*. International Organization for Standardization, Geneva, Switzerland, 1999.
- [136] ISO. *ISO/IEC 25010: 2011, Systems and Software Engineering-systems and Software Quality Requirements and Evaluation (SQuaRE)-system and Software Quality Models*, 2011.
- [137] Kathleen Jensen and Niklaus Wirth. *PASCAL User Manual and Report*. Springer-Verlag New York, Inc., New York, NY, USA, 1974. ISBN 3-540-06950-X.
- [138] Yue Jiang, Bojan Cuki, Tim Menzies, and Nick Bartlow. Comparing Design and Code Metrics for Software Quality Prediction. In *PROMISE '08: Proceedings of the 4th international workshop on Predictor models in software engineering*, pages 11–18, New York, NY, USA, 2008. ACM. ISBN 978-1-60558-036-4.
- [139] Stephen C. Johnson and Michael E. Lesk. Language Development Tools. *Bell System Technical Journal*, 56(6):2155–2176, 1987.
- [140] Steven C. Johnson. Yacc: Yet Another Compiler Compiler. In *UNIX Programmer's Manual*, volume 2, pages 353–387. Holt, Rinehart, and Winston, New York, NY, USA, 1979.
- [141] Simon Peyton Jones, Alan Blackwell, and Margaret Burnett. A User-centred Approach to Functions in Excel. *SIGPLAN Notices*, 38(9):165–176, 2003.
- [142] Stephen H. Kan. *Metrics and Models in Software Quality Engineering*. Addison-Wesley, Boston, MA, second edition, 2002. ISBN 0201729156.
- [143] Yiannis Kanellopoulos, Thimios Dimopoulos, Christos Tjortjis, and Christos Makris. Mining Source Code Elements for Comprehending Object-oriented Systems and Evaluating their Maintainability. *SIGKDD Exploring Newsletter*, 8(1):33–40, 2006. ISSN 1931-0145.

- [144] Vassilios Karakoidas. On Domain-specific Languages Usage (why DSLs really matter). *XRDS: Crossroads, The ACM Magazine for Students*, 20(3):16–17, March 2014. ISSN 1528-4972.
- [145] Vassilios Karakoidas and Diomidis Spinellis. FIRE/J: Optimizing Regular Expression Searches with Generative Programming. *Software: Practice and Experience*, 38(6):557–573, May 2008. doi: 10.1002/spe.841.
- [146] Vassilios Karakoidas and Diomidis Spinellis. J%: Integrating Domain-specific Languages with Java. In Vassilios Chrissikopoulos, Nikolaos Alexandris, Christos Douligeris, and Spyros Sioutas, editors, *PCI 2009: Proceedings of 13th Panhellenic Conference on Informatics*, pages 109–113. IEEE Computer Society, 2009.
- [147] Vassilios Karakoidas, Dimitris Mitropoulos, Georgios Gousios, Diomidis Spinellis, and Panos Louridas. Generating the blueprints of the Java ecosystem. In *MSR '15: Proceedings of the 2015 International Working Conference on Mining Software Repositories*. IEEE Computer Society, 2015. To appear.
- [148] Lennart C. L. Kats, Martin Bravenboer, and Eelco Visser. Mixing Source and Bytecode. A Case for Compilation by Normalization. In G. Kiczales, editor, *Proceedings of the 23rd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2008)*, pages 91–108, New York, NY, USA, October 2008. ACM. ISBN 978-1-60558-215-3.
- [149] Lennart C. L. Kats, Eelco Visser, and Guido Wachsmuth. Pure and Declarative Syntax Definition: Paradise Lost and Regained. In *Proceedings of Onward! 2010*. ACM, 2010.
- [150] Lennart C.L. Kats, Rob Vermaas, and Eelco Visser. Testing Domain-specific Languages. In *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*, SPLASH '11, pages 25–26, New York, NY, USA, 2011. ACM. ISBN 978-1-4503-0942-4.
- [151] Rudolf K. Keller, Reinhard Schauer, and Alistair Cockburn. Object-oriented Design Quality. In *OOPSLA '97: Addendum to the 1997 ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (Addendum)*, pages 63–67, New York, NY, USA, 1997. ACM. ISBN 1-58113-037-6.
- [152] Christian Kirkegaard, Anders Moeller, and Michael I. Schwartzbach. Static Analysis of XML Transformations in Java. *IEEE Transactions on Software Engineering*, 30(3):181–192, March 2004.
- [153] Stephen C. Kleene. *Representations of Events in Nerve Nets and Finite Automata*, volume 34 of *Annals of Mathematics Studies*, pages 3–42. Princeton University Press, Princeton, NJ, 1956.
- [154] Akif Günes Koru, Dongsong Zhang, Khaled El Emam, and Hongfang Liu. An investigation into the functional form of the size-defect relationship for software modules. *IEEE Transactions on Software Engineering*, 35(2):293–304, March 2009. ISSN 0098-5589.
- [155] T. Kos, T. Kosar, J. Knez, and M. Mernik. From DCOM Interfaces to Domain-specific Modeling Language: A Case Study on the Sequencer. *Computer Science and Information Systems*, 8(2):361–378, 2011.

- [156] Tomavz Kosar, Nuno Oliveira, Marjan Mernik, Maria J. V. Pereira, Matej Crepinsek, Daniela da Cruz, and Pedro R. Henriques. Comparing General-purpose and Domain-specific Languages: An Empirical Study. *Computer Science and Information Systems*, 7(2):247–264, May 2010.
- [157] Tomaz Kosar, Nuno Oliveira, Marjan Mernik, Maria J. V. Pereira, Matej Crepinsek, Daniela da Cruz, and Pedro R. Henriques. Comparing General-purpose and Domain-specific Languages: An Empirical Study. *Computer Science and Information Systems*, 7(2):247–264, May 2010.
- [158] Philipp W. Kutter. *MONTAGES: Engineering of Computer Languages*. PhD thesis, Swiss Federal Institute of Technology, Zurich, January 2004.
- [159] Linda Laird. *Software Measurement and Estimation: A Practical Approach*. John Wiley & Sons Inc, 2006.
- [160] Craig Larman. *Applying UML and Patterns: An Introduction to Object-oriented Analysis and Design and the Unified Process (2nd Edition)*. Prentice Hall PTR, 2 edition, 7 2001. ISBN 9780130925695.
- [161] Byeongcheol Lee, Martin Hirzel, Robert Grimm, and Kathryn S. McKinley. Debug All Your Code: Portable Mixed-environment Debugging. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications, OOPSLA '09*, pages 207–226, New York, NY, USA, 2009. ACM. ISBN 978-1-60558-766-0.
- [162] Byeongcheol Lee, Robert Grimm, Martin Hirzel, and Kathryn McKinley. Marco: Safe, Expressive Macros for Any Language. In James Noble, editor, *European Conference on Object-Oriented Programming (ECOOP 2012)*, volume 7313 of *Lecture Notes in Computer Science*, pages 589–613. Springer Berlin Heidelberg, 2012. ISBN 978-3-642-31056-0.
- [163] Sin Yeung Lee, Wai Lup Low, and Pei Yuen Wong. Learning Fingerprints for a Database Intrusion Detection System. In Dieter Gollmann, Günter Karjoth, and Michael Waidner, editors, *ESORICS '02: Proceedings of the 7th European Symposium on Research in Computer Security*, pages 264–280, London, UK, 2002. Springer-Verlag. ISBN 3-540-44345-2.
- [164] Daan Leijen and Erik Meijer. Domain-specific Embedded Compilers. In *PLAN '99: Proceedings of the 2nd conference on Domain-specific languages*, pages 109–122. ACM Press, 1999. ISBN 1-58113-255-7.
- [165] Ernst Leiss. Constructing a Finite Automaton for a Given Regular Expression. *ACM SIGACT News*, 12(3):81–87, 1980. ISSN 0163-5700.
- [166] Yuri Leontiev, M. Tamer Özsu, and Duane Szafron. On Type Systems for Object-oriented Database Programming Languages. *ACM Computing Surveys*, 34(4):409–449, 2002. ISSN 0360-0300.
- [167] Michael E. Lesk. Lex—A Lexical Analyzer Generator. Computer Science Technical Report 39, Bell Laboratories, Murray Hill, NJ, October 1975.
- [168] Wei Li and Sallie Henry. Object-oriented Metrics that Predict Maintainability. *Journal of Systems and Software*, 23(2):111–122, 1993. ISSN 0164-1212.
- [169] Tim Lindhorn and Frank Yellin. *The Java Virtual Machine Specification*. The Java Series. Addison-Wesley, Reading, MA, 2nd edition, 2003. ISBN 0-201-63452-X.

- [170] J. Linn. The Kerberos Version 5 GSS-API Mechanism (RFC1964)), 1996.
- [171] John W. Lloyd. Directions for Meta-Programming. In *In Proceedings of Fifth Generation Computer Systems (FGCS88)*, pages 609–617, 1988.
- [172] Jonathan Locke. Jakarta Regexp Library, July 2004. URL <http://jakarta.apache.org/regexp/>.
- [173] Christos K. K. Loverdos and Apostolos Syropoulos. *Steps in Scala: An Introduction to Object-functional Programming*. Cambridge University Press, 2010. ISBN 0521762170.
- [174] G. Low and S. Huan. Impact of Object-oriented Development on Software Quality. In *STEP '99: Proceedings of the Software Technology and Engineering Practice*, page 3, Washington, DC, USA, 1999. IEEE Computer Society. ISBN 0-7695-0328-4.
- [175] Mark Lutz and David Ascher. *Learning Python*. O'Reilly, Sebastopol, CA 95472, USA, 2nd edition, December 2003.
- [176] Association For Computing Machinery. The 2012 ACM Computing Classification System, 2012. URL <http://www.acm.org/about/class>.
- [177] Geoffrey Mainland. Why it's Nice to be Quoted: Quasiquoting for Haskell. In *Proceedings of the ACM SIGPLAN workshop on Haskell workshop*, Haskell '07, pages 73–82, New York, NY, USA, 2007. ACM. ISBN 978-1-59593-674-5.
- [178] Daisuke Maki and Hideya Iwasaki. A Portable Javascript Thread Library for Ajax Applications. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, OOPSLA '07, pages 817–818, New York, NY, USA, 2007. ACM. ISBN 978-1-59593-865-7.
- [179] Per. Martin-Lof. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [180] Tobias Mayer and Tracy Hall. A Critical Analysis of Current Object-oriented Design Metrics. *Software Quality Control*, 8(2):97–110, 1999. ISSN 0963-9314.
- [181] Lawrence McAfee and Kunle Olukotun. Utilizing Static Analysis and Code Generation to Accelerate Neural Networks. In *Proceedings of the 29th International Conference on Machine Learning (ICML 2012)*, ICML '12, 2012.
- [182] Russell A. McClure and Ingolf H. Krüger. SQL DOM: Compile-time Checking of Dynamic SQL Statements. In *ICSE '05: Proceedings of the 27th international conference on Software engineering*, pages 88–96, 2005. ISBN 1-59593-963-2.
- [183] Steve C. McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, Redmond, WA, second edition, 2004. ISBN 0735619670.
- [184] Warren S. McCulloch and Walter Pitts. A Logical Calculus of the Ideas Immanent in Nervous Activity. 5:18–27, 1943.
- [185] James R. McGraw. The VAL Language: Description and Analysis. *ACM Transactions on Programming Languages Systems*, 4(1):44–82, 1982. ISSN 0164-0925.
- [186] Erik Meijer, Brian Beckman, and Gavin Bierman. LINQ: Reconciling Object, Relations and XML in the .NET Framework. In *SIGMOD '06: Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, page 706, New York, NY, USA, 2006. ACM Press. ISBN 1-59593-434-0.

- [187] Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and How to Develop Domain-specific Languages. *ACM Computing Surveys*, 37(4):316–344, 2005. ISSN 0360-0300.
- [188] Subhas Chandra Misra. Modeling Design/Coding Factors that Drive Maintainability of Software Systems. *Software Quality Control*, 13(3):297–320, 2005. ISSN 0963-9314.
- [189] Dimitris Mitropoulos and Diomidis Spinellis. SDriver: Location-specific Signatures Prevent SQL Injection Attacks. *Computers and Security*, pages 121–129, May/June 2009. ISSN 0167-4048.
- [190] Anders Moeller. Automaton Library Javadocs, September 2011. URL <http://www.brics.dk/automaton/>.
- [191] Lloyd H. Nakatani, Mark A. Ardis, Robert G. Olsen, and Paul M. Pontrelli. Jargons for Domain Engineering. *SIGPLAN Notices*, 35(1):15–24, 2000. ISSN 0362-1340.
- [192] NASA. Software Quality Metrics for Object-oriented System Environments. Technical report, National Aeronautics and Space Administration, Goddard Space Flight Center, Greenbelt Maryland 20771, June 1995.
- [193] G. Navarro. NR-Grep: A Fast and Flexible Pattern-Matching Tool. *Software: Practice & Experience*, 31(13):1265–1312, 2001. ISSN 0038-0644.
- [194] G. Navarro. Regular Expression Searching on Compressed Text. *Journal of Discrete Algorithms*, 1:423–443, 2003. ISSN 1570-8667.
- [195] G. Navarro and M. Raffinot. New Techniques for Regular Expression Searching. *Algorithmica*, 41(2):89–116, 2005.
- [196] Nicholas Nguyen. SPath: An Extensible Query-language for Scala. *Journal of Object Technology*, pages 1–37, 2011.
- [197] Hane Riis Nielson and Flemming Nielson. *Semantics with Applications: An Appetizer*. Springer, 2007. ISBN 2006939147.
- [198] Tobias Nipkow and David von Oheimb. Java is Type Safe—definitely. In *POPL '98: Proceedings of the 25th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 161–170, New York, NY, USA, 1998. ACM. ISBN 0-89791-979-3.
- [199] Nathaniel Nystrom, Michael R. Clarkson, and Andrew C. Myers. Polyglot: An Extensible Compiler Framework for Java. In *Proceedings of 12th International Conference on Compiler Construction (LNCS 2622)*, pages 138—152. Springer, April 2003.
- [200] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in Scala: A Comprehensive Step-by-step Guide*. Artima Inc, 2008. ISBN 0981531601.
- [201] Hector M. Olague, Letha H. Etzkorn, Sherri L. Messimer, and Harry S. Delugach. An Empirical Validation of Object-oriented Class Complexity Metrics and their Ability to Predict Error-prone Classes in Highly Iterative, or Agile, Software: A Case Study. *Journal of Software Maintenance and Evolution: Research and Practice*, 20(3):171–197, 2008.
- [202] Dereck C. Oppen. Prettyprinting. *ACM Transactions on Programming Languages and Systems*, 2:465–483, October 1980. ISSN 0164-0925.

- [203] Dominic Orchard. The Four Rs of Programming Language Design. In *Proceedings of the 10th SIGPLAN symposium on New ideas, new paradigms, and reflections on programming and software*, ONWARD '11, pages 157–162, New York, NY, USA, 2011. ACM. ISBN 978-1-4503-0941-7.
- [204] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN 0262162091.
- [205] John Placer. Multiparadigm Research: A New Direction of Language Design. *SIGPLAN Notices*, 26(3):9–17, 1991. ISSN 0362-1340.
- [206] P.J. Plauger, Alexander Stepanov, Meng Lee, and David R. Musser. *The C++ Standard Template Library*. Prentice Hall, 2000. ISBN 0134376331.
- [207] George Pólya. *How to Solve It*. Princeton University Press, 1945. ISBN 0-691-08097-6.
- [208] Olli-Pekka Puolitaival and Teemu Kanstrén. Towards Flexible and Efficient Model-based Testing, Utilizing Domain-specific Modelling. In *Proceedings of the 10th Workshop on Domain-Specific Modeling*, DSM '10, pages 1–6, New York, NY, USA, 2010. ACM. ISBN 978-1-4503-0549-5.
- [209] Sandeep Purao and Vijay Vaishnavi. Product Metrics for Object-oriented Systems. *ACM Computing Surveys*, 35(2):191–221, 2003. ISSN 0360-0300.
- [210] Venkataraman Ramesh, Robert L. Glass, and Iris Vessey. Research in Computer Science: An Empirical Study. *Journal of Systems and Software*, pages 165–176, 2004.
- [211] Frank P. Ramsey. The Foundations of Mathematics. *Proceedings of the London Mathematical Society*, 25(5):338–384, 1925.
- [212] John Reid. The New Features of Fortran 2008. *SIGPLAN Fortran Forum*, 27(2):8–21, August 2008. ISSN 1061-7264.
- [213] L. Renggli, M. Denker, and O. Nierstrasz. Language Boxes: Bending the Host Language with Modular Language Changes. In *Software Language Engineering: Second International Conference, SLE 2009*, pages 274–293. Springer, LNCS, 2009.
- [214] Lukas Renggli, Tudor Girba, and Oscar Nierstrasz. Embedding Languages without Breaking Tools. In *In ECOOP 2010: Proceedings of the 24th European Conference on Object-Oriented Programming*, 2010.
- [215] Morten Rhiger. A Foundation for Embedded Languages. *ACM Transactions on Programming Languages and Systems*, 25(3):291–315, 2003. ISSN 0164-0925.
- [216] Jonathan Riehl. Assimilating MetaBorg: Embedding Language Tools in Languages. In *GPCE'06*, 2006.
- [217] Version Rishiyur, Rishiyur S. Nikhil, and Rishiyur S. Nikhil. ID Language Reference Manual, 1991.
- [218] Tiark Rompf and Martin Odersky. Lightweight Modular Staging: A Pragmatic Approach to Runtime Code Generation and Compiled DSLs. In *Proceedings of the ninth international conference on Generative programming and component engineering*, GPCE '10, pages 127–136, New York, NY, USA, 2010. ACM. ISBN 978-1-4503-0154-1.

- [219] Douglas T. Ross. Origins of the APT Language for Automatically Programmed Tools. *SIG-PLAN Notices*, 13(8):61–99, August 1978. ISSN 0362-1340.
- [220] D.T. Ross. Origins of the APT Language for Automatically Programmed Tools. In R.L. Wexelblat, editor, *History of Programming Languages*, pages 279–338. Academic Press, 1981.
- [221] Peter Salus. *Handbook of Programming Languages: Object-oriented Programming Languages (Vol. 1)*. Macmillan Technical Pub, 1998.
- [222] Peter Salus. *Handbook of Programming Languages: Imperative Programming Languages (Vol. 2)*. Macmillan Technical Pub, 1998.
- [223] Peter Salus. *Handbook of Programming Languages: Little Languages and Tools (Vol. 3)*. Macmillan Technical Pub, 1998.
- [224] Peter Salus. *Handbook of Programming Languages: Functional and Logic Programming Languages (Vol. 4)*. Macmillan Technical Pub, 1998.
- [225] Daniel F. Savarese. Jakarta ORO, July 2004. URL <http://jakarta.apache.org/oro/>.
- [226] Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann Publishers, 1999.
- [227] Dietmar Seipel, Andreas M. Boehm, and Markus Fröhlich. JSquash: Source Code Analysis of Embedded Database Applications for Determining SQL Statements. In *Proceedings of the 18th International Conference on Applications of Declarative Programming and Knowledge Management*, INAP’09, pages 153–169, Berlin, Heidelberg, 2011. Springer-Verlag. ISBN 978-3-642-20588-0.
- [228] Dietmar Seipel, Andreas M. Boehm, and Markus Fröhlich. JSquash: Source Code Analysis of Embedded Database Applications for Determining SQL Statements. In *Proceedings of the 18th International Conference on Applications of Declarative Programming and Knowledge Management*, INAP’09, pages 153–169, Berlin, Heidelberg, 2011. Springer-Verlag. ISBN 978-3-642-20588-0.
- [229] Tim Sheard, Zine el-abidine Benaissa, and Emir Pasalic. DSL Implementation using Staging and Monads. In *PLAN ’99: Proceedings of the 2nd conference on Domain-specific languages*, pages 81–94. ACM, 1999. ISBN 1-58113-255-7.
- [230] Gary B. Shelly and Thomas J. Cashman, editors. *Basic for the IBM PC*. Boyd & Fraser Publishing Co., Danvers, MA, USA, 1988. ISBN 0-87835-212-0.
- [231] Jack Shirazi. *Java Performance Tuning*. O’Reilly and Associates, Inc., Sebastopol, CA, 2003. ISBN 0-596-00377-3.
- [232] Yannis Smaragdakis, Shan Huang, and David Zook. Program Generators and the Tools to Make Them. In *PEPM ’04: Proceedings of the 2004 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*, pages 92–100, New York, NY, USA, 2004. ACM. ISBN 1-58113-835-0.
- [233] Maciej Sobczak, Stephen Hutton, Vadim Zeitlin, and Mateusz Loskot. *SOCI - The C++ Database Access Library*, Aug 2013.
- [234] Ian Sommerville. *Software Engineering*. Pearson, Boston, 2011. ISBN 9780137053469.

- [235] Henry Spencer. regex - Henry Spencer's Regular Expression Libraries, January 2007. URL <http://gnuwin32.sourceforge.net/packages/regex-spencer.htm>.
- [236] Diomidis Spinellis. *Programming Paradigms as Object Classes: A Structuring Mechanism for Multiparadigm Programming*. PhD thesis, Imperial College, London, UK, February 1994.
- [237] Diomidis Spinellis. Reliable Software Implementation using Domain-specific Languages. In G. I. Schuëller and P. Kafka, editors, *Proceedings ESREL '99 — The Tenth European Conference on Safety and Reliability*, pages 627–631, Rotterdam, 1999. ESRA, VDI, TUM, A. A. Balkema.
- [238] Diomidis Spinellis. Notable Design Patterns for Domain-specific Languages. *Journal of Systems and Software*, 56(1):91–99, February 2001.
- [239] Diomidis Spinellis. Book Review: Building Internet Firewalls. *ACM Computing Reviews*, pages 155–156, 2001.
- [240] Diomidis Spinellis. *Code Reading: The Open Source Perspective*. Addison-Wesley Professional, 6 2003. ISBN 9780201799408.
- [241] Diomidis Spinellis. *Code Quality: The Open Source Perspective*. Addison-Wesley, Boston, MA, 2006.
- [242] Diomidis Spinellis and V. Guruprasad. Lightweight Languages as Software Engineering Tools. In *USENIX Conference on Domain-Specific Languages*, pages 67–76, Berkeley, CA, 1997. USENIX Association.
- [243] Diomidis Spinellis, Georgios Gousios, Vassilios Karakoidas, Panagiotis Louridas, Paul J. Adams, Ioannis Samoladas, and Ioannis Stamelos. Evaluating the Quality of Open Source Software. In *SQM 2008: Second International Workshop on Software Quality and Maintainability—12th European Conference on Software Maintenance and Reengineering (CSMR 2008) satellite event*, pages 5–28. The Reengineering Forum, 2008.
- [244] Diomidis Spinellis, Georgios Gousios, Vassilios Karakoidas, Panagiotis Louridas, Paul J. Adams, Ioannis Samoladas, and Ioannis Stamelos. Evaluating the Quality of Open Source Software. *Electronic Notes in Theoretical Computer Science*, 233:5–28, March 2009.
- [245] Knut Stolze. Bringing the Power of Regular Expression Matching to SQL, January 2003.
- [246] Konstantinos Stroggylos and Diomidis Spinellis. Refactoring: Does it Improve Software Quality? In B. Boehm, S. Chulani, J. Verner, and B. Wong, editors, *5th International Workshop on Software Quality*, pages 1–6. ACM Press, 2007.
- [247] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, Reading, MA, third edition, 1997. ISBN 0201889544.
- [248] Don Syme. Proving Java Type Soundness. In *Formal Syntax and Semantics of Java*, pages 83–118, London, UK, 1999. Springer-Verlag. ISBN 3-540-66158-1.
- [249] S. Thibault. *Domain-specific Languages. Conception, Implementation and Application*. PhD thesis, University of Rennes, France, 1998.
- [250] Ken Thompson. Regular Expression Search Algorithm. *Communications of the ACM*, 11(6): 419–422, June 1968.

- [251] Sam Tobin-Hochstadt and Matthias Felleisen. Interlanguage Migration: From Scripts to Programs. In *OOPSLA '06: Companion to the 21st ACM SIGPLAN Conference on Object-oriented Programming Systems, Languages, and Applications*, pages 964–974, New York, NY, USA, 2006. ACM Press. ISBN 1-59593-491-X.
- [252] Laurence Tratt. Domain-specific Language Implementation via Compile-time Metaprogramming. *ACM Transactions on Programming Languages and Systems*, 30(6):1–40, 2008. ISSN 0164-0925.
- [253] Laurence Tratt. Domain-specific Language Implementation via Compile-Time Metaprogramming. *ACM Transactions of Programming Languages and Systems*, 2008.
- [254] Salvador Trujillo, Maider Azanza, and Oscar Diaz. Generative Metaprogramming. In *GPCE '07: Proceedings of the 6th international conference on Generative programming and component engineering*, pages 105–114, New York, NY, USA, 2007. ACM. ISBN 978-1-59593-855-8.
- [255] Nikolaos Tsantalis, Alexander Chatzigeorgiou, and George Stephanides. Predicting the Probability of Change in Object-oriented Systems. *IEEE Transactions on Software Engineering*, 31(7):601–614, 2005. ISSN 0098-5589.
- [256] Jeffrey D. Ullman. *Elements of ML Programming, ML97 Edition (2nd edition)*. Prentice Hall, 1998. ISBN 0137903871.
- [257] Arie van Deursen and Paul Klint. Little Languages: Little Maintenance. *Journal of Software Maintenance*, 10(2):75–92, 1998. ISSN 1040-550X.
- [258] Arie van Deursen, Paul Klint, and Joost Visser. Domain-specific Languages: An Annotated Bibliography. *SIGPLAN Notices*, 35(6):26–36, 2000.
- [259] Jonne van Wijngaarden. Code Generation from a Domain-specific Language. Designing and Implementing Complex Program Transformations. Master's thesis, Utrecht University, Utrecht, The Netherlands, July 2003.
- [260] Todd L. Veldhuizen. Guaranteed Optimization for Domain-specific Programming. In *Domain-Specific Program Generation*, volume 3016, pages 306–324. Springer-Verlag, 2004.
- [261] Todd L. Veldhuizen and Dennis Gannon. Active Libraries: Rethinking the Roles of Compilers and Libraries. In *In Proceedings of the SIAM Workshop on Object Oriented Methods for Inter-operable Scientific and Engineering Computing (OO'98)*. SIAM Press, 1998.
- [262] Sander D. Vermolen and Eelco Visser. Heterogeneous Coupled Evolution of Software Languages. In K. Czarnecki, I. Ober, J.-M. Bruel, A. Uhl, and M. Völter, editors, *Proceedings of the 11th International Conference on Model Driven Engineering Languages and Systems (MODELS 2008)*, volume 5301, pages 630–644, Heidelberg, September 2008. Springer. ISBN 978-3-540-87874-2.
- [263] Eelco Visser. Stratego: A Language for Program Transformation-based on Rewriting Strategies. In A. Middeldorp, editor, *Rewriting Techniques and Applications (RTA 2001)*, pages 357–361. Springer-Verlag, May 2001.
- [264] Eelco Visser. A Survey of Rewriting Strategies in Program Transformation Systems. In B. Gramlich and S. Lucas, editors, *Workshop on Reduction Strategies in Rewriting and Programming (WRS 2001)*, volume 57, Utrecht, The Netherlands, May 2001. Elsevier Science Publishers.

- [265] Eelco Visser. Meta-Programming with Concrete Object Syntax. In Don Batory, Charles Consel, and Walid Taha, editors, *Generative Programming and Component Engineering (GPCE 2002)*, pages 299–315, Pittsburgh, PA, USA, October 2002. Springer-Verlag.
- [266] Eelco Visser. Program transformation with stratego/xt: Rules, strategies, tools, and systems in strategoxt-0.9. In C. Lengauer et al., editors, *Domain-specific Program Generation*, volume 3016, pages 216–238. Springer-Verlag, June 2004.
- [267] Eelco Visser. A Survey of Strategies in Rule-based Program Transformation Systems. *Journal of Symbolic Computation*, pages 831–873, 2005.
- [268] Eelco Visser. WebDSL: A Case Study in Domain-specific Language Engineering. In R. Lämmel, J. Visser, and J. Saraiva, editors, *International Summer School on Generative and Transformational Techniques in Software Engineering (GTTSE 2007)*, volume 5235, pages 291–373, Heidelberg, October 2008. Springer. ISBN 978-3-540-88642-6.
- [269] Markus Voelter. *DSL Engineering: Designing, Implementing and Using Domain-specific Languages*. CreateSpace Independent Publishing Platform, 2013. ISBN 1481218581.
- [270] Jan Christopher Vogt. *Type Safe Integration of Query Languages into Scala*. PhD thesis, RWTH Aachen University, August 2011.
- [271] D. G. Waddington and B. Yao. High Fidelity C++ Code Transformation. In *Proceedings of the 5th workshop on Language Descriptions, Tools and Applications (LDTA 2005)*, Edinburgh University, UK, April 2005.
- [272] Larry Wall, Tom Christiansen, and Jon Orwant. *Programming Perl*. O'Reilly, Sebastopol, CA, 2000. ISBN 0-596-00027-8.
- [273] Larry Wall, Tom Christiansen, and Jon Orwant. *Programming Perl*. O'Reilly, Sebastopol, CA, 2000. ISBN 0-596-00027-8.
- [274] Gary Wassermann, Carl Gould, Zhendong Su, and Premkumar Devanbu. Static Checking of Dynamically Generated Queries in Database Applications. *ACM Transactions of Software Engineering and Methodology*, 16, September 2007. ISSN 1049-331X.
- [275] Peter Wegner. Research Paradigms in Computer Science. In *ICSE '76: Proceedings of the 2nd international conference on Software engineering*, pages 322–330, Los Alamitos, CA, USA, 1976. IEEE Computer Society Press.
- [276] Peter Wegner. Dimensions of Object-based Language Design. *SIGPLAN Notices*, 22(12): 168–182, 1987. ISSN 0362-1340.
- [277] Hedzer Westra. Configurable Transformations for High-quality Automatic Program Improvement. Cobolx: A Case Study. Master's thesis, Utrecht University, Utrecht, The Netherlands, February 2002.
- [278] E. J. Weyuker. Evaluating Software Complexity Measures. *IEEE Transactions on Software Engineering*, 14(9):1357–1365, 1988. ISSN 0098-5589.
- [279] Alfred North Whitehead and Bertrand Russel. *Principia Mathematica*. Cambridge University Press, 1910.
- [280] Wikipedia. Convention over Configuration, May 2011. URL http://en.wikipedia.org/wiki/Convention_over_configuration.

- [281] Niklaus Wirth. *Compiler Construction*. Addison-Wesley, 1996. ISBN 0201403536.
- [282] Sun Wu and Udi Manber. Fast Text Searching: Allowing Errors. *Communications of the ACM*, 35(10):83–91, 1992. ISSN 0001-0782.
- [283] M. Xenos, D. Stavrinoudis, K. Zikouli, and D. Christodoulakis. Object-oriented Metrics — A Survey. In *Proceedings of the FESMA 2000*. Federation of European Software Measurement Associations, 2000.
- [284] R. Yousefi, A. Jalili, and S. M. Fakhraie. Frequency Domain Testing: A New Approach in Online Test of VLSI Digital Signal Processing Systems. In *Proceedings of the 2009 International Conference on Computer Engineering and Technology - Volume 02*, ICCET '09, pages 136–140, Washington, DC, USA, 2009. IEEE Computer Society. ISBN 978-0-7695-3521-0.
- [285] Pamela Zave. A Compositional Approach to Multiparadigm Programming. *IEEE Software*, 6(5):15–25, 1989. ISSN 0740-7459.
- [286] Marvin V. Zelkowitz and Dolores R. Wallace. Experimental Models for Validating Technology. *Computer*, 31:23–31, May 1998. ISSN 0018-9162.
- [287] Tian Zhao. *Type Matching and Type Inference for Object-oriented Systems*. PhD thesis, Purdue University, West Lafayette, IN, USA, 2002.